

Gentoo Linux amd64 Handbook: Installing Gentoo

From Gentoo Wiki

< Handbook:AMD64 (/wiki/Special:MyLanguage/Handbook:AMD64) | Full (/wiki/Special:MyLanguage/Handbook:AMD64/Full)

Jump to:navigation Jump to:search

Contents

- 1 Introduction
 - 1.1 Welcome
 - 1.1.1 Openness
 - 1.1.2 Choice
 - 1.1.3 Power
 - 1.2 How the installation is structured
 - 1.2.1 Deciding which steps to take
 - 1.2.2 Suggested steps
 - 1.2.3 Optional steps
 - 1.2.4 Deprecated steps
 - 1.2.5 Defaults and alternatives
 - 1.3 Installation options for Gentoo
 - 1.4 Troubles
- 2 Hardware requirements
- 3 Gentoo Linux installation media
 - 3.1 Minimal installation CD
 - 3.2 The Gentoo LiveGUI
 - 3.3 What are stage files?
- 4 Downloading
 - 4.1 Obtain the media
 - 4.1.1 Navigating Gentoo mirrors
 - 4.2 Verifying the downloaded files
 - 4.2.1 Microsoft Windows-based verification
 - 4.2.2 Linux based verification
- 5 Writing the boot media
 - 5.1 Writing a bootable USB
 - 5.1.1 Writing with Linux
 - 5.1.1.1 Determining the USB device path
 - 5.1.1.2 Writing with dd
 - 5.1.2 Writing with Windows
 - 5.2 Burning a disk
 - 5.2.1 Burning with Microsoft Windows 7 and above

- 5.2.2 Burning with Linux
- 6 Booting
 - 6.1 Booting the installation media
 - 6.1.1 Kernel choices
 - 6.1.2 Hardware options
 - 6.1.3 Logical volume/device management
 - 6.1.4 Other options
 - 6.2 LiveGUI root access
 - 6.3 Extra hardware configuration
 - 6.4 Optional: User accounts
 - 6.5 Optional: Viewing documentation while installing
 - 6.5.1 TTYs
 - 6.5.2 GNU Screen
 - 6.6 Optional: Starting the SSH daemon
- 7 Automatic network configuration
 - 7.1 Using DHCP
 - 7.2 Testing the network
- 8 Obtaining interface info
- 9 Wireless Setup
 - 9.1 Optional: Checking wireless
 - 9.2 Using NetworkManager
 - 9.3 Using net-setup
 - 9.4 Manual setup
 - 9.4.1 Manually connecting to an open network
- 10 Application specific configuration
 - 10.1 Configure web proxies
 - 10.2 Using pppoe-setup for ADSL
 - 10.3 Using PPTP
- 11 Internet and IP basics
 - 11.1 Interfaces and addresses
 - 11.2 Networks and CIDR
 - 11.3 The Internet
 - 11.4 The Domain Name System
- 12 Manual and static IP network configuration
 - 12.1 Interface address configuration
 - 12.2 Default route configuration
 - 12.3 DNS configuration
- 13 Introduction to block devices
 - 13.1 Block devices
 - 13.2 Partition tables
 - 13.2.1 GUID Partition Table (GPT)
 - 13.2.2 Master boot record (MBR) or DOS boot sector
 - 13.3 Advanced storage
 - 13.4 Default partitioning scheme
- 14 Designing a partition scheme
 - 14.1 How many partitions and how big?
 - 14.2 What about swap space?
 - 14.3 What is the EFI System Partition (ESP)?

- 14.4 What is the BIOS boot partition?
- 15 Partitioning the disk with GPT for UEFI
 - 15.1 Viewing the current partition layout
 - 15.2 Creating a new disklabel / removing all partitions
 - 15.3 Creating the EFI System Partition (ESP)
 - 15.4 Creating the swap partition
 - 15.5 Creating the root partition
 - 15.6 Saving the partition layout
- 16 Partitioning the disk with MBR for BIOS / legacy boot
 - 16.1 Viewing the current partition layout
 - 16.2 Creating a new disklabel / removing all partitions
 - 16.3 Creating the boot partition
 - 16.4 Creating the swap partition
 - 16.5 Creating the root partition
 - 16.6 Saving the partition layout
- 17 Creating file systems
 - 17.1 Introduction
 - 17.2 Filesystems
 - 17.3 Applying a filesystem to a partition
 - 17.3.1 EFI system partition filesystem
 - 17.3.2 Legacy BIOS boot partition filesystem
 - 17.3.3 Small ext4 partitions
 - 17.4 Activating the swap partition
- 18 Mounting the root partition
- 19 Choosing a stage file
 - 19.1 OpenRC
 - 19.2 systemd
 - 19.3 Multilib (32 and 64-bit)
 - 19.4 No-multilib (pure 64-bit)
- 20 Downloading the stage file
 - 20.1 Setting the date and time
 - 20.1.1 Automatic
 - 20.1.2 Manual
 - 20.2 Graphical browsers
 - 20.3 Command-line browsers
 - 20.4 Verifying and validating
- 21 Installing a stage file
- 22 Configuring compile options
 - 22.1 Introduction
 - 22.2 CFLAGS and CXXFLAGS
 - 22.3 RUSTFLAGS
 - 22.4 MAKEOPTS
 - 22.5 Ready, set, go!
- 23 References
- 24 Chrooting
 - 24.1 Copy DNS info
 - 24.2 Mounting the necessary filesystems
 - 24.3 Entering the new environment

- 24.4 Preparing for a bootloader
 - 24.4.1 UEFI systems
 - 24.4.2 DOS/Legacy BIOS systems
- 25 Configuring Portage
 - 25.1 Installing a Gentoo ebuild repository snapshot from the web
 - 25.2 Optional: Selecting mirrors
 - 25.2.1 Optional: rsync mirrors
 - 25.3 Optional: Updating the Gentoo ebuild repository
 - 25.4 Reading news items
 - 25.5 Choosing the right profile
 - 25.5.1 No-multilib
 - 25.6 Optional: Adding a binary package host
 - 25.6.1 Repository configuration
 - 25.6.2 Installing binary packages
 - 25.7 Optional: Configuring the USE variable
 - 25.7.1 CPU_FLAGS_*
 - 25.7.2 VIDEO_CARDS
 - 25.8 Optional: Configure the ACCEPT_LICENSE variable
 - 25.9 Optional: Updating the @world set
 - 25.9.1 Removing obsolete packages
- 26 Optional: Using systemd as the system and service manager
- 27 Timezone
- 28 Configure locales
 - 28.1 Locale generation
 - 28.2 Locale selection
- 29 References
- 30 Installing firmware and/or microcode
 - 30.1 Firmware
 - 30.1.1 Suggested: Linux Firmware
 - 30.1.1.1 Firmware Loading
 - 30.1.2 SOF Firmware
 - 30.2 Suggested: Microcode
- 31 sys-kernel/installkernel
 - 31.1 Bootloader
 - 31.1.1 GRUB
 - 31.1.2 systemd-boot
 - 31.1.3 EFI stub
 - 31.1.4 Traditional layout, other bootloaders (e.g. (e)lilo, syslinux, etc.)
 - 31.2 Initramfs
 - 31.3 Optional: Unified Kernel Image
 - 31.3.1 Generic Unified Kernel Image (systemd only)
 - 31.3.2 Secure Boot
- 32 Kernel selection
 - 32.1 Distribution kernels
 - 32.1.1 Optional: Signed kernel modules
 - 32.1.2 Optional: Signing the kernel image (Secure Boot)
 - 32.1.3 Installing a distribution kernel
 - 32.1.4 Upgrading and cleaning up

- 32.1.5 Post-install/upgrade tasks
 - 32.1.5.1 Manually rebuilding the initramfs or Unified Kernel Image
- 32.2 Alternative: Manual configuration
 - 32.2.1 Installing and Configuring the kernel sources
 - 32.2.1.1 Option 1 - Modprobed-db process
 - 32.2.1.2 Option 2 - Assisted manual process
 - 32.2.1.3 Option 3 - Configuring by hand
 - 32.2.2 Optional: Signed kernel modules
 - 32.2.3 Optional: Signing the kernel image (Secure Boot)
 - 32.2.4 Compiling and installing
- 33 Filesystem information
 - 33.1 Filesystem labels and UUIDs
 - 33.2 Partition labels and UUIDs
 - 33.3 About fstab
 - 33.4 Creating the fstab file
 - 33.4.1 DOS/Legacy BIOS systems
 - 33.4.2 UEFI systems
 - 33.4.3 DPS UEFI PARTUUID
- 34 Networking information
 - 34.1 Hostname
 - 34.1.1 Set the hostname (OpenRC or systemd)
 - 34.1.2 systemd
 - 34.2 The hosts file
 - 34.3 Network
 - 34.3.1 DHCP via dhcpcd (any init system)
 - 34.3.2 NetworkManager (any init system)
 - 34.3.3 netifrc (OpenRC)
 - 34.3.3.1 Configuring the network
 - 34.3.3.2 Automatically start networking at boot
 - 34.3.4 systemd-networkd (systemd)
 - 34.3.5 Optional: Networking tools
 - 34.3.5.1 Installing a PPPoE client
 - 34.3.5.2 Install wireless networking tools
- 35 System information
 - 35.1 Root password
 - 35.2 Init and boot configuration
 - 35.2.1 OpenRC
 - 35.2.2 systemd
- 36 System logger
 - 36.1 OpenRC
 - 36.2 systemd
- 37 Cron daemon
 - 37.1 OpenRC
 - 37.1.1 Default: cronie
 - 37.1.2 Alternative: dcron
 - 37.1.3 Alternative: fcron
 - 37.1.4 Alternative: bcron
 - 37.1.5 modprobed-db users

- 37.2 systemd
 - 37.2.1 modprobed-db users
- 38 Optional: File indexing
- 39 Optional: Remote shell access
 - 39.1 OpenRC
 - 39.2 systemd
- 40 Optional: Shell completion
 - 40.1 Bash
- 41 Suggested: Time synchronization
 - 41.1 chrony
 - 41.1.1 OpenRC
 - 41.1.2 systemd
 - 41.2 systemd-timesyncd
- 42 Filesystem tools
- 43 Selecting a boot loader
- 44 Default: GRUB
 - 44.1 Emerge
 - 44.2 Install
 - 44.2.1 DOS/Legacy BIOS systems
 - 44.2.2 UEFI systems
 - 44.2.2.1 Optional: Secure Boot
 - 44.2.2.2 Debugging GRUB
 - 44.3 Configure
- 45 Alternative 1: systemd-boot
 - 45.1 Emerge
 - 45.2 Installation
 - 45.3 Optional: Secure Boot
- 46 Alternative 2: EFI Stub
 - 46.1 Unified Kernel Image
- 47 Other Alternatives
- 48 Rebooting the system
- 49 Portage maintenance
- 50 User administration
 - 50.1 Adding a user for daily use
 - 50.2 Temporarily elevating privileges
 - 50.3 Disabling root login
- 51 Disk cleanup
 - 51.1 Removing installation artifacts
- 52 Where to go from here
 - 52.1 Additional documentation
 - 52.2 Gentoo online
 - 52.2.1 Forums and IRC
 - 52.2.2 Mailing lists
 - 52.2.3 Bugs
 - 52.2.4 Development guide
- 53 Closing thoughts

Introduction

Welcome

Welcome to Gentoo! Gentoo is a free operating system based on Linux that can be automatically optimized and customized for just about any application or need. It is built on an ecosystem of free software and does not hide what is running beneath the hood from its users.

Openness

Gentoo's premier tools are built from simple programming languages. Portage (</wiki/Portage>), Gentoo's package maintenance system, is written in Python (<https://gitweb.gentoo.org/proj/portage.git/>). Ebuilds, which provide package definitions for Portage are written in bash (<https://gitweb.gentoo.org/repo/gentoo.git>). Our users are encouraged to review, modify, and enhance the source code for all parts of Gentoo.

By default, packages are only patched when necessary to fix bugs or provide interoperability within Gentoo. They are installed to the system by compiling source code provided by upstream projects into binary format (although support for precompiled binary packages is included too). Configuring Gentoo happens through text files.

For the above reasons and others: *openness* is built-in as a *design principle*.

Choice

Choice is another Gentoo *design principle*.

When installing Gentoo, choice is made clear throughout the Handbook. System administrators can choose two fully supported init systems (Gentoo's own OpenRC (</wiki/OpenRC>) and Freedesktop.org's systemd (</wiki/Systemd>)), partition structure for storage disk(s), what file systems to use on the disk(s), a target system profile (</wiki/Profile>), remove or add features on a global (system-wide) or package specific level via USE flags, bootloader, network management utility, and much, much more.

As a development philosophy, Gentoo's authors (<https://www.gentoo.org/inside-gentoo/developers/>) try to avoid forcing users onto a specific system profile or desktop environment. If something is offered in the GNU/Linux ecosystem, it's likely available in Gentoo. If not, then we'd love to see it so. For new packages, it is recommended to first submit a package to GURU (</wiki/GURU>). Once it has matured and a Gentoo developer has volunteered to sponsor the new package, it can then be added to the official Gentoo package repository.

Power

Being a source-based operating system allows Gentoo to be ported onto new computer instruction set architectures (https://en.wikipedia.org/wiki/instruction_set_architecture) and also allows all installed packages to be tuned. This strength surfaces another Gentoo *design principle*: *power*.

A system administrator who has successfully installed and customized Gentoo has compiled a tailored operating system from source code. The entire operating system can be tuned at a binary level via the mechanisms included in Portage's **make.conf** (</wiki/etc/portage/make.conf>) file. If so desired, adjustments can be made on a per-package basis, or a package group basis. In fact, entire sets of functionality can be added or removed using USE flags.

It is very important that the Handbook reader understands that these design principles are what makes Gentoo unique. With the principles of great power, many choices, and extreme openness highlighted, diligence, thought, and intentionality should be employed while using Gentoo.

How the installation is structured

The Gentoo Installation can be seen as a 10-step procedure, corresponding to the next set of chapters. Each step results in a certain state:

Step	Result
1	The user is in a working environment ready to install Gentoo.
2	The Internet connection is ready to install Gentoo.
3	The hard disks are initialized to host the Gentoo installation.
4	The installation environment is prepared and the user is ready to chroot (/wiki/Chroot) into the new environment.
5	Core packages, which are the same on all Gentoo installations, are installed.
6	The Linux kernel is installed.
7	Most of the Gentoo system configuration files are created.
8	The necessary system tools are installed.
9	The proper boot loader has been installed and configured.
10	The freshly installed Gentoo Linux environment is ready to be explored.

Deciding which steps to take

The handbook presents an overwhelming amount of options, especially for someone who has never installed Linux without an installer.

It's important to realize that the handbook is designed to describe the steps required to install on a very wide variety of hardware, with different install needs. Because of this, many options presented in the handbook are unnecessary for a particular install and can be skipped.

Suggested steps

Prefixed with "**Suggested:**", some steps are not strictly required, but help in most cases, such as installing `sys-kernel/linux-firmware` (<https://packages.gentoo.org/packages/sys-kernel/linux-firmware>) .

Optional steps

Prefixed with "**Optional:**", many sections in the handbook are purely optional, and can be skipped if the user is looking for a simple, mostly vanilla install.

Examples of this include compiler flag customization, using a totally custom kernel, and disabling root login.

📌 Tip

When following optional steps, it's important to be careful that all prerequisites were satisfied. Some optional steps depend on other optional steps.

Deprecated steps

Gentoo has existed for a long time. Some install processes are described in the handbook because they used to be more relevant, but are now largely deprecated. Instead of immediately removing this information, as it may still be helpful for some users, it may be designated as **Deprecated:** before removal. Once removed, the *history* must be used to view this content.

Defaults and alternatives

Whenever choices are presented, the handbook will try to explain the pros and cons of each choice.

If potential choices are mutually exclusive, "**Default:**" is used to designate the most supported or commonly chosen option, while alternatives are listed with "**Alternative**".

📌 Important

Alternative options are not inferior to **Defaults**, but **Default** options are typically more widely used and may have better support.

Installation options for Gentoo

Gentoo can be installed in many different ways. It can be downloaded and installed from official Gentoo installation media such as our bootable ISO images. The installation media can be installed on a USB stick or accessed via a netbooted environment. Alternatively, Gentoo can be installed from non-official media such as an already installed distribution or a non-Gentoo bootable disk (such as Linux Mint (<https://linuxmint.com/>)).

This document covers the installation using official Gentoo Installation media or, in certain cases, netbooting.

📌 Note

For help on the other installation approaches, including using non-Gentoo bootable media, please read our Alternative installation guide (/wiki/Installation_alternatives).

We also provide a Gentoo installation tips and tricks (/wiki/Gentoo_installation_tips_and_tricks) document that might be useful.

Troubles

If a problem is found in the installation (or in the installation documentation), please visit our bug tracking system (<https://bugs.gentoo.org>) and check if the bug is known. If not, please create a bug report for it so we can take care of it. Do not be afraid of the developers who are assigned to the bugs - they (generally) don't eat people.

Although this document is architecture-specific, it may contain references to other architectures as well, because large parts of the Gentoo Handbook use text that is identical for all architectures (to avoid duplication of effort). Such references have been kept to a minimum, to avoid confusion.

If there is some uncertainty about whether or not the problem is a user-problem (some error made despite having read the documentation carefully) or a software-problem (some error we made despite having tested the installation/documentation carefully) everybody is welcome to join the #gentoo (<ircs://irc.libera.chat/#gentoo>) (webchat (<https://web.libera.chat/#gentoo>)) channel on irc.libera.chat. Of course, everyone is welcome otherwise too as our chat channel covers the broad Gentoo spectrum.

Speaking of which, if there are any additional questions regarding Gentoo, check out the Frequently Asked Questions (</wiki/FAQ>) article. There are also FAQs (<https://forums.gentoo.org/viewforum.php?f=40>) on the Gentoo Forums (<https://forums.gentoo.org>).

Hardware requirements

Gentoo can be installed on all manners of systems, so listing minimal hardware specification is more down to how long a user is willing to wait. Any reasonably modern machine can complete the install process in less than an hour, however it is not a race so, take your time, and remember it takes as long as it takes.

For a smooth install process though, the Handbook recommends at least 40GB of space for the root partition.

Gentoo Linux installation media

📌 Tip

While it's recommended to use the official Gentoo boot media when installing, it's possible to use other installation environments. However, there is no guarantee they will contain required components. If an alternate install environment is used, skip to [Preparing the disks \(/wiki/Handbook:AMD64/Installation/Disks\)](/wiki/Handbook:AMD64/Installation/Disks).

Minimal installation CD

The *Gentoo minimal installation CD*, also known as the `installcd`, is a small, bootable image: a self-contained Gentoo environment. This image is maintained by Gentoo developers (</wiki/Project:RelEng>) and designed to allow any user with an Internet connection to install Gentoo. During the boot process, the hardware is detected, and appropriate drivers are automatically loaded.

Minimal Installation CD releases are named using the format: **install-`<arch>`-minimal-`<release timestamp>`.iso**.

❗ Important

The Gentoo minimal installation CD requires at least 140MB of RAM to boot.

The Gentoo LiveGUI

The LiveGUI provides a more convenient environment for the manual installation process, using the KDE desktop environment. This allows a user to open a graphical based web browser, such as Mozilla Firefox, to make reading the handbook whilst installing in terminal more pleasant.

The LiveGUI also includes more wireless driver support than the minimal CD provides and can be more easily connected using the NetworkManager applet in KDE

❗ Important

The Gentoo LiveGUI requires at least 2GB of RAM to boot.

What are stage files?

A stage file (/wiki/Stage_file) is an archive which serves as the seed for a Gentoo environment.

Stage 3 files can be downloaded from **releases/amd64/autobuilds/** on any of the official Gentoo mirrors (<https://www.gentoo.org/downloads/mirrors/>). Stages are updated frequently and are therefore not included within official live images.

✔ Tip

For now, *stage files* can be ignored. They will be described in greater detail later when they are needed

📌 Note

Historically, Gentoo provided stages lower than 3. Over time these type of installations have become obsolete due to better methods and no longer provided for this reason.

Downloading

Obtain the media

The default installation media used by Gentoo Linux are the *minimal installation CDs*, which provide a very small, bootable, Gentoo Linux environment. This environment contains the necessary tools to install Gentoo. The images themselves can be downloaded from the downloads page (<https://www.gentoo.org/downloads/>) (recommended) or by manually browsing to the ISO location on one of the many available mirrors (<https://www.gentoo.org/downloads/mirrors/>).

Navigating Gentoo mirrors

If downloading from a mirror, the minimal installation CDs can be found by:

1. Connect to the mirror, typically using a local one found at Gentoo source mirrors (<https://www.gentoo.org/downloads/mirrors/>).
2. Navigate to the **releases/** directory.
3. Select the directory for the relevant target architecture (such as **amd64/**).

4. Select the **autobuilds/** directory.
5. For **amd64** and **x86** architectures select either the **current-install-amd64-minimal/** or **current-install-x86-minimal/** directory (respectively). For all other architectures navigate to the **current-iso/** directory.

Note

Some target architectures such as **arm**, **mips**, and **s390** will not have minimal install CDs. At this time the Gentoo Release Engineering project (/wiki/Project:RelEng) does not support building **.iso** files for these targets.

Inside this location, the installation media file is the file with the **.iso** suffix. For instance, take a look at the following listing:

CODE Example list of downloadable files at releases/amd64/autobuilds/current-install-amd64-minimal/

[TXT]	install-amd64-minimal-20231112T170154Z.iso.asc	2023-11-12
20:41	488	
[TXT]	install-amd64-minimal-20231119T164701Z.iso.asc	2023-11-19
18:41	488	
[TXT]	install-amd64-minimal-20231126T163200Z.iso.asc	2023-11-26
18:41	488	
[TXT]	install-amd64-minimal-20231203T170204Z.iso.asc	2023-12-03
18:41	488	
[TXT]	install-amd64-minimal-20231210T170356Z.iso.asc	2023-12-10
19:01	488	
[TXT]	install-amd64-minimal-20231217T170203Z.iso.asc	2023-12-17
20:01	488	
[TXT]	install-amd64-minimal-20231224T164659Z.iso.asc	2023-12-24
20:41	488	
[TXT]	install-amd64-minimal-20231231T163203Z.iso.asc	2023-12-31
19:01	488	
[]	install-amd64-minimal-20240107T170309Z.iso	2024-01-07
20:42	466M	
[]	install-amd64-minimal-20240107T170309Z.iso.CONTENTS.gz	2024-01-07
20:42	9.8K	
[]	install-amd64-minimal-20240107T170309Z.iso.DIGESTS	2024-01-07
21:01	1.3K	
[TXT]	install-amd64-minimal-20240107T170309Z.iso.asc	2024-01-07
21:01	488	
[]	install-amd64-minimal-20240107T170309Z.iso.sha256	2024-01-07
21:01	660	
[TXT]	latest-install-amd64-minimal.txt	2024-01-08
02:01	653	

In the above example, the **install-amd64-minimal-20240107T170309Z.iso** file is the minimal installation CD itself. But as can be seen, other related files exist as well:

- A **.CONTENTS.gz** file which is a gz-compressed text file listing all files available on the installation media. This file can be useful to check if particular firmware or drivers are available on the installation media before downloading it.

- A **.asc** file which is a cryptographic signature of the ISO file. This is used to verify image integrity and authenticity - that the download is indeed provided by the Gentoo Release Engineering team (/wiki/Project:RelEng), free from tampering.

Ignore the other files available at this location for now - those will come back when the installation has proceeded further. Download the **.iso** file and, if verification of the download is wanted, download the **.iso.asc** file for the **.iso** file as well.

Verifying the downloaded files

This step ensures that the downloaded file is not corrupt and has indeed been provided by the Gentoo Infrastructure team (/wiki/Project:Infrastructure).

The **.asc** file provides a cryptographic signature of the ISO. Validating it ensures the installation file is provided by the Gentoo Release Engineering team and is intact and unmodified.

Microsoft Windows-based verification

To first verify the cryptographic signature, tools such as GPG4Win (<https://www.gpg4win.org/>) can be used. After installation, the public keys of the Gentoo Release Engineering team need to be imported. The list of keys is available on the signatures page (<https://www.gentoo.org/downloads/signatures/>). Once imported, the downloaded ISO can be verified using the **.asc** file.

Linux based verification

On a Linux system, the most common method for verifying the cryptographic signature is to use the app-crypt/gnupg (<https://packages.gentoo.org/packages/app-crypt/gnupg>) [Ⓐ] software. With this package installed, the following command is used to verify the cryptographic signature in the **.asc** file.

❗ Important

When importing Gentoo keys, verify that the 16-character key ID (BB572E0E2D182910) matches.

Gentoo keys can be downloaded from **hkps://keys.gentoo.org** using fingerprints available on the signatures page (<https://www.gentoo.org/downloads/signatures/>):

```
user $ gpg --keyserver hkps://keys.gentoo.org --recv-keys
13EBDBEDE7A12775DFDB1BABB572E0E2D182910
```

```
gpg: directory '/root/.gnupg' created
gpg: keybox '/root/.gnupg/pubring.kbx' created
gpg: /root/.gnupg/trustdb.gpg: trustdb created
gpg: key BB572E0E2D182910: public key "Gentoo Linux Release Engineering
(Automated Weekly Release Key) <releng@gentoo.org>" imported
gpg: Total number processed: 1
gpg:             imported: 1
```

Alternatively, WKD (/wiki/WKD) can be used to download the keys:

```
user $ gpg --auto-key-locate=clear,nodefault,wkd --locate-key releng@gentoo.org
```

```

gpg: key 9E6438C817072058: public key "Gentoo Linux Release Engineering
(Gentoo Linux Release Signing Key) <releng@gentoo.org>" imported
gpg: key BB572E0E2D182910: public key "Gentoo Linux Release Engineering
(Automated Weekly Release Key) <releng@gentoo.org>" imported
gpg: Total number processed: 2
gpg:             imported: 2
gpg: no ultimately trusted keys found
pub   dsa1024 2004-07-20 [SC] [expires: 2025-07-01]
       D99EAC7379A850BCE47DA5F29E6438C817072058
uid           [ unknown] Gentoo Linux Release Engineering (Gentoo Linux
Release Signing Key) <releng@gentoo.org>
sub   elg2048 2004-07-20 [E] [expires: 2025-07-01]

```

Or if using official Gentoo release media, import the key from **/usr/share/openpgp-keys/gentoo-release.asc** (provided by

sec-keys/openpgp-keys-gentoo-release (<https://packages.gentoo.org/packages/sec-keys/openpgp-keys-gentoo-release>)

```
user $ gpg --import /usr/share/openpgp-keys/gentoo-release.asc
```

```

gpg: directory '/home/larry/.gnupg' created
gpg: keybox '/home/larry/.gnupg/pubring.kbx' created
gpg: key DB6B8C1F96D8BF6D: 2 signatures not checked due to missing keys
gpg: /home/larry/.gnupg/trustdb.gpg: trustdb created
gpg: key DB6B8C1F96D8BF6D: public key "Gentoo ebuild repository signing key
(Automated Signing Key) <infrastructure@gentoo.org>" imported
gpg: key 9E6438C817072058: 3 signatures not checked due to missing keys
gpg: key 9E6438C817072058: public key "Gentoo Linux Release Engineering
(Gentoo Linux Release Signing Key) <releng@gentoo.org>" imported
gpg: key BB572E0E2D182910: 1 signature not checked due to a missing key
gpg: key BB572E0E2D182910: public key "Gentoo Linux Release Engineering
(Automated Weekly Release Key) <releng@gentoo.org>" imported
gpg: key A13D0EF1914E7A72: 1 signature not checked due to a missing key
gpg: key A13D0EF1914E7A72: public key "Gentoo repository mirrors (automated
git signing key) <repomirrorci@gentoo.org>" imported
gpg: Total number processed: 4
gpg:             imported: 4
gpg: no ultimately trusted keys found

```

Next verify the cryptographic signature:

```
user $ gpg --verify install-amd64-minimal-20240107T170309Z.iso.asc install-amd64-minimal-20240107T170309Z.iso
```

```

gpg: Signature made Sun 07 Jan 2024 03:01:10 PM CST
gpg:             using RSA key 534E4209AB49EEE1C19D96162C44695DB9F6043D
gpg: Good signature from "Gentoo Linux Release Engineering (Automated
Weekly Release Key) <releng@gentoo.org>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:             There is no indication that the signature belongs to the
owner.
Primary key fingerprint: 13EB BDBE DE7A 1277 5DFD  B1BA BB57 2E0E 2D18 2910
Subkey fingerprint: 534E 4209 AB49 EEE1 C19D  9616 2C44 695D B9F6 043D

```

To be absolutely certain that everything is valid, verify the fingerprint shown with the fingerprint on the Gentoo signatures page (<https://www.gentoo.org/downloads/signatures/>).

Note

It's generally good practice to mark an imported key as trusted, once it's certain the key is trustworthy. When trusted keys are verified, **gpg** will not say *unknown* and warn about the signature being untrusted.

Writing the boot media

Of course, with just an ISO file downloaded, the Gentoo Linux installation cannot be started. The ISO file must be written to bootable media. This generally requires that the image is extracted to a filesystem, or written directly to a device.

Writing a bootable USB

Warning

Using tools such as Ventoy to write to a USB drive can cause boot failures.

Most modern systems support booting from a USB device.

Writing with Linux

dd is typically available on most Linux distros, and can be used to write the Gentoo boot media to a USB drive.

Determining the USB device path

Before writing, the path to the desired storage device must be determined.

dmesg will display detailed information describing the storage device as it is added to the system:

```
root # dmesg
```

```
[268385.319745] sd 19:0:0:0: [sdd] 60628992 512-byte logical blocks: (31.0 GB/28.9 GiB)
```

Alternatively, **lsblk** can be used to display available storage devices:

```
root # lsblk
```

sdd	8:48	1	28.9G	0	disk
└sdd1	8:49	1	246K	0	part
└sdd2	8:50	1	2.8M	0	part
└sdd3	8:51	1	463.5M	0	part
└sdd4	8:52	1	300K	0	part

Once the device name has been determined, this can be added to the path prefix **/dev/** to get the device path **/dev/sdd**.

✔ Tip

Using the base device path, ie. **sdd** opposed to **sdd1**, is recommend as the Gentoo *boot media* contains a full GPT (/wiki/GPT) partition scheme.

Writing with dd

⚠ Warning

Be sure to check the target (*of=target*) path before executing **dd**, as it will be overwritten.

With the device path (**/dev/sdd**) and *boot media* **install-amd64-minimal-<release timestamp>.iso** ready:

```
root # dd if=install-amd64-minimal-<release timestamp>.iso of=/dev/sdd bs=4096
status=progress && sync
```

📄 Note

if= specifies the *input file*, **of=** specifies the *output file*, which in this case, is a device.

✔ Tip

bs=4096 is used as it speeds up transfers in most cases, **status=progress** displays transfers stats.

Writing with Windows

win32diskimager (<https://sourceforge.net/projects/win32diskimager/>) is simple to use Windows utility to write iso images to a USB drive.

They also provide excellent documentation to assist if needed.

⚠ Warning

Only download from <https://sourceforge.net/projects/win32diskimager/> (<https://sourceforge.net/projects/win32diskimager/>)

Burning a disk

🔍 See also

A more elaborate set of instructions can be found in [CD/DVD/BD_writing#Image_writing \(/wiki/CD/DVD/BD_writing#Image_writing\)](#).

Burning with Microsoft Windows 7 and above

Versions of Microsoft Windows 7 and above can both mount and burn ISO images to optical media without the requirement for third-party software. Simply insert a burnable disk, browse to the downloaded ISO files, right click the file in Windows Explorer, and select "Burn disk image".

Burning with Linux

The **cdrecord** utility from the package `app-cdr/cdrtools` (<https://packages.gentoo.org/packages/app-cdr/cdrtools>) [Ⓐ] can burn ISO images on Linux.

To burn the ISO file on the CD in the `/dev/sr0` device (this is the first CD device on the system - substitute with the right device file if necessary):

```
user $ cdrecord dev=/dev/sr0 install-amd64-minimal-20141204.iso
```

Users that prefer a graphical user interface can use K3B, part of the `kde-apps/k3b` (<https://packages.gentoo.org/packages/kde-apps/k3b>) [Ⓐ] package. In K3B, go to **Tools** and use **Burn CD Image**.

Bootimg

Bootimg the installation media

Once the installation media is ready, it is time to boot it. Insert the media in the system, reboot, and enter the motherboard's firmware user interface. This is usually performed by pressing a keyboard key such as `DEL`, `F1`, `F10`, or `ESC` during the Power-On Self-Test (POST) process. The 'trigger' key varies depending on the system and motherboard. If it is not obvious use an internet search engine and do some research using the motherboard's model name as the search keyword. Results should be easy to determine. Once inside the motherboard's firmware menu, change the boot order so that the external bootable media (CD/DVD disks or USB drives) are tried *before* the internal disk devices. Without this change, the system will most likely reboot to the internal disk device, ignoring the newly attached bootable media.

❗ Important

When installing Gentoo on a system with an UEFI firmware interface, ensure the live image has been booted in UEFI mode. In the accidental event that DOS/legacy BIOS boot was initiated, then it will be necessary to reboot in UEFI mode before finalizing the Gentoo Linux installation.

Ensure that the installation media is inserted or plugged into the system, and reboot. A GRUB boot prompt should be shown with various boot entries. At this screen, **Enter** will begin the boot process with the default boot options. To boot the installation media with customized boot options, such as passing additional kernel parameters or the following hardware options, highlight a boot entry, then press the **e** key to edit the boot entry. Make the necessary modification(s), then press **ctrl + x** or **F10** to boot the modified entry.

Note

In all likelihood, the default gentoo kernel, as mentioned above, without specifying any of the optional parameters will work just fine. For boot troubleshooting and expert options, continue on with this section. Otherwise, just press **Enter** and skip ahead to Extra hardware configuration ([/wiki/Handbook:AMD64/Installation/Media#Extra_hardware_configuration](#)).

At the boot prompt, users get the option of displaying the available kernels (**F1**) and boot options (**F2**). If no choice is made within 15 seconds (either displaying information or using a kernel) then the installation media will fall back to booting from disk. This allows installations to reboot and try out their installed environment without the need to remove the CD from the tray (something well appreciated for remote installations).

Specifying a kernel was mentioned. On the Minimal installation media, only two predefined kernel boot entries are provided. The default option is called **gentoo**. The other being the **-nofb** variant; this disables kernel framebuffer support.

The next section displays a short overview of the available kernels and their descriptions:

Kernel choices

gentoo

Default kernel with support for K8 CPUs (including NUMA support) and EM64T CPUs.

gentoo-nofb

Same as *gentoo* but without framebuffer support.

memtest86

Test the system RAM for errors.

Alongside the kernel, boot options help in tuning the boot process further.

Hardware options

acpi=on

This loads support for ACPI and also causes the acpid daemon to be started by the CD on boot. This is only needed if the system requires ACPI to function properly. This is not required for Hyperthreading support.

acpi=off

Completely disables ACPI. This is useful on some older systems and is also a requirement for using APM. This will disable any Hyperthreading support of your processor.

console=X

This sets up serial console access for the CD. The first option is the device, usually ttyS0, followed by any connection options, which are comma separated. The default options are 9600,8,n,1.

dmraid=X

This allows for passing options to the device-mapper RAID subsystem. Options should be encapsulated in quotes.

doapm

This loads APM driver support. This also requires that `acpi=off`.

dopcmcia

This loads support for PCMCIA and Cardbus hardware and also causes the `pcmcia cardmgr` to be started by the CD on boot. This is only required when booting from PCMCIA/Cardbus devices.

doscsi

This loads support for most SCSI controllers. This is also a requirement for booting most USB devices, as they use the SCSI subsystem of the kernel.

sda=stroke

This allows the user to partition the whole hard disk even when the BIOS is unable to handle large disks. This option is only used on machines with an older BIOS. Replace `sda` with the device that requires this option.

ide=nodma

This forces the disabling of DMA in the kernel and is required by some IDE chipsets and also by some CDROM drives. If the system is having trouble reading from the IDE CDROM, try this option. This also disables the default `hdparm` settings from being executed.

noapic

This disables the Advanced Programmable Interrupt Controller that is present on newer motherboards. It has been known to cause some problems on older hardware.

nodetect

This disables all of the autodetection done by the CD, including device autodetection and DHCP probing. This is useful for debugging a failing CD or driver.

nodhcp

This disables DHCP probing on detected network cards. This is useful on networks with only static addresses.

nodmraid

Disables support for device-mapper RAID, such as that used for on-board IDE/SATA RAID controllers.

nofirewire

This disables the loading of Firewire modules. This should only be necessary if your Firewire hardware is causing a problem with booting the CD.

nogpm

This disables `gpm` console mouse support.

nohotplug

This disables the loading of the `hotplug` and `coldplug` init scripts at boot. This is useful for debugging a failing CD or driver.

nokeymap

This disables the keymap selection used to select non-US keyboard layouts.

nolapic

This disables the local APIC on Uniprocessor kernels.

nosata

This disables the loading of Serial ATA modules. This is used if the system is having problems with the SATA subsystem.

nosmp

This disables SMP, or Symmetric Multiprocessing, on SMP-enabled kernels. This is useful for debugging SMP-related issues with certain drivers and motherboards.

nosound

This disables sound support and volume setting. This is useful for systems where sound support causes problems.

nousb

This disables the autoloading of USB modules. This is useful for debugging USB issues.

slowusb

This adds some extra pauses into the boot process for slow USB CDROMs, like in the IBM BladeCenter.

Logical volume/device management

dolvm

This enables support for Linux's Logical Volume Management.

Other options

debug

Enables debugging code. This might get messy, as it displays a lot of data to the screen.

docache

This caches the entire runtime portion of the CD into RAM, which allows the user to umount `/mnt/cdrom` and mount another CDROM. This option requires that there is at least twice as much available RAM as the size of the CD.

doload=X

This causes the initial ramdisk to load any module listed, as well as dependencies. Replace X with the module name. Multiple modules can be specified by a comma-separated list.

dosshd

Starts sshd on boot, which is useful for unattended installs.

passwd=foo

Sets whatever follows the equals as the root password, which is required for *dosshd* since the root password is by default scrambled.

noload=X

This causes the initial ramdisk to skip the loading of a specific module that may be causing a problem. Syntax matches that of *doload*.

nonfs

Disables the starting of portmap/nfsmount on boot.

nox

This causes an X-enabled LiveCD to not automatically start X, but rather, to drop to the command line instead.

scandelay

This causes the CD to pause for 10 seconds during certain portions the boot process to allow for devices that are slow to initialize to be ready for use.

scandelay=X

This allows the user to specify a given delay, in seconds, to be added to certain portions of the boot process to allow for devices that are slow to initialize to be ready for use.

Replace X with the number of seconds to pause.

Note

The bootable media will check for `no*` options before `do*` options, so that options can be overridden in the exact order specified.

Now boot the media, select a kernel (if the default **gentoo** kernel does not suffice) and boot options. As an example, we boot the **gentoo** kernel, with `dopcmcia` as a kernel parameter:

```
boot: gentoo dopcmcia
```

Next the user will be greeted with a boot screen and progress bar. If the installation is done on a system with a non-US keyboard, make sure to immediately press `Alt + F1` to switch to verbose mode and follow the prompt. If no selection is made in 10 seconds the default (US keyboard) will be accepted and the boot process will continue. Once the boot process completes, the user is automatically logged in to the "Live" Gentoo Linux environment as the *root* user, the super user. A root prompt is displayed on the current console, and one can switch to other consoles by pressing `Alt + F2`, `Alt + F3` and `Alt + F4`. Get back to the one started on by pressing `Alt + F1`.

LiveGUI root access

sudo has been configured to run without the need of a password on the LiveGUI as both the root and gentoo have a scrambled password.

To gain access to the superuser account, in any terminal run:

```
root # sudo -i
```

Extra hardware configuration

When the Installation medium boots, it tries to detect all the hardware devices and loads the appropriate kernel modules to support the hardware. In the vast majority of cases, it does a very good job. However, in some cases it may not auto-load the kernel modules needed by the system. If the PCI auto-detection missed some of the system's hardware, the appropriate kernel modules have to be loaded manually.

In the next example the `8139too` module (which supports certain kinds of network interfaces) is loaded:

```
root # modprobe 8139too
```

Optional: User accounts

If other people need access to the installation environment, or there is need to run commands as a non-root user on the installation medium (such as to chat using **irssi** without root privileges for security reasons), then an additional user account needs to be created and the root password set to a strong password.

To change the root password, use the **passwd** utility:

```
root # passwd
```

New password: (Enter the new password)

Re-enter password: (Re-enter the password)

To create a user account, first enter their credentials, followed by the account's password. The **useradd** and **passwd** commands are used for these tasks.

In the next example, a user called *larry* is created and is added to the `users` and `wheel` groups:

```
root # useradd -m -G users,wheel larry
```

```
root # passwd larry
```

```
New password: (Enter larry's password)
Re-enter password: (Re-enter larry's password)
```

To switch from the (current) *root* user to the newly created user account, use the **su** command:

```
root # su - larry
```

Optional: Viewing documentation while installing

TTYs

To view the Gentoo handbook from a TTY during the installation, first create a user account as described above, then press **Alt** + **F2** to go to a new terminal (TTY) and login as the newly created user. Following the principle of least privilege (https://en.wikipedia.org/wiki/Principle_of_least_privilege), it is best practice to avoid browsing the web or generally performing any task with higher privileges than necessary. The root account has full control of the system and therefore must be used sparingly.

During the installation, the **links** web browser can be used to browse the Gentoo handbook - of course only from the moment that the Internet connection is working.

```
user $ links https://wiki.gentoo.org/wiki/Handbook:AMD64
```

To go back to the original terminal, press **Alt** + **F1**.

Tip

When booted to the Gentoo minimal or Gentoo admin environments, seven TTYs will be available. They can be switched by pressing **Alt** then a function key between **F1** - **F7**. It can be useful to switch to a new terminal when waiting for job to complete, to open documentation, etc.

GNU Screen

The Screen (</wiki/Screen>) utility is installed by default on official Gentoo installation media. It may be more efficient for the seasoned Linux enthusiast to use **screen** to view installation instructions via split panes rather than the multiple TTY method mentioned above.

Optional: Starting the SSH daemon

To allow other users to access the system during the installation (perhaps to provide/receive support during an installation, or even do it remotely), a user account needs to be created (as was documented earlier on) and the SSH daemon needs to be started.

To fire up the SSH daemon on an OpenRC init, execute the following command:

```
root # rc-service sshd start
```

❏ Note

If users log on to the system, they will see a message that the host key for this system needs to be confirmed (through what is called a fingerprint). This behavior is typical and can be expected for initial connections to an SSH server. However, later when the system is set up and someone logs on to the newly created system, the SSH client will warn that the host key has been changed. This is because the user now logs on to - for SSH - a different server (namely the freshly installed Gentoo system rather than the live environment that the installation is currently using). Follow the instructions given on the screen then to replace the host key on the client system.

To be able to use `sshd`, the network needs to function properly. Continue with the chapter on Configuring the network (</wiki/Handbook:AMD64/Installation/Networking>).

Automatic network configuration

Maybe it just works?

If the system is connected to an Ethernet network with an IPv6 router or DHCP server, it's very likely that the system's network was configured automatically. If additional, advanced configuration is not required, Internet connectivity can be tested (/wiki/Handbook:AMD64/Installation/Networking#Testing_the_network).

Using DHCP

DHCP (Dynamic Host Configuration Protocol) assists in network configuration, and can automatically provide configuration for a variety of parameters including: IP address, network mask, routes, DNS servers, NTP servers, etc.

DHCP requires that a server be running on the same *Layer 2 (Ethernet)* segment as the client requesting a *lease*. DHCP is often used on RFC1918 (*private*) networks, but is also used to acquire public IP information from ISPs.

By default the Gentoo live media uses NetworkManager (</wiki/NetworkManager>) which will automatically connect to via DHCP, if this is not the case then check the Ethernet cable for issues.

Testing the network

A properly configured *default* route is a critical component of Internet connectivity, route configuration can be checked with:

```
root # ip route
```

```
default via 192.168.0.1 dev enp1s0
```

If no *default* route is defined, Internet connectivity is unavailable, and additional configuration is required.

Basic internet connectivity can be confirmed with a ping:

```
root # ping -c 3 1.1.1.1
```

Tip

It's helpful to start by pinging a known IP address instead of a hostname. This can isolate DNS issues from basic Internet connectivity issues.

Outbound HTTPS access and DNS resolution can be confirmed with:

```
root # curl --location gentoo.org --output /dev/null
```

Unless **curl** reports an error, or other tests fail, the installation process can be continued with disk preparation (/wiki/Handbook:AMD64/Installation/Disks).

If **curl** reports an error, but Internet-bound pings work, DNS may need configuration (/wiki/Handbook:AMD64/Installation/Networking#DNS_configuration).

If Internet connectivity has not been established, first interface information should be verified (/wiki/Handbook:AMD64/Installation/Networking#Obtaining_interface_info), then:

- **nmtui** can be used (/wiki/Handbook:AMD64/Installation/Networking#Using_NetworkManager) to assist in network configuration.
- Application specific configuration (/wiki/Handbook:AMD64/Installation/Networking#Application_specific_configuration) may be required.
- Manual network configuration (/wiki/Handbook:AMD64/Installation/Networking#Manual_network_configuration) can be attempted.

Obtaining interface info

If networking doesn't work out of the box, additional steps must be taken to enable Internet connectivity. Generally, the first step is to enumerate host network interfaces.

The **ip** command, part of the `sys-apps/iproute2` (<https://packages.gentoo.org/packages/sys-apps/iproute2>) package, can be used to query and configure system networking.

The **link** argument can be used to display network interface links:

```
root # ip link
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
  DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
  state UP mode DEFAULT group default qlen 1000
    link/ether e8:40:f2:ac:25:7a brd ff:ff:ff:ff:ff:ff
```

The **address** argument can be used to query device address information:

```
root # ip address
```



```
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
state UP group default qlen 1000<pre>
    link/ether e8:40:f2:ac:25:7a brd ff:ff:ff:ff:ff:ff
    inet 10.0.20.77/22 brd 10.0.23.255 scope global enp1s0
        valid_lft forever preferred_lft forever
    inet6 fe80::ea40:f2ff:feac:257a/64 scope link
        valid_lft forever preferred_lft forever
```

The output of this command contains information for each network interface on the system. Entries begin with the device index, followed by the device name: **enp1s0**.

✔ Tip

If no interfaces other than the **lo** (*loopback*) are displayed, then the networking hardware is faulty, or the driver for the interface has not been loaded into the kernel. Both situations reach beyond the scope of this Handbook. Please ask for support in contact [#gentoo](https://web.libera.chat/#gentoo) ([ircs://irc.libera.chat/#gentoo](https://web.libera.chat/#gentoo)) ([webchat \(https://web.libera.chat/#gentoo\)](https://web.libera.chat/#gentoo)).

For consistency, the handbook will assume that the primary network interface is called **enp1s0**.

📌 Note

As a result of the shift toward predictable network interface names (<https://www.freedesktop.org/wiki/Software/systemd/PredictableNetworkInterfaceNames/>), the interface name on the system can be quite different than the old **eth0** naming convention. Modern Gentoo boot media uses interface names with prefixes such as **eno0**, **ens1**, or **enp5s0**.

Wireless Setup

Optional: Checking wireless

iw may be used to display the current wireless settings on the card, this should also show that the kernel wireless stack is working (note that the **iw** command is only available on the following architectures: **amd64**, **x86**, **arm**, **arm64**, **ppc**, **ppc64**, and **riscv**):

```
root # iw dev wlp9s0 info
```

```
Interface wlp9s0
    ifindex 3
    wdev 0x1
    addr 00:00:00:00:00:00
    type managed
    wiphy 0
    channel 11 (2462 MHz), width: 20 MHz (no HT), center1: 2462 MHz
    txpower 30.00 dBm
```

Note

Some wireless cards may have a device name of **wlan0** or **ra0** instead of **wlp9s0**. Run **ip link** to determine the correct device name.

Using NetworkManager

The fastest way to enable wireless connectivity in the Gentoo Live media is by using the **nmtui** command and following the on-screen instructions to setup the wireless network.

```
root # nmtui
```

LiveGUI users can click on the network icon in the bottom right of the taskbar to also configure WiFi.

Using net-setup

On some architectures, e.g. **HPPA**, a user may be required to use the tool **net-setup** to configure a wireless connection if NetworkManager isn't available yet.

```
root # killall dhcpcd
```

```
root # net-setup enp1s0
```

net-setup will ask some questions about the network environment and will use that information to configure **wpa_supplicant** or *static addressing*.

Important

Network status should be tested (/wiki/Handbook:AMD64/Installation/Networking#Testing_the_network) after any configuration steps are taken. In the event that the configuration scripts do not work, manual network configuration (/wiki/Handbook:AMD64/Installation/Networking#Manual_network_configuration) is required.

Manual setup

Wireless networks may alternatively be set up with a daemon such as **wpa_supplicant** or **iwd**. For more information on configuring wireless networking in Gentoo Linux, please read the Wireless networking chapter (</wiki/Handbook:AMD64/Networking/Wireless>) in the Gentoo Handbook.}}

Manually connecting to an open network

For most users, there are only two settings needed to connect, the ESSID (aka wireless network name) and, optionally, the WEP key.

- First, ensure the interface is active:

```
root # ip link set dev wlp9s0 up
```

- To connect to an open network with the name *GentooNode*:

```
root # iw dev wlp9s0 connect -w GentooNode
```

Application specific configuration

📌 Tip

These steps are provided for users where using **nmtui** is not able to configure their network's needs.

The following methods are not generally required, but may be helpful in situations where additional configuration is required for Internet connectivity.

Configure web proxies

If the internet is accessed through a web proxy, then it will be necessary to define proxy information to for Portage to properly access the proxy for each supported protocol. Portage observes the *http_proxy*, *ftp_proxy*, and *RSYNC_PROXY* environment variables in order to download packages via its **wget** and **rsync** retrieval mechanisms.

Certain text-mode web browsers such as **links** can also make use of environment variables that define web proxy settings; in particular for the HTTPS access it also will require the *https_proxy* environment variable to be defined. While Portage will be influenced without passing extra run time parameters during invocation, **links** will require proxy settings to be set.

In most cases, it is sufficient to define environment variables using the server hostname. In the following example, it is assumed the proxy server host is called proxy.gentoo.org and the port is 8080.

📌 Note

The `#` symbol in the following commands is a comment. It has been added for clarity only and does *not* need to be typed when entering the commands.

To define an HTTP proxy (for HTTP and HTTPS traffic):

```
root # export http_proxy="http://proxy.gentoo.org:8080" # Applies to Portage and Links
```

```
root # export https_proxy="http://proxy.gentoo.org:8080" # Only applies for Links
```

If the HTTP proxy requires authentication, set a username and password with the following syntax:

```
root # export http_proxy="http://username:password@proxy.gentoo.org:8080" # Applies to Portage and Links
```

```
root # export https_proxy="http://username:password@proxy.gentoo.org:8080" # Only applies for Links
```

Start **links** using the following parameters for proxy support:

```
user $ links -http-proxy ${http_proxy} -https-proxy ${https_proxy}
```

To define an FTP proxy for Portage and/or **links**:

```
root # export ftp_proxy="ftp://proxy.gentoo.org:8080" # Applies to Portage and Links
```

Start **links** using the following parameter for a FTP proxy:

```
user $ links -ftp-proxy ${ftp_proxy}
```

To define an RSYNC proxy for Portage:

```
root # export RSYNC_PROXY="proxy.gentoo.org:8080" # Applies to Portage; Links
does not support a rsync proxy
```

Using pppoe-setup for ADSL

If PPPoE is required for Internet access, the Gentoo *boot media* includes the **pppoe-setup** script to simplify **ppp** configuration.

During setup, **pppoe-setup** will ask for:

- The name of the Ethernet *interface* connected to the ADSL modem.
- The PPPoE username and password.
- DNS server IPs.
- Whether or not a firewall is needed.

```
root # pppoe-setup
```

```
root # pppoe-start
```

In the event of failure, credentials in **/etc/ppp/pap-secrets** or **/etc/ppp/chap-secrets** should be verified. If credentials are correct, PPPoE Ethernet interface selection should be checked.

Using PPTP

If PPTP support is needed, **pptpclient** can be used, but requires configuration prior to usage.

Edit **/etc/ppp/pap-secrets** or **/etc/ppp/chap-secrets** so it contains the correct username/password combination:

```
root # nano /etc/ppp/chap-secrets
```

Then adjust **/etc/ppp/options.pptp** if necessary:

```
root # nano /etc/ppp/options.pptp
```

Once configuration is complete, run **pptp** (along with the options that couldn't be set in **options.pptp**) to connect the server:

```
root # pptp <server ipv4 address>
```

Internet and IP basics

If all of the above fails, the network must be configured manually. This is not particularly difficult, but should be done with consideration. This section serves to clarify terminology and introduce users to basic networking concepts pertaining to manually configuring an Internet connection.

☑ Tip

Some **CPE (Carrier Provided Equipment)** combines the functions of a *router*, *access*

point, modem, DHCP server, and DNS server into one unit. It's important to differentiate the functions of a device from the physical appliance.

Interfaces and addresses

Network *interfaces* are logical representations of network devices. An *interface* needs an *address* to communicate with other devices on the *network*. While only a single *address* is required, multiple addresses can be assigned to a single *interface*. This is especially useful for dual stack (IPv4 + IPv6) configurations.

For consistency, this primer will assume the interface **enp1s0** will be using the address 192.168.0.2.

❗ Important

IP addresses can be set arbitrarily. As a result, it's possible for multiple devices to use the same IP address, resulting in an *address conflict*. Address conflicts should be avoided by using DHCP or SLAAC.

💡 Tip

IPv6 typically uses **StateLess Address AutoConfiguration** (SLAAC) for address configuration. In most cases, manually setting IPv6 addresses is a bad practice. If a specific address suffix is preferred, interface identification tokens ([/wiki/IPv6_Static_Addresses_using_Tokens](#)) can be used.

Networks and CIDR

Once an address is chosen, how does a device know how to talk to other devices?

IP *addresses* are associated with *networks*. IP *networks* are contiguous logical ranges of addresses.

Classless Inter-Domain Routing or *CIDR* notation is used to distinguish network sizes.

- The *CIDR* value, often notated starting with a */*, represents the size of the network.
 - The formula $2^{(32 - CIDR)}$ can be used to calculate network size.
 - Once network size is calculated, usable node count must be reduced by 2.
 - The first IP in a network is the *Network address*, and the last is typically the *Broadcast address*. These addresses are special and cannot be used by normal hosts.

💡 Tip

The most common *CIDR* values are **/24**, and **/32**, representing **254** nodes and a single node respectively.

A *CIDR* of **/24** is the de-facto default network size. This corresponds to a subnet mask of 255.255.255.0, where the last 8 bits are reserved for IP addresses for nodes on a network.

The notation: **192.168.0.2/24** can be interpreted as:

- The *address* **192.168.0.2**
- On the *network* 192.168.0.0

- With a size of **254** ($2^{(32 - 24)} - 2$)
 - Usable IPs are in the range 192.168.0.1 - 192.168.0.254
- With a *broadcast address* of 192.168.0.255
 - In most cases, the last address on a network is used as the *broadcast address*, but this can be changed.

Using this configuration, a device should be able to communicate with any host on the same *network* (**192.168.0.0**).

The Internet

Once a device is on a network, how does it know how to talk to devices on the Internet?

To communicate with devices outside of local *networks*, *routing* must be used. A *router* is simply a network device that forwards traffic for other devices. The term *default route* or *gateway* typically refers to whatever device on the current network is used for external network access.

✔ Tip

It's a standard practice to make the *gateway* the first or last IP on a network.

If an Internet-connected router is available at **192.168.0.1**, it can be used as the *default route*, granting Internet access.

To summarize:

- Interfaces must be configured with an *address* and *network information*, such as the *CIDR* value.
- Local network access is used to access a *router* on the same network.
- The *default route* is configured, so traffic destined for external networks is forwarded to the *gateway*, providing Internet access.

The Domain Name System

Remembering IPs is hard. The *Domain Name System* was created to allow mapping between *Domain Names* and *IP addresses*.

Linux systems use **/etc/resolv.conf** to define *nameservers* to be used for *DNS resolution*.

✔ Tip

Many *routers* can also function as a DNS server, and using a local DNS server can augment privacy and speed up queries through caching.

Many ISPs run a DNS server that is generally advertised to the *gateway* over DHCP. Using a local DNS server tends to improve query latency, but most public DNS servers will return the same results, so server usage is largely based on preference.

Manual and static IP network configuration

Interface address configuration

❗ Important

When manually configuring IP addresses, the local network topology must be considered. IP addresses can be set arbitrarily; conflicts may cause network disruption.

To configure **enp1s0** with the address **192.168.0.2** and CIDR /24:

```
root # ip address add 192.168.0.2/24 dev enp1s0
```

💡 Tip

The start of this command can be shortened to **ip a**.

Default route configuration

Configuring address and network information for an interface will configure **link** routes, allowing communication with that network segment:

```
root # ip route
```

```
192.168.0.0/24 dev enp1s0 proto kernel scope link src 192.168.0.2
```

💡 Tip

This command can be shortened to **ip r**.

The **default** route can be set to **192.168.0.1** with:

```
root # ip route add default via 192.168.0.1
```

DNS configuration

Nameserver info is typically acquired using DHCP, but can be set manually by adding **nameserver** entries to **/etc/resolv.conf**.

⚠ Warning

If **dhcpcd** is running, changes to **/etc/resolv.conf** will not persist. Status can be checked with `ps x | grep dhcpcd`.

nano is included in Gentoo *boot media* and can be used to edit **/etc/resolv.conf** with:

```
root # nano /etc/resolv.conf
```

Lines containing the keyword **nameserver** followed by a DNS server IP address are queried in order of definition:

FILE **/etc/resolv.conf** Use Quad9 DNS.

```
nameserver 9.9.9.9
nameserver 149.112.112.112
```

FILE `/etc/resolv.conf` **Use Cloudflare DNS.**

```
nameserver 1.1.1.1
nameserver 1.0.0.1
```

DNS status can be checked by pinging a domain name:

```
root # ping -c 3 gentoo.org
```

Once connectivity has been verified ([/wiki/Handbook:AMD64/Installation/Networking#Testing_the_network](#)), continue with Preparing the disks ([/wiki/Handbook:AMD64/Installation/Disks](#)).

Introduction to block devices

Block devices

Let's take a good look at disk-oriented aspects of Gentoo Linux and Linux in general, including block devices, partitions, and Linux filesystems. Once the ins and outs of disks are understood, partitions and filesystems can be established for installation.

To begin, let's look at block devices. SCSI and Serial ATA drives are both labeled under device handles such as: `/dev/sda`, `/dev/sdb`, `/dev/sdc`, etc. On more modern machines, PCI Express based NVMe solid state disks have device handles such as `/dev/nvme0n1`, `/dev/nvme0n2`, etc.

The following table will help readers determine where to find a certain type of block device on the system:

Type of device	Default device handle	Editorial notes and considerations
IDE, SATA, SAS, SCSI, or USB flash	<code>/dev/sda</code>	Found on hardware from roughly 2007 until the present, this device handle is perhaps the most commonly used in Linux. These types of devices can be connected via the SATA bus (https://en.wikipedia.org/wiki/Serial_ATA), SCSI (https://en.wikipedia.org/wiki/SCSI), USB (https://en.wikipedia.org/wiki/USB) bus as block storage. As example, the first partition on the first SATA device is called <code>/dev/sda1</code> .
NVM Express (NVMe)	<code>/dev/nvme0n1</code>	The latest in solid state technology, NVMe (https://en.wikipedia.org/wiki/NVM_Express) drives are connected to the PCI Express bus and have the fastest transfer block speeds on the market. Systems from around 2014 and newer may have support for NVMe hardware. The first partition on the first NVMe device is called <code>/dev/nvme0n1p1</code> .

MMC, eMMC, and SD	<code>/dev/mmcblk0</code>	embedded MMC (https://en.wikipedia.org/wiki/MultiMediaCard#eMMC) devices, SD cards, and other types of memory cards can be useful for data storage. That said, many systems may not permit booting from these types of devices. It is suggested to not use these devices for active Linux installations; rather consider using them to transfer files, which is their typical design intention. Alternatively this storage type could be useful for short-term file backups or snapshots.
VirtIO	<code>/dev/vda</code>	Virtualized (/wiki/Virtualization) devices are found only within a QEMU (/wiki/QEMU) virtual emulator. <code>virtio_blk</code> driver provides this access to host's allocated drive space for this virtual machine. That said, it is a great way to try out Gentoo inside a virtual machine.

The block devices above represent an abstract interface to the disk. User programs can use these block devices to interact with the disk without worrying about whether the drives are SATA, SCSI, or something else. The program can simply address the storage on the disk as a bunch of contiguous, randomly-accessible 4096-byte (4K) blocks.

Partition tables

Although it is theoretically possible to use a raw, unpartitioned disk to house a Linux system (when creating a btrfs RAID for example), this is almost never done in practice. Instead, disk block devices are split up into smaller, more manageable block devices. On **amd64** systems, these are called partitions. There are currently two standard partitioning technologies in use: MBR (sometimes also called DOS disklabel) and GPT; these are tied to the two boot process types: legacy BIOS boot and UEFI.

GUID Partition Table (GPT)

The *GUID Partition Table (GPT)* setup (also called GPT disklabel) uses 64-bit identifiers for the partitions. The location in which it stores the partition information is much bigger than the 512 bytes of the MBR partition table (DOS disklabel), which means there is practically no limit on the number of partitions for a GPT disk. Also, the maximum partition size is much larger (almost 8 ZiB -- yes, zebibytes).

When a system's software interface between the operating system and firmware is UEFI (instead of BIOS), GPT is almost mandatory as compatibility issues will arise with DOS disklabel.

GPT also takes advantage of checksumming and redundancy. It carries CRC32 checksums to detect errors in the header and partition tables and has a backup GPT at the end of the disk. This backup table can be used to recover damage of the primary GPT near the beginning of the disk.

❗ Important

There are a few caveats regarding GPT:

- Using GPT on a BIOS-based computer works, but the user won't be able to dual-boot with a Microsoft Windows operating system, since Microsoft Windows refuses to boot from a GPT partition when in BIOS mode.

- Some buggy (old) motherboard firmware configured to boot in BIOS/CSM/legacy mode might also have problems with booting from GPT labeled disks.

Master boot record (MBR) or DOS boot sector

The *Master boot record* (https://en.wikipedia.org/wiki/Master_boot_record) boot sector (also called DOS boot sector, DOS disklabel, and - more recently, in contrast to GPT/UEFI setups - legacy BIOS boot) was first introduced in 1983 with PC DOS 2.x. MBR uses 32-bit identifiers for the start sector and length of the partitions, and supports three partition types: primary, extended, and logical. Primary partitions have their information stored in the master boot record itself - a very small (usually 512 bytes) location at the very beginning of a disk. Due to this small space, only four primary partitions are supported (for instance, `/dev/sda1` to `/dev/sda4`).

In order to support more partitions, one of the primary partitions in the MBR can be marked as an *extended* partition. This partition can then contain additional logical partitions (partitions within a partition).

❗ Important

Although still supported by most motherboard manufacturers, MBR boot sectors and their associated partitioning limitations are considered legacy. Unless working with hardware that is pre-2010, it best to partition a disk with GUID Partition Table (https://en.wikipedia.org/wiki/GUID_Partition_Table). Readers who must proceed with setup type should knowingly acknowledge the following information:

- Most post-2010 motherboards consider using MBR boot sectors a legacy (supported, but not ideal) boot mode.
- Due to using 32-bit identifiers, partition tables in the MBR cannot address storage space that is larger than 2 TiBs in size.
- Unless an extended partition is created, MBR supports a maximum of four partitions.
- This setup does not provide a backup boot sector, so if something overwrites the partition table, all partition information will be lost.

That said, MBR and legacy BIOS boot may still be used in virtualized cloud environments such as AWS.

The Handbook authors suggest using GPT whenever possible for Gentoo installations.

Advanced storage

The official Gentoo boot media provides support for many advanced filesystem and tool setups, which offer more flexible changes, snapshots and some cases more caching abilities.

Although usage is not covered in the handbook, below is a list of helpful guides to get the system running:

- Btrfs (/wiki/Btrfs/Native_System_Root_Guide)
- ZFS (</wiki/ZFS/rootfs>)
- LVM (Logical Volume Manager) (</wiki/LVM>)

Disk encryption is also handled in the same manner:

- Rootfs encryption (/wiki/Rootfs_encryption)

Default partitioning scheme

Throughout the remainder of the handbook, we will discuss and explain two cases:

1. UEFI firmware with GUID Partition Table (GPT) disk.
2. MBR DOS/legacy BIOS firmware with a MBR partition table disk.

While it is possible to mix and match boot types with certain motherboard firmware, mixing goes beyond the intention of the handbook. As previously stated, it is strongly recommended for installations on modern hardware to use UEFI boot with a GPT disklabel disk.

The following partitioning scheme will be used as a simple example layout.

 **Important**

The first row of the following table contains exclusive information for *either* a GPT disklabel *or* a MBR DOS/legacy BIOS disklabel. When in doubt, proceed with GPT, since **amd64** machines manufactured after the year 2010 generally support UEFI firmware and GPT boot sector.

Partition	Filesystem	Size	Description
/dev/ sda1	fat32	1 GiB	EFI System Partition details.
	xfs		MBR DOS/legacy BIOS boot partition details.
/dev/ sda2	linux-swap	RAM size * 2	Swap partition details.
/dev/ sda3	xfs	Remainder of the disk	Root partition details.

If this suffices as information, the advanced reader can directly skip ahead to the actual partitioning.

Both **fdisk** and **parted** are partitioning utilities included within the official Gentoo live image environments. **fdisk** is well known, stable, and handles both MBR and GPT disks. **parted** was one of the first Linux block device management utilities to support GPT partitions. It can be used as an alternative to **fdisk** if the reader prefers, however the handbook will only provide instructions for **fdisk**, since it is commonly available on most Linux environments.

Before going to the creation instructions, the first set of sections will describe in more detail how partitioning schemes can be created and mention some common pitfalls.

Designing a partition scheme

How many partitions and how big?

The design of disk partition layout is highly dependent on the demands of the system and the file system(s) applied to the device. If there are lots of users, then it is advised to have **/home** on a separate partition which will increase security and make backups and other types of maintenance easier. If Gentoo is being installed to perform as a mail server, then **/var** should be a separate partition as all mails are stored inside the **/var** directory. Game servers may have a separate **/opt** partition since most gaming server software is installed therein. The reason for these recommendations is similar to the **/home** directory: security, backups, and maintenance.

In most situations on Gentoo, **/usr** and **/var** should be kept relatively large in size. **/usr** hosts the majority of applications available on the system and the Linux kernel sources (under **/usr/src**). By default, **/var** hosts the Gentoo ebuild repository (located at **/var/db/repos/gentoo**) which, depending on the file system, generally consumes around 650 MiB of disk space. This space estimate *excludes* the **/var/cache/distfiles** and **/var/cache/binpkgs** directories, which will gradually fill with source files and (optionally) binary packages respectively as they are added to the system.

How many partitions and how big very much depends on considering the trade-offs and choosing the best option for the circumstance. Separate partitions or volumes have the following advantages:

- Choose the best performing filesystem for each partition or volume.
- The entire system cannot run out of free space if one defunct tool is continuously writing files to a partition or volume.
- If necessary, file system checks are reduced in time, as multiple checks can be done in parallel (although this advantage is realized more with multiple disks than it is with multiple partitions).
- Security can be enhanced by mounting some partitions or volumes read-only, `nosuid` (setuid bits are ignored), `noexec` (executable bits are ignored), etc.

However, multiple partitions have certain disadvantages as well:

- If not configured properly, the system might have lots of free space on one partition and little free space on another.
- A separate partition for **/usr/** may require the administrator to boot with an `initramfs` to mount the partition before other boot scripts start. Since the generation and maintenance of an `initramfs` is beyond the scope of this handbook, **we recommend that newcomers do not use a separate partition for /usr/**.
- There is also a 15-partition limit for SCSI and SATA unless the disk uses GPT labels.

Note

Installations that intend to use `systemd` as the service and `init` system must have the /

usr directory available at boot, either as part of the root filesystem or mounted via an initramfs.

What about swap space?

Recommendations for swap space size

RAM size	Suspend support?	Hibernation support?
2 GB or less	4GB	4GB
2 to 8 GB	RAM amount	2 * RAM
8 to 64 GB	8 GB minimum, 16 maximum	1.5 * RAM
64 GB or greater	8 GB minimum	Hibernation not recommended!

There is no perfect value for swap space size. The purpose of the space is to provide disk storage to the kernel when internal dynamic memory (RAM) is under pressure. A swap space allows for the kernel to move memory pages that are not likely to be accessed soon to disk (swap or page-out), which will free memory in RAM for the current task. Of course, if the pages swapped to disk are suddenly needed, they will need to be put back in memory (page-in) which will take considerably longer than reading from RAM (as disks are very slow compared to internal memory).

When a system is not going to run memory intensive applications or has lots of RAM available, then it probably does not need much swap space. However do note in case of hibernation that swap space is used to store *the entire contents of memory* (likely on desktop and laptop systems rather than on server systems). If the system requires support for hibernation, then swap space larger than or equal to the amount of memory is necessary.

As a general rule for RAM amounts less than 4 GB, the swap space size is recommended to be twice the internal memory (RAM). For systems with multiple hard disks, it is wise to create one swap partition on each disk so that they can be utilized for parallel read/write operations. The faster a disk can swap, the faster the system will run when data in swap space must be accessed. When choosing between rotational and solid state disks, it is better for performance to put swap on the solid state hardware.

It is worth noting that swap *files* can be used as an alternative to swap *partitions*; this is mostly helpful for systems with very limited disk space.

💡 Tip

With the rise in popularity with zram ([/wiki/Zram](#)), it should be noted that Gentoo is more likely to suffer with performance hits, as compiling uses large amounts of RAM, and programs can't be directly compiled in swap. It's usually better to create a smaller zram as a cache swap area with higher priority over the disk based swap of around 512MB, if a

user really wants to use zram to benefit their desktop needs and not harm compile times.

What is the EFI System Partition (ESP)?

When installing Gentoo on a system that uses UEFI to boot the operating system (instead of BIOS) it is essential that an EFI System Partition (ESP) is created. The instructions below contain the necessary pointers to correctly handle this operation. **The EFI system partition is not required when booting in BIOS/Legacy mode.**

The ESP *must* be a FAT variant (sometimes shown as *vfat* on Linux systems). The official UEFI specification (http://www.uefi.org/sites/default/files/resources/UEFI%20_5.pdf) denotes FAT12, 16, or 32 filesystems will be recognized by the UEFI firmware, although FAT32 is recommended for the ESP. After partitioning, format the ESP accordingly:

```
root # mkfs.fat -F 32 /dev/sda1
```

❗ Important

If the ESP is not formatted with a FAT variant, the system's UEFI firmware will not find the bootloader (or Linux kernel) and will most likely be unable to boot the system!

What is the BIOS boot partition?

A *BIOS boot partition* is a very small (1 to 2 MB) partition in which boot loaders like GRUB2 can put additional data that doesn't fit in the allocated storage. It is only needed when a disk is formatted with a GPT disklabel, but the system's firmware will be booting via GRUB2 in legacy BIOS/MBR DOS boot mode. **It is not required when booting in EFI/UEFI mode, and also not required when using a MBR/Legacy DOS disklabel.** A *BIOS boot partition* will not be used in this guide.

Partitioning the disk with GPT for UEFI

The following parts explain how to create an example partition layout for a single GPT disk device which will conform to the UEFI Specification and Discoverable Partitions Specification (DPS). DPS is a specification provided as part of the Linux Userspace API (UAPI) Group Specification and is recommended, but entirely optional. The specifications are implemented using the **fdisk** utility, which is part of the `sys-apps/util-linux` (<https://packages.gentoo.org/packages/sys-apps/util-linux>) $\supset$ package.

The table provides a recommended default for a trivial Gentoo installation. Additional partitions can be added according to personal preference or system design goals.

Device path (sysfs)	Mount point	File system	DPS UUID (<i>Type-UUID</i>)	Description
<code>/dev/sda1</code>	<code>/efi</code>	vfat	c12a7328-f81f-11d2-ba4b-00a0c93ec93b	EFI system partition (ESP) details.
<code>/dev/sda2</code>	N/A.	swap	0657fd6d-a4ab-43c4-84e5-0933c84b4f4f	Swap partition details.

/dev/sda3	/	xfs	4f68bce3-e8cd-4db1-96e7-fbcaf984b709	Root partition details.
-----------	---	-----	--------------------------------------	-------------------------

Viewing the current partition layout

fdisk is a popular and powerful tool to split a disk into partitions. Fire up **fdisk** against the disk (in our example, we use **/dev/sda**):

```
root # fdisk /dev/sda
```

Use the **p** key to display the disk's current partition configuration:

Command (m for help): p

```
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: HGST HTS721010A9
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: gpt
Disk identifier: 3E56EE74-0571-462B-A992-9872E3855D75

<!--T:236-->
Device            Start          End      Sectors   Size Type
/dev/sda1          2048          2099199    2097152    1G EFI System
/dev/sda2        2099200        10487807    8388608    4G Linux swap
/dev/sda3        10487808       1953523711 1943035904 926.5G Linux root (x86-64)
```

This particular disk was configured to house two Linux filesystems (each with a corresponding partition listed as "Linux") as well as a swap partition (listed as "Linux swap").

Creating a new disklabel / removing all partitions

Pressing the **g** key will instantly remove all existing disk partitions and create a new GPT disklabel:

Command (m for help): g

```
Created a new GPT disklabel (GUID: 3E56EE74-0571-462B-A992-9872E3855D75).
```

Alternatively, to keep an existing GPT disklabel (see the output of **p** above), consider removing the existing partitions one by one from the disk. Press **d** to delete a partition. For instance, to delete an existing **/dev/sda1**:

Command (m for help): d

```
Partition number (1-4): 1
```

The partition has now been scheduled for deletion. It will no longer show up when printing the list of partitions (**p**), but it will not be erased until the changes have been saved. This allows users to abort the operation if a mistake was made - in that case, press **q** immediately and hit **Enter** and the partition will not be deleted.

Repeatedly press **p** to print out a partition listing and then press **d** and the number of the partition to delete it. Eventually, the partition table will be empty:

Command (m for help): p

```
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: HGST HTS721010A9
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: gpt
Disk identifier: 3E56EE74-0571-462B-A992-9872E3855D75
```

Now that the in-memory partition table is empty, we're ready to create the partitions.

Creating the EFI System Partition (ESP)

Note

A smaller ESP is possible but not recommended, especially given it may be shared with other OSes.

First create a small EFI system partition, which will also be mounted as `/efi`. Type `n` to create a new partition, followed by `1` to select the first partition. When prompted for the first sector, make sure it starts from 2048 (which may be needed for the boot loader) and hit `Enter`. When prompted for the last sector, type `+1G` to create a partition 1 gibibyte in size:

Command (m for help): n

```
Partition number (1-128, default 1): 1
First sector (2048-1953525134, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-1953525134, default
1953523711): +1G

Created a new partition 1 of type 'Linux filesystem' and of size 1 GiB.
Partition #1 contains a vfat signature.

<!--T:247-->
Do you want to remove the signature? [Y]es/[N]o: Y
The signature will be removed by a write command.
```

Mark the partition as an EFI system partition:

Command (m for help): t

```
Selected partition 1
Partition type or alias (type L to list all): 1
Changed type of partition 'Linux filesystem' to 'EFI System'.
```

Creating the swap partition

Next, to create the swap partition, press `n` to create a new partition, then press `2` to create the second partition, `/dev/sda2`. When prompted for the first sector, hit `Enter`. When prompted for the last sector, type `+4G` (or any other size needed for the swap space) to create a partition 4 GiB in size.

Command (m for help): n


```
Partition number (2-128, default 2):  
First sector (2099200-1953525134, default 2099200):  
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2099200-1953525134, default  
1953523711): +4G
```

```
Created a new partition 2 of type 'Linux filesystem' and of size 4 GiB.
```

After this, press **t** to set the partition type, **2** to select the partition just created and then type in 19 to set the partition type to "Linux Swap".

Command (m for help): t

```
Partition number (1,2, default 2): 2  
Partition type or alias (type L to list all): 19  
  
Changed type of partition 'Linux filesystem' to 'Linux swap'.
```

Creating the root partition

Finally, to create the root partition, press **n** to create a new partition, and then press **3** to create the third partition: **/dev/sda3**. When prompted for the first sector, press **Enter**. When prompted for the last sector, hit **Enter** to create a partition that takes up the rest of the remaining space on the disk.

Command (m for help): n

```
Partition number (3-128, default 3): 3  
First sector (10487808-1953525134, default 10487808):  
Last sector, +/-sectors or +/-size{K,M,G,T,P} (10487808-1953525134, default  
1953523711):  
  
<!--T:238-->  
Created a new partition 3 of type 'Linux filesystem' and of size 926.5  
GiB..
```

Note

Setting the root partition's type to "Linux root (x86-64)" is not required and the system will function normally if it is set to the "Linux filesystem" type. This filesystem type is only necessary for cases where a bootloader that supports it (i.e. systemd-boot) is used and a fstab file is not wanted.

After creating the root partition, press **t** to set the partition type, **3** to select the partition just created, and then type in 23 to set the partition type to "Linux Root (x86-64)".

Command(m for help): t

```
Partition number (1-3, default 3): 3  
Partition type or alias (type L to list all): 23  
  
<!--T:239-->  
Changed type of partition 'Linux filesystem' to 'Linux root (x86-64)'
```

After completing these steps, pressing **p** should display a partition table that looks similar to the following:

Command (m for help): p

```
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: HGST HTS721010A9
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: gpt
Disk identifier: 3E56EE74-0571-462B-A992-9872E3855D75

<!--T:240-->
Device          Start      End      Sectors  Size Type
/dev/sda1        2048      2099199  2097152   1G EFI System
/dev/sda2     2099200   10487807  8388608   4G Linux swap
/dev/sda3    10487808  1953523711 1943035904 926.5G Linux root (x86-64)

<!--T:241-->
Filesystem/RAID signature on partition 1 will be wiped.
```

Saving the partition layout

Press **w** to save the partition layout and exit the **fdisk** utility:

Command (m for help): w

```
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

With partitions now available, the next installation step is to fill them with filesystems.

Partitioning the disk with MBR for BIOS / legacy boot

The following table provides a recommended partition layout for a trivial MBR DOS / legacy BIOS boot installation. Additional partitions can be added according to personal preference or system design goals.

Device path (sysfs)	Mount point	File system	DPS UUID (PARTUUID)	Description
/dev/sda1	/boot	xfs	N/A	MBR DOS / legacy BIOS boot partition details.
/dev/sda2	N/A.	swap	0657fd6d-a4ab-43c4-84e5-0933c84b4f4f	Swap partition details.
/dev/sda3	/	xfs	4f68bce3-e8cd-4db1-96e7-fbcaf984b709	Root partition details.

Change the partition layout according to personal preference.

Viewing the current partition layout

Fire up **fdisk** against the disk (in our example, we use **/dev/sda**):

```
root # fdisk /dev/sda
```

Use the **p** key to display the disk's current partition configuration:

Command (m for help): p

```
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: HGST HTS721010A9
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: dos
Disk identifier: 0xf163b576

<!--T:242-->
Device      Boot      Start          End      Sectors      Size Id Type
/dev/sda1    *           2048       2099199       2097152        1G 83 Linux
/dev/sda2                2099200       10487807       8388608        4G 82 Linux swap /
Solaris
/dev/sda3                10487808  1953525167  1943037360  926.5G 83 Linux
```

This particular disk was until now configured to house two Linux filesystems (each with a corresponding partition listed as "Linux") as well as a swap partition (listed as "Linux swap"), using a GPT table.

Creating a new disklabel / removing all partitions

Pressing **o** will instantly remove all existing disk partitions and create a new MBR disklabel (also named DOS disklabel):

Command (m for help): o

```
Created a new DOS disklabel with disk identifier 0xf163b576.
The device contains 'gpt' signature and it will be removed by a write
command. See fdisk(8) man page and --wipe option for more details.
```

Alternatively, to keep an existing DOS disklabel (see the output of **p** above), consider removing the existing partitions one by one from the disk. Press **d** to delete a partition. For instance, to delete an existing **/dev/sda1**:

Command (m for help): d

```
Partition number (1-4): 1
```

The partition has now been scheduled for deletion. It will no longer show up when printing the list of partitions (**p**), but it will not be erased until the changes have been saved. This allows users to abort the operation if a mistake was made - in that case, type **q** immediately and hit **Enter** and the partition will not be deleted.

Repeatedly press **p** to print out a partition listing and then press **d** and the number of the partition to delete it. Eventually, the partition table will be empty:

Command (m for help): p

```
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: HGST HTS721010A9
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: dos
Disk identifier: 0xf163b576
```

The disk is now ready to create new partitions.

Creating the boot partition

First, create a small partition which will be mounted as **/boot**. Press **n** to create a new partition, followed by **p** for a *primary* partition and **1** to select the first primary partition. When prompted for the first sector, make sure it starts from 2048 (which may be needed for the boot loader) and press **Enter**. When prompted for the last sector, type +1G to create a partition 1 GB in size:

Command (m for help): n

```
Partition type
  p   primary (0 primary, 0 extended, 4 free)
  e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-1953525167, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-1953525167, default
1953525167): +1G

<!--T:243-->
Created a new partition 1 of type 'Linux' and of size 1 GiB.
```

Mark the partition as bootable by pressing the **a** key and pressing **Enter**:

Command (m for help): a

```
Selected partition 1
The bootable flag on partition 1 is enabled now.
```

Note: if more than one partition is available on the disk, then the partition to be flagged as bootable will have to be selected.

Creating the swap partition

Next, to create the swap partition, press **n** to create a new partition, then **p**, then type **2** to create the second primary partition, **/dev/sda2**. When prompted for the first sector, press **Enter**. When prompted for the last sector, type +4G (or any other size needed for the swap space) to create a partition 4GB in size.

Command (m for help): n

```
Partition type
  p   primary (1 primary, 0 extended, 3 free)
  e   extended (container for logical partitions)
Select (default p): p
Partition number (2-4, default 2): 2
First sector (2099200-1953525167, default 2099200):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2099200-1953525167, default
1953525167): +4G
```

Created a new partition 2 of type 'Linux' and of size 4 GiB.

After all this is done, press **t** to set the partition type, **2** to select the partition just created and then type in 82 to set the partition type to "Linux Swap".

Command (m for help): t

```
Partition number (1,2, default 2): 2
Hex code (type L to list all codes): 82

<!--T:244-->
Changed type of partition 'Linux' to 'Linux swap / Solaris'.
```

Creating the root partition

Finally, to create the root partition, press **n** to create a new partition. Then press **p** and **3** to create the third primary partition, **/dev/sda3**. When prompted for the first sector, hit **Enter**. When prompted for the last sector, hit **Enter** to create a partition that takes up the rest of the remaining space on the disk:

Command (m for help): n

```
Partition type
  p   primary (2 primary, 0 extended, 2 free)
  e   extended (container for logical partitions)
Select (default p): p
Partition number (3,4, default 3): 3
First sector (10487808-1953525167, default 10487808):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (10487808-1953525167, default
1953525167):

<!--T:245-->
Created a new partition 3 of type 'Linux' and of size 926.5 GiB.
```

After completing these steps, pressing **p** should display a partition table that looks similar to this:

Command (m for help): p

```
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: HGST HTS721010A9
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: dos
Disk identifier: 0xf163b576
```

```
<!--T:246-->
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	2048	2099199	2097152	1G	83	Linux
/dev/sda2		2099200	10487807	8388608	4G	82	Linux swap / Solaris
/dev/sda3		10487808	1953525167	1943037360	926.5G	83	Linux

Saving the partition layout

Press **w** to write the partition layout and exit **fdisk**:

Command (m for help): w

```
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

Now it is time to put filesystems on the partitions.

Creating file systems

⚠ Warning

When using an SSD or NVMe drive, it is wise to check for firmware upgrades. Some Intel SSDs in particular (600p and 6000p) require a firmware upgrade for possible data corruption (https://bugzilla.redhat.com/show_bug.cgi?id=1402533) induced by XFS I/O

usage patterns. The problem is at the firmware level and not any fault of the XFS filesystem. The **smartctl** utility can help check the device model and firmware version.

Introduction

Now that the partitions have been created, it is time to place a filesystem on them. In the next section the various file systems that Linux supports are described. Readers that already know which filesystem to use can continue with Applying a filesystem to a partition (/wiki/Handbook:AMD64/Installation/Disks#Applying_a_filesystem_to_a_partition). The others should read on to learn about the available filesystems...

Filesystems

Linux supports several dozen filesystems, although many of them are only wise to deploy for specific purposes. Only certain filesystems may be found stable on the amd64 architecture - it is advised to read up on the filesystems and their support state before selecting a more experimental one for important partitions. **XFS is the recommended all-purpose, all-platform filesystem.** The below is a non-exhaustive list:

XFS (/wiki/XFS)

Filesystem with metadata journaling which comes with a robust feature-set and is optimized for scalability. It has been continuously upgraded to include modern features. The only downside is that XFS partitions cannot yet be shrunk, although this is being worked on. XFS notably supports reflinks and Copy on Write (CoW) which is particularly helpful on Gentoo systems because of the amount of compiles users complete. XFS is the recommended modern all-purpose all-platform filesystem. Requires a partition to be at least 300MB.

ext4 (/wiki/Ext4)

Ext4 is a reliable, all-purpose all-platform filesystem, although it lacks modern features like reflinks.

VFAT (/wiki/VFAT)

Also known as FAT32, is supported by Linux but does not support standard UNIX permission settings. It is mostly used for interoperability/interchange with other operating systems (Microsoft Windows or Apple's macOS) but is also a necessity for some system bootloader firmware (like UEFI). Users of UEFI systems will need an EFI System Partition (/wiki/EFI_System_Partition) formatted with VFAT in order to boot.

btrfs (/wiki/Btrfs)

Newer generation filesystem. Provides advanced features like snapshotting, self-healing through checksums, transparent compression, subvolumes, and integrated RAID. Kernels prior to 5.4.y are not guaranteed to be safe to use with btrfs in production because fixes for serious issues are only present in the more recent releases of the LTS kernel branches. RAID 5/6 and quota groups unsafe on all versions of btrfs.

F2FS (/wiki/F2FS)

The Flash-Friendly File System was originally created by Samsung for the use with NAND flash memory. It is a decent choice when installing Gentoo to microSD cards, USB drives, or other flash-based storage devices.

NTFS (/wiki/NTFS)

This "New Technology" filesystem is the flagship filesystem of Microsoft Windows since Windows NT 3.1. Similarly to VFAT, it does not store UNIX permission settings or extended attributes necessary for BSD or Linux to function properly, therefore it should

not be used as a root filesystem for most cases. It should *only* be used for interoperability or data interchange with Microsoft Windows systems (note the emphasis on *only*).

ZFS (/wiki/ZFS)

❗ Important: ZFS pools can be created on the minimal installation CD, for further information, refer to ZFS/rootfs (/wiki/ZFS/rootfs)

Next generation file system created by Matthew Ahrens and Jeff Bonwick. It was designed around a few key ideas: Administration of storage should be simple, redundancy should be handled by the filesystem, file systems should never be taken offline for repair, automated simulations of worst case scenarios before shipping code is important, and data integrity is paramount.

More extensive information on filesystems can be found in the community maintained Filesystem article (/wiki/Filesystem).

Applying a filesystem to a partition

📌 Note

Please make sure to emerge the relevant user space utilities package for the chosen filesystem before rebooting. There will be a reminder to do so near the end of the installation process.

To create a filesystem on a partition or volume, there are user space utilities available for each possible filesystem. Click the filesystem's name in the table below for additional information on each filesystem:

Filesystem	Creation command	Within the live environment?	Package
XFS (/wiki/XFS)	mkfs.xfs	✔ Yes	sys-fs/xfsprogs (https://packages.gentoo.org/packages/sys-fs/xfsprogs)
ext4 (/wiki/Ext4)	mkfs.ext4	✔ Yes	sys-fs/e2fsprogs (https://packages.gentoo.org/packages/sys-fs/e2fsprogs)
VFAT (/wiki/VFAT) (FAT32, ...)	mkfs.vfat	✔ Yes	sys-fs/dosfstools (https://packages.gentoo.org/packages/sys-fs/dosfstools)
btrfs (/wiki/Btrfs)	mkfs.btrfs	✔ Yes	sys-fs/btrfs-progs (https://packages.gentoo.org/packages/sys-fs/btrfs-progs)
F2FS (/wiki/F2FS)	mkfs.f2fs	✔ Yes	sys-fs/f2fs-tools (https://packages.gentoo.org/packages/sys-fs/f2fs-tools)
NTFS (/wiki/NTFS)	mkfs.ntfs	✔ Yes	sys-fs/ntfs3g (https://packages.gentoo.org/packages/sys-fs/ntfs3g)
ZFS (/wiki/ZFS)	zpool create ...	✔ Yes	sys-fs/zfs (https://packages.gentoo.org/packages/sys-fs/zfs)

❗ Important

The handbook recommends new partitions as part of the installation process, but it is important to note running any **mkfs** command will erase any data contained within the partition. When necessary, ensure any data that exists within is appropriately backed up before creating a new filesystem.

For instance, to have the root partition (**/dev/sda3**) as xfs as used in the example partition structure, the following commands would be used:

```
root # mkfs.xfs -c options=/usr/share/xfsprogs/mkfs/lts_6.18.conf /dev/sda3
```

EFI system partition filesystem

The EFI system partition (**/dev/sda1**) must be formatted as FAT32:

```
root # mkfs.vfat -F 32 /dev/sda1
```

Legacy BIOS boot partition filesystem

Systems booting via legacy BIOS with a MBR/DOS disklabel can use any filesystem format supported by the bootloader.

For example, to format with XFS:

```
root # mkfs.xfs -c options=/usr/share/xfsprogs/mkfs/lts_6.18.conf /dev/sda1
```

Small ext4 partitions

When using the ext4 filesystem on a small partition (less than 8 GiB), the filesystem should be created with the proper options to reserve enough inodes. This can be specified using the **-T small** option:

```
root # mkfs.ext4 -T small /dev/<device>
```

Doing so will quadruple the number of inodes for a given filesystem, since its "bytes-per-inode" reduces from one every 16kB to one every 4kB.

Activating the swap partition

mkswap is the command that is used to initialize swap partitions:

```
root # mkswap /dev/sda2
```

📌 Note

Installations which were previously started, but did not finish the installation process can resume the installation from this point in the handbook. Use this link as the permalink: [Resumed installations start here \(/wiki/Handbook:AMD64/Installation/Disks#Resumed_installations_start_here\)](#).

To activate the swap partition, use **swapon**:

```
root # swapon /dev/sda2
```

This 'activation' step is only necessary because the swap partition is newly created within the live environment. Once the system has been rebooted, as long as the swap partition is properly defined within `fstab` or other mount mechanism, swap space will activate automatically.

Mounting the root partition

Certain live environments may be missing the suggested mount point for Gentoo's root partition (`/mnt/gentoo`), or mount points for additional partitions created in the partitioning section:

```
root # mkdir --parents /mnt/gentoo
```

Continue creating additional mount points necessary for any additional (custom) partition(s) created during previous steps by using the **mkdir** command.

With mount points created, it is time to make the partitions accessible via **mount** command.

Mount the root partition:

```
root # mount /dev/sda3 /mnt/gentoo
```

For EFI installs only, the ESP should be mounted under the root partition location:

```
root # mkdir --parents /mnt/gentoo/efi
```

Continue mounting additional (custom) partitions as necessary using the **mount** command.

Note

If `/tmp/` needs to reside on a separate partition, be sure to change its permissions after mounting:

```
root # chmod 1777 /mnt/gentoo/tmp
```

This also holds for `/var/tmp`.

Later in the instructions, the `proc` filesystem (a virtual interface with the kernel) as well as other kernel pseudo-file systems will be mounted. But first the Gentoo stage file (`/wiki/Handbook:AMD64/Installation/Stage`) must be extracted.

Choosing a stage file

Tip

On supported architectures, it is recommended for users targeting a desktop (graphical) operating system environment to use a stage file with the term `desktop` within the name. These files include packages such as `llvm-core/llvm` (<https://packages.gentoo.org/packages/llvm-core/llvm>)  and `dev-lang/rust-bin` (<https://packages.gentoo.org/packages/dev-lang/rust-bin>)  and USE flag tuning which will greatly improve install time.

The stage file (/wiki/Stage_file) acts as the seed of a Gentoo install. Stage files are generated by the Release Engineering Team (</wiki/Project:RelEng>) with Catalyst (</wiki/Catalyst>). Stage files are based on specific profiles ([/wiki/Profile_\(Portage\)](/wiki/Profile_(Portage))), and contain an almost-complete system.

When choosing a stage file, it's important to pick one with profile targets corresponding to the desired system type.

❗ Important

While it's possible to make major profile changes after an installation has been established, switching requires substantial effort and consideration, and is outside the scope of this installation manual. Switching init systems is difficult, but switching from `no-multilib` to `multilib` requires extensive Gentoo and low-level toolchain knowledge.

💡 Tip

Most users should not need to use the 'advanced' tarballs options; they are for atypical or advanced software or hardware configurations.

OpenRC

OpenRC (</wiki/OpenRC>) is a dependency-based init system (responsible for starting up system services once the kernel has booted) that maintains compatibility with the system provided init program, normally located in `/sbin/init`. It is Gentoo's native and original init system, but is also deployed by a few other Linux distributions and BSD systems.

systemd

systemd is a modern SysV-style init and rc replacement for Linux systems. It is used as the primary init system by a majority of Linux distributions. systemd is fully supported in Gentoo and works for its intended purpose. If something seems lacking in the Handbook for a systemd install path, review the systemd article (</wiki/Systemd>) *before* asking for support.

Multilib (32 and 64-bit)

📌 Note

Not every architecture has a multilib option. Many only run with native code. Multilib is most commonly applied to **amd64**.

The multilib profile uses 64-bit libraries when possible, and only falls back to the 32-bit versions when strictly necessary for compatibility. This is an excellent option for the majority of installations because it provides a great amount of flexibility for customization in the future.

✔ Tip

Using `multilib` targets makes it easier to switch profiles later, compared to `no-multilib`

No-multilib (pure 64-bit)

⚠ Warning

Readers who are just starting out with Gentoo should *not* choose a no-multilib tarball unless it is absolutely necessary. Migrating from a no-multilib to a multilib system requires an extremely well-working knowledge of Gentoo and the lower-level toolchain (it may even cause our Toolchain developers (/wiki/Project:Toolchain) to shudder a little). It is not for the faint of heart and is beyond the scope of this guide.

Selecting a no-multilib tarball to be the base of the system provides a complete 64-bit operating system environment - free of 32-bit software. This effectively renders the ability to switch to `multilib` profiles burdensome, although still technically possible.

Downloading the stage file

Before downloading the *stage file*, the current directory should be set to the location of the mount used for the install:

```
root # cd /mnt/gentoo
```

Setting the date and time

Stage archives are generally obtained using HTTPS which requires relatively accurate system time. Clock skew can prevent downloads from working, and can cause unpredictable errors if the system time is adjusted by any considerable amount after installation.

The current date and time can be verified with **date**:

```
root # date
```

```
Mon Oct  3 13:16:22 UTC 2021
```

If the displayed date and time is more than few minutes off, it should be updated using one of the following methods.

Automatic

Using `Network_Time_Protocol` (/wiki/Network_Time_Protocol) to correct clock skew is typically easier and more reliable than manually setting the system clock.

chronyd, part of

`net-misc/chrony` (<https://packages.gentoo.org/packages/net-misc/chrony>) [Ⓐ] can be used to update the system clock to UTC with:

```
root # chronyd -q
```

❗ Important

Systems without a functioning Real-Time Clock (RTC) must sync the system clock at every

system start, and on regular intervals thereafter. This is also beneficial for systems with a RTC, as the battery could fail, and clock skew can accumulate.

⚠ Warning

Standard NTP traffic is not authenticated, it is important to verify time data obtained from the network.

Manual

When NTP access is unavailable, **date** can be used to manually set the system clock.

📌 Note

UTC time is recommended for all Linux systems. Later, a system timezone is defined, which changes the offset when the date is displayed.

The following argument format is used to set the time: **MMDDhhmmYYYY** syntax (**M**onth, **D**ay, **h**our, **m**inute and **Y**ear).

For instance, to set the date to October 3rd, 13:16 in the year 2021, issue:

```
root # date 100313162021
```

Graphical browsers

Those using environments with fully graphical web browsers will have no problem copying a stage file URL from the main website's download section (<https://www.gentoo.org/downloads/#other-arches>). Simply select the appropriate tab, right click the link to the stage file, then **Copy Link** to copy the link to the clipboard, then paste the link to the **wget** utility on the command-line to download the stage file:

```
root # wget <PASTED_STAGE_FILE_URL>
```

Command-line browsers

More traditional readers or 'old timer' Gentoo users, working exclusively from command-line may prefer using **links**

(`www-client/links` (<https://packages.gentoo.org/packages/www-client/links>) ) , a non-graphical, menu-driven browser. To download a stage, surf to the Gentoo mirror list like so:

```
root # links https://www.gentoo.org/downloads/mirrors/
```

To use an HTTP proxy with **links**, pass on the URL with the `-http-proxy` option:

```
root # links -http-proxy proxy.server.com:8080 https://www.gentoo.org/downloads/mirrors/
```

Next to **links** there is also the **lynx**

(`www-client/lynx` (<https://packages.gentoo.org/packages/www-client/lynx>) ) browser. Like **links** it is a non-graphical browser but it is not menu-driven.

```
root # lynx https://www.gentoo.org/downloads/mirrors/
```

If a proxy needs to be defined, export the `http_proxy` and/or `ftp_proxy` variables:

```
root # export http_proxy="http://proxy.server.com:port"
```

```
root # export ftp_proxy="http://proxy.server.com:port"
```

On the mirror list, select a mirror close by. Usually HTTP mirrors suffice, but other protocols are available as well. Move to the **releases/amd64/autobuilds/** directory. There all available stage files are displayed (they might be stored within subdirectories named after the individual sub-architectures). Select one and press **d** to download.

After the stage file download completes, it is possible to verify the integrity and validate the contents of the stage file. Those interested should proceed to the next section (/wiki/Handbook:AMD64/Installation/Stage#Verifying_and_validating).

Those not interested in verifying and validating the stage file can close the command-line browser by pressing **q** and can move directly to the Installing a stage file (/wiki/Handbook:AMD64/Installation/Stage#Installing_a_stage_file) section.

Verifying and validating

Like with the minimal installation CDs, additional downloads to verify and validate the stage file are available. The extra files are available under the root of the mirrors directory. Browse to the appropriate location for the hardware architecture and the system profile and download the associated **.CONTENTS.gz**, **.DIGESTS**, and **.sha256** files.

```
root # wget https://distfiles.gentoo.org/releases/
```

- **.CONTENTS.gz** - A compressed file that contains a list of all files inside the stage file.
- **.DIGESTS** - Contains checksums of the stage file in using several cryptographic hash algorithms.
- **.sha256** - Contains a PGP signed SHA256 checksum of the stage file. This file may not be available for download for all stage files.

Just like with the ISO file, the cryptographic signature of the **tar.xz** file must be verified using **gpg** to ensure no tampering has been performed on the tarball.

For official Gentoo live images, the

`sec-keys/openpgp-keys-gentoo-release` (<https://packages.gentoo.org/packages/sec-keys/openpgp-keys-gentoo-release>) package provides PGP signing keys for automated releases. The keys must first be imported into the user's session in order to be used for verification:

```
root # gpg --import /usr/share/openpgp-keys/gentoo-release.asc
```

For all non-official live images which offer **gpg** and **wget** in the live environment, refer to the earlier Verifying the downloaded files (/wiki/Handbook:AMD64/Installation/Media#Verifying_the_downloaded_files) section.

Verify the signature of the tarball and, optionally, associated checksum files:

```
root # gpg --verify stage3-amd64-<release>-<init>.tar.xz.asc stage3-amd64-<release>-<init>.tar.xz
```

```
root # gpg --output stage3-amd64-<release>-<init>.tar.xz.DIGESTS.verified --verify stage3-amd64-<release>-<init>.tar.xz.DIGESTS
```

```
root # gpg --output stage3-amd64-<release>-<init>.tar.xz.sha256.verified --verify stage3-amd64-<release>-<init>.tar.xz.sha256
```

If verification succeeds, "Good signature from" will be in the output of the previous command(s).

The fingerprints of the OpenPGP keys used for signing release media can be found on the release media signatures page (<https://www.gentoo.org/downloads/signatures/>).

Note

Most stages are now explicitly suffixed (<https://www.gentoo.org/news/2021/07/20/more-downloads.html>) with the init system type (**openrc** or **systemd**), although some architectures may still be missing these for now.

Cryptographic tools and utilities such as **openssl**, **sha256sum**, or **sha512sum** can be used to compare the output with the checksums provided by the **.DIGESTS** file.

To verify the SHA512 checksum with **openssl**:

```
root # openssl dgst -r -sha512 stage3-amd64-<release>-<init>.tar.xz
```

dgst instructs the **openssl** command to use the Message Digest sub-command, **-r** prints the digest output in coreutils format, and **-sha512** selects the SHA512 digest.

To verify the BLAKE2B512 checksum with **openssl**:

```
root # openssl dgst -r -blake2b512 stage3-amd64-<release>-<init>.tar.xz
```

Compare the output(s) of the checksum commands with the hash and filename paired values contained within the **.DIGESTS.verified** file. The paired values need to match the output of the checksum commands, otherwise the downloaded file is corrupt, and should be discarded and re-downloaded.

To verify the SHA256 hash from an associated **.sha256** file using the **sha256sum** utility:

```
root # sha256sum --check stage3-amd64-<release>-<init>.tar.xz.sha256.verified
```

The **--check** option instructs **sha256sum** to read a list of expected files and associated hashes, and then print an associated "OK" for each file that calculates correctly or a "FAILED" for files that do not.

Installing a stage file

Once the *stage file* has been downloaded and verified, it can be extracted using **tar**:

```
root # tar xpvf stage3-*.tar.xz --xattrs-include='*.*' --numeric-owner -C /mnt/gentoo
```

Before extracting verify the options:

- **x** **extract**, instructs **tar** to extract the contents of the archive.
- **p** **preserve permissions**.
- **v** **verbose output**.
- **f** **file**, provides **tar** with the name of the input archive.
- **--xattrs-include='*.*'** Preserves extended attributes in all namespaces stored in the archive.
- **--numeric-owner** Ensure that the user and group IDs of files being extracted from the tarball remain the same as Gentoo's release engineering team intended (even if adventurous users are not using official Gentoo live environments for the installation process).
- **-C /mnt/gentoo** Extract files to the root partition regardless of the current directory.

Now that the stage file is unpacked, proceed with Configuring compile options ([/wiki/Handbook:AMD64/Installation/Stage#Configuring_compile_options](#)).

Configuring compile options

Introduction

To optimize the system, it is possible to set variables which impact the behavior of Portage, Gentoo's officially supported package manager. All those variables can be set as environment variables (using **export**) but setting via **export** is not permanent.

❏ Note

Technically variables can be exported via the shell's ([/wiki/Shell](#)) profile or rc files, however that is not best practice for basic system administration.

Portage reads in the **make.conf** ([/wiki/Make.conf](#)) file when it runs, which will change runtime behavior depending on the values saved in the file. **make.conf** can be considered the primary configuration file for Portage, so treat its content carefully.

✔ Tip

A commented listing of all possible variables can be found in **/mnt/gentoo/usr/share/portage/config/make.conf.example**. Additional documentation on **make.conf** can be found by running **man 5 make.conf**.

For a successful Gentoo installation only the variables that are mentioned below need to be set.

Fire up an editor (in this guide we use **nano**) to alter the optimization variables we will discuss hereafter.

```
root # nano /mnt/gentoo/etc/portage/make.conf
```

From the **make.conf.example** file it is obvious how the file should be structured: commented lines start with **#**, other lines define variables using the **VARIABLE="value"** syntax. Several of those variables are discussed in the next section.

CFLAGS and CXXFLAGS

The **CFLAGS** and **CXXFLAGS** variables define the optimization flags for GCC C and C++ compilers respectively. Although those are defined generally here, for maximum performance one would need to optimize these flags for each program separately. The reason for this is because every program is different. However, this is not manageable, hence the definition of these flags in the **make.conf** file.

In **make.conf** one should define the optimization flags that will make the system the most responsive generally. Don't place experimental settings in this variable; too much optimization can make programs misbehave (crash, or even worse, malfunction).

The Handbook will not explain all possible optimization options. To understand them all, read the GNU Online Manual(s) (<https://gcc.gnu.org/onlinedocs/>) or the gcc info page (**info gcc**). The **make.conf.example** file itself also contains lots of examples and information; don't forget to read it too.

A first setting is the `-march=` or `-mtune=` flag, which specifies the name of the target architecture. Possible options are described in the `make.conf.example` file (as comments). A commonly used value is *native* as that tells the compiler to select the target architecture of the current system (the one users are installing Gentoo on).

A second one is the `-O` flag (that is a capital O, not a zero), which specifies the gcc optimization class flag. Possible classes are `s` (for size-optimized), `0` (zero - for no optimizations), `1`, `2` or even `3` for more speed-optimization flags (every class has the same flags as the one before, plus some extras). `-O2` is the recommended default. `-O3` is known to cause problems when used system-wide, so we recommend to stick to `-O2`.

Another popular optimization flag is `-pipe` (use pipes rather than temporary files for communication between the various stages of compilation). It has no impact on the generated code, but uses more memory. On systems with low memory, gcc might get killed. In that case, do not use this flag.

Using `-fomit-frame-pointer` (which doesn't keep the frame pointer in a register for functions that don't need one) might have serious repercussions on the debugging of applications.

When the `CFLAGS` and `CXXFLAGS` variables are defined, combine the several optimization flags in one string. The default values contained in the stage file archive should be good enough. The following one is just an example:

CODE Example `CFLAGS` and `CXXFLAGS` variables

```
# Compiler flags to set for all languages
COMMON_FLAGS="-march=native -O2 -pipe"
# Use the same settings for both variables
CFLAGS="${COMMON_FLAGS}"
CXXFLAGS="${COMMON_FLAGS}"
```

📌 Tip

Although the GCC optimization (/wiki/GCC_optimization) article has more information on how the various compilation options can affect a system, the Safe `CFLAGS` (/wiki/Safe_CFLAGS) article may be a more practical place for beginners to start optimizing their systems.

RUSTFLAGS

Many programs are now written in Rust which has its own way of optimising. By default Rust optimizes to level 3 on all release builds unless a project changes it so this should be left as is. A full optimization list (known as codegen) that can be passed to the rust compiler is available at <https://doc.rust-lang.org/rustc/codegen-options/index.html> (<https://doc.rust-lang.org/rustc/codegen-options/index.html>).

The most useful optimization would be to tell Rust to compile for your CPU using the following example:

FILE `/etc/portage/make.conf` **RUSTFLAGS** Example

```
RUSTFLAGS="${RUSTFLAGS} -C target-cpu=native"
```

To get a list of supported CPUs in Rust run:

```
root # rustc -C target-cpu=help
```

This will show what the default and also which CPU will be selected with native.

❏ Note

The above command only works on desktop stage3 tarballs or after emerging `dev-lang/rust-bin` (<https://packages.gentoo.org/packages/dev-lang/rust-bin>) [Ⓐ] or `dev-lang/rust` (<https://packages.gentoo.org/packages/dev-lang/rust>) [Ⓐ].

MAKEOPTS

The `MAKEOPTS` variable defines how many parallel compilations should occur when installing a package. As of Portage version 3.0.31^[1], if left undefined, Portage's default behavior is to set the `MAKEOPTS` jobs value to the same number of threads returned by `nproc`.

Further, as of Portage 3.0.53^[2], if left undefined, Portage's default behavior is to set the `MAKEOPTS` load-average value to the same number of threads returned by `nproc`.

A good choice is the smaller of: the number of threads the CPU has, or the total amount of system RAM divided by 2 GiB.

⚠ Warning

Using a large number of jobs can significantly impact memory consumption. A good recommendation is to have at least 2 GiB of RAM for every job specified (so, e.g. `-j6` requires *at least* 12 GiB). To avoid running out of memory, lower the number of jobs to fit the available memory.

💡 Tip

When using parallel emerges (`--jobs`), the effective number of jobs run can grow exponentially (up to make jobs multiplied by emerge jobs). This can be worked around by running a localhost-only distcc configuration that will limit the number of compiler instances per host.

FILE `/etc/portage/make.conf` Example `MAKEOPTS` declaration

```
# If left undefined, Portage's default behavior is to:
# - set the MAKEOPTS jobs value to the same number of threads returned by
`nproc`
# - set the MAKEOPTS load-average value slightly above the number of
threads returned by `nproc`, due to it being a damped value
# Please replace '4' as appropriate for the system (min(RAM/2GB, threads),
or leave it unset.
MAKEOPTS="-j4 -l5"
```

Search for *MAKEOPTS* in *man 5 make.conf* for more details.

Ready, set, go!

Update the `/mnt/gentoo/etc/portage/make.conf` file to match personal preference and save (nano users would press `Ctrl + o` to write the change and then `Ctrl + x` to quit).

References

0. <https://gitweb.gentoo.org/proj/portage.git/commit/?id=5d2af567772bb12b073f1671daea6263055cbdc2> (<https://gitweb.gentoo.org/proj/portage.git/commit/?id=5d2af567772bb12b073f1671daea6263055cbdc2>)
0. <https://gitweb.gentoo.org/proj/portage.git/commit/?id=de7be7f45ee54e3f789def46542919550687d15e> (<https://gitweb.gentoo.org/proj/portage.git/commit/?id=de7be7f45ee54e3f789def46542919550687d15e>)

Chrooting

Copy DNS info

One thing still remains to be done before entering the new environment and that is copying over the DNS information in `/etc/resolv.conf`. This needs to be done to ensure that networking still works even after entering the new environment. `/etc/resolv.conf` contains the name servers for the network.

To copy this information, it is recommended to pass the `--dereference` option to the `cp` command. This ensures that, if `/etc/resolv.conf` is a symbolic link, that the link's target file is copied instead of the symbolic link itself. Otherwise in the new environment the symbolic link would point to a non-existing file (as the link's target is most likely not available inside the new environment).

```
root # cp --dereference /etc/resolv.conf /mnt/gentoo/etc/
```

Mounting the necessary filesystems

📌 Tip

If using Gentoo's install media, this step can be replaced with simply: **arch-chroot /mnt/gentoo.**

In a few moments, the Linux root will be changed towards the new location.

The filesystems that need to be made available are:

- `/proc/` is a pseudo-filesystem. It looks like regular files, but is generated on-the-fly by the Linux kernel

- **/sys/** is a pseudo-filesystem, like **/proc/** which it was once meant to replace, and is more structured than **/proc/**
- **/dev/** is a regular file system which contains all device. It is partially managed by the Linux device manager (usually **udev**)
- **/run/** is a temporary file system used for files generated at runtime, such as PID files or locks

The **/proc/** location will be mounted on **/mnt/gentoo/proc/** whereas the others are bind-mounted. The latter means that, for instance, **/mnt/gentoo/sys/** will actually *be* **/sys/** (it is just a second entry point to the same filesystem) whereas **/mnt/gentoo/proc/** is a new mount (instance so to speak) of the filesystem.

```
root # mount --types proc /proc /mnt/gentoo/proc
root # mount --rbind /sys /mnt/gentoo/sys
root # mount --make-rslave /mnt/gentoo/sys
root # mount --rbind /dev /mnt/gentoo/dev
root # mount --make-rslave /mnt/gentoo/dev
root # mount --bind /run /mnt/gentoo/run
root # mount --make-slave /mnt/gentoo/run
```

Note

The `--make-rslave` operations are needed for systemd support later in the installation.

Warning

When using non-Gentoo installation media, this might not be sufficient. Some distributions make **/dev/shm** a symbolic link to **/run/shm/** which, after the chroot, becomes invalid. Making **/dev/shm/** a proper tmpfs mount up front can fix this:

```
root # test -L /dev/shm && rm /dev/shm && mkdir /dev/shm
root # mount --types tmpfs --options nosuid,nodev,noexec shm /dev/shm
Also ensure that mode 1777 is set:
root # chmod 1777 /dev/shm /run/shm
```

Entering the new environment

Now that all partitions are initialized and the base environment installed, it is time to enter the new installation environment by chrooting into it. This means that the session will change its root (most top-level location that can be accessed) from the current installation environment (installation CD or other installation medium) to the installation system (namely the initialized partitions). Hence the name, *change root* or *chroot*.

This chrooting is done in three steps:

1. The root location is changed from **/** (on the installation medium) to **/mnt/gentoo/** (on the partitions) using **chroot** or **arch-chroot**, if available.
2. Some settings (those in **/etc/profile**) are reloaded in memory using the **source** command
3. The primary prompt is changed to help us remember that this session is inside a chroot environment.

```
root # chroot /mnt/gentoo /bin/bash
```

```
root # source /etc/profile
root # export PS1="(chroot) ${PS1}"
```

❗ Important

If the above section was skipped to use **arch-chroot**. Use

```
root # arch-chroot /mnt/gentoo
root # export PS1="(chroot) ${PS1}"
```

From this point, all actions performed are immediately on the new Gentoo Linux environment.

✔ Tip

If the Gentoo installation is interrupted anywhere after this point, it *should* be possible to 'resume' the installation at this step. There is no need to re-partition the disks again! Simply check the date and time, mount the root partition ([/wiki/Handbook:AMD64/Installation/Disks#Mounting_the_root_partition](#)) and run the steps above starting with copying the DNS info ([/wiki/Handbook:AMD64/Installation/Base#Copy_DNS_info](#)) to re-enter the working environment. This is also useful for fixing bootloader issues. More information can be found in the chroot ([/wiki/Chroot](#)) article.

Preparing for a bootloader

Now that the new environment has been entered, it is necessary to prepare the new environment for the bootloader. It will be important to have the correct partition mounted when it is time to install the bootloader.

UEFI systems

For UEFI systems, `/dev/sda1` was formatted with the FAT32 filesystem and will be used as the EFI System Partition (ESP). Create a new `/efi` directory (if not yet created), and then mount ESP there:

```
root # mount /dev/sda1 /efi
```

DOS/Legacy BIOS systems

For DOS/Legacy BIOS systems, the bootloader will be installed into the `/boot` directory, therefore mount as follows:

```
root # mount /dev/sda1 /boot
```

Configuring Portage

Installing a Gentoo ebuild repository snapshot from the web

Next step is to install a snapshot of the Gentoo ebuild repository. This snapshot contains a collection of files that informs Portage about available software titles (for installation), which profiles the system administrator can select, package or profile specific news items, etc.

The use of **emerge-webrsync** is recommended for those who are behind restrictive firewalls (it uses HTTP/FTP protocols for downloading the snapshot) and saves network bandwidth.

This will fetch the latest snapshot (which is released on a daily basis) from one of Gentoo's mirrors and install it onto the system:

```
root # emerge-webrsync
```

Note

During this operation, **emerge-webrsync** might complain about a missing `/var/db/repos/gentoo/` location. This is to be expected and nothing to worry about - the tool will create the location.

From this point onward, Portage might mention that certain updates are recommended to be executed. This is because system packages installed through the stage file might have newer versions available; Portage is now aware of new packages because of the repository snapshot. Package updates can be safely ignored for now; updates can be delayed until after the Gentoo installation has finished.

Optional: Selecting mirrors

In order to download source code quickly it is recommended to select a fast, geographically close mirror. Portage will look in the **make.conf** file for the `GENTOO_MIRRORS` variable and use the mirrors listed therein. It is possible to surf to the Gentoo mirror list and search for a mirror (or multiple mirrors) close to the system's physical location (as those are most frequently the fastest ones).

A tool called **mirrorselect** provides a pretty text interface to more quickly query and select suitable mirrors. Just navigate to the mirrors of choice and press **Spacebar** to select one or more mirrors.

```
root # emerge --ask --verbose --oneshot app-portage/mirrorselect
root # mirrorselect -i -o >> /etc/portage/make.conf
```

Alternatively, a list of active mirrors are available online (<https://www.gentoo.org/downloads/mirrors/>).

Optional: rsync mirrors

Gentoo also has many rsync mirrors which can speed up downloading the available package list using `emerge --sync` (explained in more detail later) by selecting a mirror closer that is geographically closer to the user.

```
root # mkdir /etc/portage/repos.conf
root # cp /usr/share/portage/config/repos.conf /etc/portage/repos.conf/
gentoo.conf
```

Select a mirror close to the system's location from <https://www.gentoo.org/support/rsync-mirrors/> (<https://www.gentoo.org/support/rsync-mirrors/>) and replace the `sync-uri` default mirror in **/etc/portage/repos.conf/gentoo.conf** with the desired mirror location.

FILE `/etc/portage/repos.conf/gentoo.conf` **UK-based rsync mirror example**

[DEFAULT]

main-repo = gentoo

[gentoo]

```
location = /var/db/repos/gentoo
sync-type = rsync
sync-uri = rsync://rsync.uk.gentoo.org/gentoo-portage/
auto-sync = yes
sync-rsync-verify-jobs = 1
sync-rsync-verify-metamanifest = yes
sync-rsync-verify-max-age = 3
sync-openpgp-key-path = /usr/share/openpgp-keys/gentoo-release.asc
sync-openpgp-keyserver = hkps://keys.gentoo.org
sync-openpgp-key-refresh-retry-count = 40
sync-openpgp-key-refresh-retry-overall-timeout = 1200
sync-openpgp-key-refresh-retry-delay-exp-base = 2
sync-openpgp-key-refresh-retry-delay-max = 60
sync-openpgp-key-refresh-retry-delay-mult = 4
sync-webrsync-verify-signature = yes
sync-git-verify-commit-signature = true
```

Optional: Updating the Gentoo ebuild repository

It is possible to update the Gentoo ebuild repository to the latest version. The previous **emerge-webrsync** command will have installed a very recent snapshot (usually recent up to 24h) so this step is definitely optional.

Suppose there is a need for the latest package updates (up to 1 hour), then use **emerge --sync**. This command will use the rsync protocol to update the Gentoo ebuild repository (which was fetched earlier on through **emerge-webrsync**) to the latest state.

```
root # emerge --sync
```

On slow terminals, such as certain frame buffers or serial consoles, it is recommended to use the **--quiet** option to speed up the process:

```
root # emerge --sync --quiet
```

Reading news items

When the Gentoo ebuild repository is synchronized, Portage may output informational messages similar to the following:

```
* IMPORTANT: 2 news items need reading for repository 'gentoo'.
```

```
* Use eselect news to read news items.
```

News items were created to provide a communication medium to push critical messages to users via the Gentoo ebuild repository. To manage them, use **eselect news**. The **eselect** application is a Gentoo-specific utility that allows for a common management interface for system administration. In this case, **eselect** is asked to use its **news** module.

For the **news** module, three operations are most used:

- With **list** an overview of the available news items is displayed.
- With **read** the news items can be read.

- With `purge` news items can be removed once they have been read and will not be reread anymore.

```
root # eselect news list
```

```
root # eselect news read
```

More information about the news reader is available through its manual page:

```
root # man news.eselect
```

Choosing the right profile

✔ Tip

Desktop profiles are not exclusively for *desktop environments*. They are also suitable for minimal window managers like `i3` or `sway`.

A *profile* is a building block for any Gentoo system. Not only does it specify default values for *USE*, *CFLAGS*, and other important variables, it also locks the system to a certain range of package versions. These settings are all maintained by Gentoo's developers.

To see what profile the system is currently using, run **eselect** using the `profile` module:

✔ Tip

On an install media without a scroll-able terminal, `eselect profile list | less` can provide an easy way to list all available profiles while providing the ability to scroll.

```
root # eselect profile list
```

Available profile symlink targets:

- [1] default/linux/amd64/23.0 *
- [2] default/linux/amd64/23.0/desktop
- [3] default/linux/amd64/23.0/desktop/gnome
- [4] default/linux/amd64/23.0/desktop/plasma

📄 Note

The output of the command is just an example and evolves over time.

📄 Note

To use **systemd**, select a profile which has "systemd" in the name and vice versa, if not

There are also desktop sub-profiles available for some architectures which include software packages commonly necessary for a desktop experience.

⚠ Warning

Profile upgrades are not to be taken lightly. When selecting the initial profile, use the profile corresponding to **the same version** as the one initially used by the stage file (e.g. 23.0). Each new profile version is announced through a news item containing migration instructions; be sure to carefully follow the instructions before switching to a newer profile.

After viewing the available profiles for the amd64 architecture, users can select a different profile for the system:

```
root # eselect profile set 2
```

No-multilib

In order to select a pure 64-bit environment, with no 32-bit applications or libraries, use a no-multilib profile:

```
root # eselect profile list
```

Available profile symlink targets:

- [1] default/linux/amd64/23.0 *
- [2] default/linux/amd64/23.0/desktop
- [3] default/linux/amd64/23.0/desktop/gnome
- [4] default/linux/amd64/23.0/desktop/kde
- [5] default/linux/amd64/23.0/no-multilib

Next select the *no-multilib* profile:

```
root # eselect profile set 5
```

```
root # eselect profile list
```

Available profile symlink targets:

- [1] default/linux/amd64/23.0
- [2] default/linux/amd64/23.0/desktop
- [3] default/linux/amd64/23.0/desktop/gnome
- [4] default/linux/amd64/23.0/desktop/kde
- [5] default/linux/amd64/23.0/no-multilib *

Note

The `developer` sub-profile is specifically for Gentoo Linux development and is not meant to be used by casual users.

Optional: Adding a binary package host

Since December 2023, Gentoo's Release Engineering team (</wiki/Project:RelEng>) has offered an official binary package host (/wiki/Binary_package_quickstart) (colloquially shortened to just "binhost") for use by the general community to retrieve and install binary packages (binpkgs).^[1]

Adding a binary package host allows Portage to install cryptographically signed, compiled packages. In many cases, adding a binary package host will *greatly* decrease the mean time to package installation and adds much benefit when running Gentoo on older, slower, or low power systems.

Repository configuration

The repository configuration for a binhost is found in Portage's `/etc/portage/binrepos.conf/` directory, which functions similarly to the configuration mentioned in the Gentoo ebuild repository (`/wiki/Handbook:AMD64/Installation/Base#Gentoo_ebuild_repository`) section.

When defining a binary host, there are two important aspects to consider:

1. The architecture and profile targets within the `sync-uri` value *do* matter and should align to the respective computer architecture (**amd64** in this case) and system profile selected in the Choosing the right profile (`/wiki/Handbook:AMD64/Installation/Base#Choosing_the_right_profile`) section.
2. Selecting a fast, geographically close mirror will generally shorten retrieval time. Review the **mirrorselect** tool mentioned in the Optional: Selecting mirrors (`/wiki/Handbook:AMD64/Installation/Base#Gentoo_ebuild_repository`) section or review the online list of mirrors (<https://www.gentoo.org/downloads/mirrors/>) where URL values can be discovered.

FILE `/etc/portage/binrepos.conf/gentoo.conf` **CDN-based binary package host example**

```
[gentoo]
priority = 9959
# NOTE: Must adjust <arch> and <variant> as appropriate!
sync-uri = https://distfiles.gentoo.org/releases/<arch>/binpackages/
<variant>
# x86-64 example sync-uri
# sync-uri = https://distfiles.gentoo.org/releases/amd64/binpackages/23.0/
x86-64/

# Introduced in portage-3.0.74 for per-repo verification choices
verify-signature = true
# Default value with >=portage-3.0.77
location = /var/cache/binhost/gentoo
```

If the CPU supports x86-64-v3 (https://en.wikipedia.org/wiki/X86-64#Microarchitecture_levels) then the binhost offers binpkgs which are compiled to support these features as well.

For ease of explaining all main line Intel, Haswell and newer support this and same for AMD's Ryzen range. For other CPU lines please check or just use binhost example given above.

FILE `/etc/portage/binrepos.conf/gentoo.conf` **CDN-based binary package x86-64-v3 host example**

```
[gentoo-x86-64-v3]
priority = 9999
sync-uri = https://distfiles.gentoo.org/releases/amd64/binpackages/23.0/
x86-64-v3/
# Introduced in portage-3.0.74 for per-repo verification choices
verify-signature = true
# Default value with >=portage-3.0.77
location = /var/cache/binhost/gentoo-x86-64-v3
```

❗ Important

If using the x86-64-v3 binhost then is important to add both examples to **/etc/portage/binrepos.conf/gentoo.conf** as sometimes a package might not yet be available in a x86-64-v3 variant.

Installing binary packages

Portage will compile packages from code source by default. It can be instructed to use binary packages in the following ways:

1. The `--getbinpkg` option can be passed when invoking the **emerge** command. This method of binary package installation is useful to install only a particular binary package.
2. Changing the system's default via Portage's *FEATURES* variable, which is exposed through the **/etc/portage/make.conf** file. Applying this configuration change will cause Portage to query the binary package host for the package(s) to be requested and fall back to compiling locally when no results are found.

For example, to have Portage always install available binary packages:

FILE **/etc/portage/make.conf** **Configure Portage to use binary packages by default**

```
# Appending getbinpkg to the list of values within the FEATURES variable
FEATURES="${FEATURES} getbinpkg"
# Require signatures
FEATURES="${FEATURES} binpkg-request-signature"
```

Please also run **getuto** for Portage to set up the necessary keyring for verification:

```
root # getuto
```

Additional Portage features will be discussed in the the next chapter (/wiki/Handbook:AMD64/Working/Features#Portage_features) of the handbook.

Optional: Configuring the USE variable

USE is one of the most powerful variables Gentoo provides to its users. Several programs can be compiled with or without optional support for certain items. For instance, some programs can be compiled with support for GTK+ or with support for Qt. Others can be compiled with or without SSL support. Some programs can even be compiled with framebuffer support (svglib) instead of X11 support (X-server).

Most distributions compile their packages with support for as much as possible, increasing the size of the programs and startup time, not to mention an enormous amount of dependencies. With Gentoo, users can define what options for which a package should be compiled. This is where *USE* comes into play.

In the *USE* variable users define keywords which are mapped onto compile-options. For instance, `ssl` will compile SSL support in the programs that support it. `-x` will remove X-server support (note the minus sign in front). `gnome gtk -kde -qt5` will compile programs with GNOME (and GTK+) support, and not with KDE (and Qt) support, making the system fully tweaked for GNOME (if the architecture supports it).

The default *USE* settings are placed in the **make.defaults** files of the Gentoo profile used by the system. Gentoo uses a complex inheritance system for system profiles, which will not be covered in depth during the installation process. The easiest way to check the currently active *USE* settings is to run **emerge --info** and select the line that starts with *USE*:

```
root # emerge --info | grep ^USE
```

```
USE="X acl alsa amd64 berkdb bindist bzip2 cli cracklib crypt cxx dri ..."
```

Note

The above example is truncated, the actual list of *USE* values is much, much larger.

A full description on the available *USE* flags can be found on the system in `/var/db/repos/gentoo/profiles/use.desc`.

```
root # less /var/db/repos/gentoo/profiles/use.desc
```

Inside the **less** command, scrolling can be done using the  and  keys, and exited by pressing .

As an example we show a *USE* setting for a KDE-based system with DVD, ALSA, and CD recording support:

```
root # nano /etc/portage/make.conf
```

FILE `/etc/portage/make.conf` **Enabling flags for a KDE/Plasma-based system with DVD, ALSA, and CD recording support**

```
USE="-gtk -gnome qt5 kde dvd alsa cdr"
```

When a *USE* value is defined in `/etc/portage/make.conf` it is *added* to the system's *USE* flag list. *USE* flags can be globally *removed* by adding a  minus sign in front of the value in the the list. For example, to disable support for X graphical environments, `-x` can be set:

FILE `/etc/portage/make.conf` **Ignoring default *USE* flags**

```
USE="-X acl alsa"
```

Warning

Although possible, setting `-*` (which will disable all *USE* values except the ones specified in **make.conf**) is *strongly* discouraged and unwise. Ebuild developers choose certain default *USE* flag values in ebuilds in order to prevent conflicts, enhance security, and

avoid errors, and other reasons. Disabling *all* USE flags will negate default behavior and may cause major issues.

CPU_FLAGS_*

Some architectures (including AMD64/X86, ARM, PPC) have a USE_EXPAND (/wiki/USE_EXPAND) variable called CPU_FLAGS_<ARCH> (/wiki/CPU_FLAGS_X86), where <ARCH> is replaced with the relevant system architecture name.

❗ Important

Do not be confused! **AMD64** and **X86** systems share some common architecture, so the proper variable name for **AMD64** systems is *CPU_FLAGS_X86*.

This is used to configure the build to compile in specific assembly code or other intrinsics, usually hand-written or otherwise extra, and is **not** the same as asking the compiler to output optimized code for a certain CPU feature (e.g. `-march=`).

Users should set this variable in addition to configuring their *COMMON_FLAGS* as desired.

A few steps are needed to set this up:

```
root # emerge --ask --oneshot app-portage/cpuid2cpuflags
```

Inspect the output manually if curious:

```
root # cpuid2cpuflags
```

Then copy the output into **package.use**:

```
root # echo "*/ * $(cpuid2cpuflags)" > /etc/portage/package.use/00cpu-flags
```

VIDEO_CARDS

By default a profile already sets a few video cards. For many reasons this is not ideal, but a useful safety net.

To configure the system correctly the user needs to first unset the preset video cards with `VIDEO_CARDS: - *` then set the correct card for that system.

FILE `/etc/portage/package.use/00video_cards` **Example**

```
*/ * VIDEO_CARDS: - * <GPU DRIVER NAME>
```

Below is a table that can be used to easily compare the different video card types to their respective `VIDEO_CARDS` value.

GPU	VIDEO_CARDS
(Official) Nvidia Maxwell and newer	nvidia
Nouveau Nvidia Supported list (/wiki/NVIDIA)	nouveau
AMD since Sea Islands	amdgpu radeonsi
ATI and older AMD	See radeon#Feature support (/wiki/Radeon#Feature_support)
Intel Nehalem and newer	intel

Intel Gen 2 and 3 Supported list (/wiki/Intel#.23Feature_support)	intel i915
QEMU/KVM	virgl
WSL	d3d12

Below is a few examples of a properly set **package.use** for *VIDEO_CARDS*:

FILE /etc/portage/package.use/00video_cards **AMD example**

```
*/ * VIDEO_CARDS: - * amdgpu radeonsi
```

FILE /etc/portage/package.use/00video_cards **Intel example**

```
*/ * VIDEO_CARDS: - * intel
```

FILE /etc/portage/package.use/00video_cards **Nvidia example**

```
*/ * VIDEO_CARDS: - * nvidia
```

Details for various GPU(s) can be found at the AMDGPU (/wiki/AMDGPU), Intel (/wiki/Intel), Nouveau (Open Source) (/wiki/Nouveau), or NVIDIA (Proprietary) (/wiki/NVIDIA) articles.

Optional: Configure the ACCEPT_LICENSE variable

Starting with Gentoo Linux Enhancement Proposal 23 (GLEP 23), a mechanism was created to allow system administrators the ability to "regulate the software they install with regards to licenses... Some want a system free of any software that is not OSI-approved; others are simply curious as to what licenses they are implicitly accepting."^[2] With a motivation to have more granular control over the type of software running on a Gentoo system, the *ACCEPT_LICENSE* variable was born.

During the installation process, Portage considers the value(s) set within the *ACCEPT_LICENSE* variable to determine if the requested package(s) meet the sysadmin's determination of an acceptable license. Here in lies a problem: the Gentoo ebuild repository is filled with *thousands* of ebuilds which results in *hundreds* of distinct software licenses (<https://gitweb.gentoo.org/repo/gentoo.git/tree/licenses>)... Does this implicate sysadmin into individually approving each and every new software license? Thankfully no; GLEP 23 also outlines a solution to this problem, a concept called license groups.

For the convenience of system administration, legally-similar software licenses have been bundled together - each according to its like-kind. License group definitions are available for viewing (https://gitweb.gentoo.org/repo/gentoo.git/tree/profiles/license_groups) and are managed by the Gentoo Licenses project (/wiki/Project:Licenses). While an individual license is not, license groups are syntactically preceded with an @ symbol, enabling them to be easily distinguished in the *ACCEPT_LICENSE* variable.

Some common license groups include:

A list of software licenses grouped according to their kinds.

Name	Description
@GPL - COMPATIBLE	GPL compatible licenses approved by the Free Software Foundation [a_license 1]

@FSF-APPROVED	Free software licenses approved by the FSF (includes @GPL-COMPATIBLE)
@OSI-APPROVED	Licenses approved by the Open Source Initiative ^[a_license 2]
@MISC-FREE	Misc licenses that are probably free software, i.e. follow the Free Software Definition ^[a_license 3] but are not approved by either FSF or OSI
@FREE-SOFTWARE	Combines @FSF-APPROVED , @OSI-APPROVED , and @MISC-FREE .
@FSF-APPROVED-OTHER	FSF-approved licenses for "free documentation" and "works of practical use besides software and documentation" (including fonts)
@MISC-FREE-DOCS	Misc licenses for free documents and other works (including fonts) that follow the free definition ^[a_license 4] but are NOT listed in @FSF-APPROVED-OTHER .
@FREE-DOCUMENTS	Combines @FSF-APPROVED-OTHER and @MISC-FREE-DOCS .
@FREE	Metaset of all licenses with the freedom to use, share, modify and share modifications. Combines @FREE-SOFTWARE and @FREE-DOCUMENTS .
@BINARY-REDISTRIBUTABLE	Licenses that at least permit free redistribution of the software in binary form. Includes @FREE .
@EULA	License agreements that try to take away your rights. These are more restrictive than "all-rights-reserved" or require explicit approval

0. <https://www.gnu.org/licenses/license-list.html> (<https://www.gnu.org/licenses/license-list.html>)

0. <https://www.opensource.org/licenses> (<https://www.opensource.org/licenses>)

0. <https://www.gnu.org/philosophy/free-sw.html> (<https://www.gnu.org/philosophy/free-sw.html>)

0. <https://freedomdefined.org/> (<https://freedomdefined.org/>)

Currently set system wide acceptable license values can be viewed via:

```
user $ portageq envvar ACCEPT_LICENSE
```

```
@FREE
```

As visible in the output, the default value is to only allow software which has been grouped into the @FREE category to be installed.

Specific licenses or licenses groups for a system can be defined in the following locations:

- System wide within the selected profile - this sets the default value.
- System wide within the **/etc/portage/make.conf** file. System administrators override the profile's default value within this file.
- Per-package within a **/etc/portage/package.license** file.
- Per-package within a **/etc/portage/package.license/** *directory* of files.

The system wide license default in the profile is overridden within the `/etc/portage/make.conf`:

FILE `/etc/portage/make.conf` **Accept licenses with ACCEPT_LICENSE system wide**

```
# Overrides the profile's ACCEPT_LICENSE default value
ACCEPT_LICENSE="-* @FREE @BINARY-REDISTRIBUTABLE"
```

Optionally system administrators can also define accepted licenses per-package as shown in the following directory of files example. Note that the `package.license` directory will need created if it does not already exist:

```
root # mkdir /etc/portage/package.license
```

Software license details for an individual Gentoo package are stored within the `LICENSE` variable of the associated ebuild. One package may have one or many software licenses, therefore it be necessary to specify multiple acceptable licenses for a single package.

FILE `/etc/portage/package.license/kernel` **Accepting licenses on a per-package basis**

```
app-arch/unrar unrar
sys-kernel/linux-firmware linux-fw-redistributable
sys-firmware/intel-microcode intel-ucode
```

❗ Important

The `LICENSE` variable in an ebuild is only a guideline for Gentoo developers and users. It is *not* a legal statement, and there is *no guarantee* that it will reflect reality. It is recommended to not solely rely on a ebuild developer's interpretation of a software package's license; but check the package itself in depth, including all files that have been installed to the system.

Optional: Updating the @world set

✔ Tip

If a desktop environment profile target has been selected from a non-desktop stage file, the @world update process could greatly extend the amount of time necessary for the install process. Those in a time crunch can work by this 'rule of thumb': the shorter the profile name, the less specific the system's @world set (`/wiki/World_set_(Portage)`). The less specific the @world set, the fewer packages the system will require. E.g.:

- Selecting `default/linux/amd64/23.0` will likely require fewer packages to be updated, whereas
- Selecting `default/linux/amd64/23.0/desktop/gnome/systemd` will likely require more packages to be installed since the profile target has a larger @system (`/wiki/Package_sets#.40system`) and @profile (`/wiki/Package_sets#.40profile`) sets: dependencies supporting the GNOME desktop environment.

Updating the system's @world set (`/wiki/World_set_(Portage)`) is *optional* and will be unlikely to perform functional changes unless one or more of the following optional steps have been performed:

1. A profile target *different* from the stage file has been selected.
2. Additional USE flags have been set for installed packages.

Readers who are performing an 'install Gentoo speed run' may safely skip @world set updates until *after* their system has rebooted into the new Gentoo environment.

Readers who are performing a slow run can have Portage perform updates for package, profile, and/or USE flag changes at the present time:

```
root # emerge --ask --verbose --update --deep --changed-use @world
```

Readers who added a binary host above (/wiki/Handbook:AMD64/Installation/Base#Optional:_Adding_a_binary_package_host) can add **--getbinpkg** (or **-g**) in order to fetch packages from the binary host instead of compiling them:

```
root # emerge --ask --verbose --update --deep --newuse --getbinpkg @world
```

Removing obsolete packages

It is important to always *depclean* after system upgrades to remove obsolete packages. Review the output carefully with **emerge --depclean --pretend** to see if any of the to-be-cleaned packages should be kept if personally using them. To keep a package which would otherwise be depcleaned, use **emerge --noreplace foo**.

```
root # emerge --ask --pretend --depclean
```

If happy, then proceed with a real depclean:

```
root # emerge --ask --depclean
```

Optional: Using systemd as the system and service manager

The remainder of the Gentoo handbook will provide systemd steps alongside OpenRC (/wiki/OpenRC) (the traditional Gentoo init system) where separate steps or recommendations are necessary. System administrators should also consult the systemd (/wiki/Systemd) article for more details on managing systemd as the system and service manager.

Timezone

Note

This step does not apply to users of the musl libc. Users who do not know what that means should perform this step.

Warning

Please avoid the **/usr/share/zoneinfo/Etc/GMT*** timezones as their names do not indicate the expected zones. For instance, **GMT-8** is in fact **GMT+8**.

Select the timezone for the system. Look for the available timezones in **/usr/share/zoneinfo/**:

```
root # ls -l /usr/share/zoneinfo
```

```
total 352
drwxr-xr-x 2 root root 1120 Jan 7 17:41 Africa
drwxr-xr-x 6 root root 2960 Jan 7 17:41 America
drwxr-xr-x 2 root root 280 Jan 7 17:41 Antarctica
drwxr-xr-x 2 root root 60 Jan 7 17:41 Arctic
drwxr-xr-x 2 root root 2020 Jan 7 17:41 Asia
drwxr-xr-x 2 root root 280 Jan 7 17:41 Atlantic
drwxr-xr-x 2 root root 500 Jan 7 17:41 Australia
drwxr-xr-x 2 root root 120 Jan 7 17:41 Brazil
-rw-r--r-- 1 root root 2094 Dec 3 17:19 CET
-rw-r--r-- 1 root root 2310 Dec 3 17:19 CST6CDT
drwxr-xr-x 2 root root 200 Jan 7 17:41 Canada
drwxr-xr-x 2 root root 80 Jan 7 17:41 Chile
-rw-r--r-- 1 root root 2416 Dec 3 17:19 Cuba
-rw-r--r-- 1 root root 1908 Dec 3 17:19 EET
-rw-r--r-- 1 root root 114 Dec 3 17:19 EST
-rw-r--r-- 1 root root 2310 Dec 3 17:19 EST5EDT
-rw-r--r-- 1 root root 2399 Dec 3 17:19 Egypt
-rw-r--r-- 1 root root 3492 Dec 3 17:19 Eire
drwxr-xr-x 2 root root 740 Jan 7 17:41 Etc
drwxr-xr-x 2 root root 1320 Jan 7 17:41 Europe
...
```

```
root # ls -l /usr/share/zoneinfo/Europe/
```

```
total 256
-rw-r--r-- 1 root root 2933 Dec 3 17:19 Amsterdam
-rw-r--r-- 1 root root 1742 Dec 3 17:19 Andorra
-rw-r--r-- 1 root root 1151 Dec 3 17:19 Astrakhan
-rw-r--r-- 1 root root 2262 Dec 3 17:19 Athens
-rw-r--r-- 1 root root 3664 Dec 3 17:19 Belfast
-rw-r--r-- 1 root root 1920 Dec 3 17:19 Belgrade
-rw-r--r-- 1 root root 2298 Dec 3 17:19 Berlin
-rw-r--r-- 1 root root 2301 Dec 3 17:19 Bratislava
-rw-r--r-- 1 root root 2933 Dec 3 17:19 Brussels
...
```

Suppose the timezone of choice is **Europe/Brussels**, to select this timezone, a symlink can be created from this zoneinfo file to **/etc/localtime**:

```
root # ln -sf ../usr/share/zoneinfo/Europe/Brussels /etc/localtime
```

💡 Tip

The target path with `../` at the start is *relative to the link location*, not the current directory.

Note

An absolute path can be used for the symlink, but a relative link is also created by systemd's `timedatectl` and is more compatible with alternate *ROOTs*.

Configure locales

Note

This step does not apply to users of the musl libc. Users who do not know what that means should perform this step.

Locale generation

A default installation of Gentoo Linux provides an archive that contains all supported locales, numbering 500 or more. However, it is typical for an administrator to require only one or two of these. In that case, the `/etc/locale.gen` configuration file may be populated with a list of the required locales. By default, `locale-gen` shall read this file and compile only the locales that are specified, saving both time and space in the longer term.

Locales specify not only the language that the user should use to interact with the system, but also the rules for sorting strings, displaying dates and times, etc. Locales are *case sensitive* and must be represented exactly as described. A full listing of available locales can be found in the `/usr/share/i18n/SUPPORTED` file.

```
root # nano /etc/locale.gen
```

The following locales are an example to get both English (United States) and German (Germany/Deutschland) with the accompanying character formats (like UTF-8).

 `/etc/locale.gen` **Enabling US and DE locales with the appropriate character formats**

```
en_US.UTF-8 UTF-8
de_DE.UTF-8 UTF-8

# Non UTF-8 locales are discouraged and should only be used in special
# circumstances.
#en_US ISO-8859-1
#de_DE ISO-8859-1
```

Warning

Many applications require least one UTF-8 locale to build properly.

The next step is to run the `locale-gen` command. This command generates all locales specified in the `/etc/locale.gen` file.

```
root # locale-gen
```

To verify that the selected locales are now available, run `locale -a`.

On systemd installs, **localectl** can be used, e.g. **localectl set-locale ...** or **localectl list-locales**.

Locale selection

Once done, it is now time to set the system-wide locale settings. Again **eselect** is used, now with the **locale** module.

With **eselect locale list**, the available targets are displayed:

```
root # eselect locale list
```

```
Available targets for the LANG variable:
```

```
[1]  C
[2]  C.UTF-8
[3]  POSIX
[4]  de_DE.UTF-8
[5]  en_US.UTF-8
[ ]  (free form)
```

With **eselect locale set <NUMBER>**, the correct locale can be selected:

```
root # eselect locale set 2
```

Manually, this can still be accomplished through the **/etc/env.d/02locale** file, and for systemd the **/etc/locale.conf** file:

FILE **/etc/env.d/02locale** **Manually setting system locale definitions**

```
LANG="de_DE.UTF-8"
LC_COLLATE="C.UTF-8"
```

Setting the locale will avoid warnings and errors during kernel and software compilations later in the installation.

Now reload the environment:

```
root # env-update && source /etc/profile && export PS1="(chroot) ${PS1}"
```

For additional guidance through the locale selection process, read also the Localization guide (</wiki/Localization/Guide>) and the UTF-8 (</wiki/UTF-8>) guide.

References

0. <https://www.gentoo.org/news/2023/12/29/Gentoo-binary.html> (<https://www.gentoo.org/news/2023/12/29/Gentoo-binary.html>)
0. <https://www.gentoo.org/glep/glep-0023.html#id7> (<https://www.gentoo.org/glep/glep-0023.html#id7>)

Installing firmware and/or microcode

Firmware

Suggested: Linux Firmware

On many systems, non-FOSS firmware is required for certain hardware to function. The `sys-kernel/linux-firmware` (<https://packages.gentoo.org/packages/sys-kernel/linux-firmware>) [Ⓐ] package contains firmware for many, but not all, devices.

✔ Tip

Most wireless cards and GPUs require firmware to function.

```
root # emerge --ask sys-kernel/linux-firmware
```

📄 Note

Installing certain firmware packages often requires accepting the associated firmware licenses. If necessary, visit the license handling section (</wiki/Handbook:AMD64/Working/Portage#Licenses>) of the Handbook for help on accepting licenses.

Firmware Loading

Firmware files are typically loaded when the associated kernel module is loaded. This means the firmware must be built into the kernel using **CONFIG_EXTRA_FIRMWARE** if the kernel module is set to *Y* instead of *M*. In most cases, building-in a module which required firmware can complicate or break loading.

SOF Firmware

Sound Open Firmware (SOF) is a new open source audio driver meant to replace the proprietary Smart Sound Technology (SST) audio driver from Intel. 10th gen+ and Apollo Lake (Atom E3900, Celeron N3350, and Pentium N4200) Intel CPUs require this firmware for certain features and certain AMD APUs also have support for this firmware. SOF's supported platforms matrix can be found here (<https://thesofproject.github.io/latest/platforms/index.html>) for more information.

```
root # emerge --ask sys-firmware/sof-firmware
```

Suggested: Microcode

In addition to discrete graphics hardware and network interfaces, CPUs can also require firmware updates. Typically this kind of firmware is referred to as *microcode*. Newer revisions of microcode are sometimes necessary to patch instability, security concerns, or other miscellaneous bugs in CPU hardware.

Microcode updates for AMD CPUs are distributed within the aforementioned `sys-kernel/linux-firmware` (<https://packages.gentoo.org/packages/sys-kernel/linux-firmware>) [Ⓐ] package. Microcode for Intel CPUs can be found within the

`sys-firmware/intel-microcode` (<https://packages.gentoo.org/packages/sys-firmware/intel-microcode>) [Ⓐ] package, which will need to be installed separately. See the [Microcode](/wiki/Microcode) article (</wiki/Microcode>) for more information on how to apply microcode updates.

sys-kernel/installkernel

`installkernel` (</wiki/Installkernel>) may be used to automate the kernel installation, `initramfs` (</wiki/Initramfs>) generation, unified kernel image (/wiki/Unified_kernel_image) generation and bootloader configuration, among other things.

`sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>) [Ⓐ] implements two paths of achieving this: the traditional **installkernel** originating from Debian and `systemd` (</wiki/Systemd>)'s **kernel-install**. Which one to choose depends, among other things, on the system's bootloader. By default, `systemd`'s **kernel-install** is used on `systemd` profiles, while the traditional **installkernel** is the default for other profiles.

Bootloader

Now is the time to think about which bootloader the user wants for the system.

❗ Important

Only one selection is required in the following subsection, if unsure of which to use then go with the first listed for now. It's always possible to switch at a later date if required.

GRUB

Users of GRUB can use either `systemd`'s **kernel-install** or the traditional Debian **installkernel**. The

`systemd` (<https://packages.gentoo.org/useflags/systemd>) [Ⓐ] (/wiki/USE_flag) `USE` flag switches between these implementations. To automatically run **grub-mkconfig** when installing the kernel, enable the `grub` (<https://packages.gentoo.org/useflags/grub>) [Ⓐ] (/wiki/USE_flag) `USE` flag (/wiki/USE_flag).

📄 Note

GRUB requires kernels to be installed to **/boot**.

FILE `/etc/portage/package.use/installkernel`

```
sys-kernel/installkernel grub
```

```
root # emerge --ask sys-kernel/installkernel
```

systemd-boot

When using `systemd-boot` (</wiki/Systemd/systemd-boot>) (formerly `gummiboot`) as the bootloader, `systemd`'s **kernel-install** must be used. Therefore ensure the `systemd` (<https://packages.gentoo.org/useflags/systemd>) [Ⓐ] (/wiki/USE_flag) and the

`systemd-boot` (<https://packages.gentoo.org/useflags/systemd-boot>) [Ⓐ] (/wiki/USE_flag) `USE` flags are enabled on

`sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>) [Ⓐ], and then install the relevant package for `systemd-boot`.

FILE `/etc/portage/package.use/systemd-boot`

```
sys-apps/systemd boot
sys-apps/systemd-utils boot kernel-install
sys-kernel/installkernel systemd systemd-boot
```

The kernel command line to use for new kernels should be specified in `/etc/kernel/cmdline` and `/etc/cmdline`. As they both need to be updated together, it make sense to symlink the two files now.

FILE `/etc/kernel/cmdline`

```
quiet splash
```

```
root # mkdir -p /etc/cmdline.d
```

```
root # ln -s /etc/kernel/cmdline /etc/cmdline.d/00-installkernel.conf
```

```
root # emerge --ask sys-kernel/installkernel
```

Note

systemd-boot requires kernels to be installed to `/efi`.

EFI stub

UEFI-based computer systems technically do not need secondary bootloaders in order to boot kernels. Secondary bootloaders exist to *extend* the functionality of UEFI firmware during the boot process. That being said, using a secondary bootloader is typically easier and more robust because it offers a more flexible approach for quickly modifying kernel parameters at boot time. Note also that UEFI implementations strongly differ between vendors and between models and there is no guarantee that a given firmware follows the UEFI specification. Therefore, EFI Stub booting is not guaranteed to work on every UEFI-based system.

```
root # emerge --ask sys-kernel/installkernel
```

```
root # mkdir -p /efi/EFI/Gentoo
```

Note

When

`app-emulation/virt-firmware` (<https://packages.gentoo.org/packages/app-emulation/virt-firmware>)  is used to configure the UEFI ensure that the `kernel-bootcfg-boot-successful` service is enabled before attempting to install the kernel. This implementation of EFistub booting is the default for systemd systems.

```
root # systemctl enable kernel-bootcfg-boot-successful.service
```

Note

EFistub requires kernels to be installed to `/efi`.

Traditional layout, other bootloaders (e.g. (e)lilo, syslinux, etc.)

The traditional `/boot` layout (for e.g. (e)LILO, syslinux, etc.) is used by default if the `grub` (<https://packages.gentoo.org/useflags/grub>) , `systemd-boot` (<https://packages.gentoo.org/useflags/systemd-boot>) , or `syslinux` (<https://packages.gentoo.org/useflags/syslinux>)  is selected in the `USE` flags.

efistub (<https://packages.gentoo.org/useflags/efistub>) $\Rightarrow$ (/wiki/USE_flag) and uki (<https://packages.gentoo.org/useflags/uki>) $\Rightarrow$ (/wiki/USE_flag) USE flags are **not** enabled. No further action is required.

Initramfs

An **initial ram**-based **file system**, or initramfs (/wiki/Initramfs), may be required for a system to boot. A wide of variety of cases may necessitate one, but common cases include:

- Kernels where storage/filesystem drivers are modules.
- Layouts with **/usr/** or **/var/** on separate partitions.
- Encrypted root filesystems.

Tip

Distribution kernels (/wiki/Project:Distribution_Kernel) are designed to be used with an initramfs, as many storage and filesystem drivers are built as modules.

In addition to mounting the root filesystem, an initramfs may also perform other tasks such as:

- Running **file system consistency check** **fsck**, a tool to check and repair consistency of a file system in such events of uncleanly shutdown a system.
- Providing a recovery environment in the event of late-boot failures.

Installkernel (/wiki/Installkernel) can automatically generate an initramfs when installing the kernel if the dracut (<https://packages.gentoo.org/useflags/dracut>) $\Rightarrow$ (/wiki/USE_flag) or ugrd (<https://packages.gentoo.org/useflags/ugrd>) $\Rightarrow$ (/wiki/USE_flag) USE flag is enabled:

FILE /etc/portage/package.use/installkernel

```
sys-kernel/installkernel dracut
```

```
root # emerge --ask sys-kernel/installkernel
```

Optional: Unified Kernel Image

A Unified Kernel Image (/wiki/Unified_Kernel_Image) (UKI) combines, among other things, the kernel, the initramfs and the kernel command line into a single executable. Since the kernel command line is embedded into the unified kernel image, it should be specified before generating the unified kernel image (see below). Note that any kernel command line arguments supplied by the bootloader or firmware at boot are ignored when booting with secure boot enabled.

A unified kernel image requires a stub loader. Currently, the only one available is **systemd-stub**. To enable it:

For systemd systems:

FILE /etc/portage/package.use/uki

```
sys-apps/systemd boot
```

```
root # emerge --ask sys-apps/systemd
```

For OpenRC systems:

FILE `/etc/portage/package.use/uki`

```
sys-apps/systemd-utils boot kernel-install
```

root # emerge --ask sys-apps/systemd-utils

Installkernel ([/wiki/Installkernel](#)) can automatically generate a unified kernel image using either dracut ([/wiki/Unified_kernel_image#dracut](#)) or ukify ([/wiki/Unified_kernel_image#ukify](#)) by enabling the respective flag and the uki (<https://packages.gentoo.org/useflags/uki>) $\Rightarrow$ ([/wiki/USE_flag](#)) USE flag.

For dracut:

FILE `/etc/portage/package.use/uki`

```
sys-kernel/installkernel dracut uki
```

FILE `/etc/dracut.conf.d/uki.conf`

```
uefi="yes"
kernel_cmdline="some-kernel-command-line-arguments"
```

root # emerge --ask sys-kernel/installkernel

For ukify:

FILE `/etc/portage/package.use/uki`

```
sys-apps/systemd boot ukify                # For systemd systems
sys-apps/systemd-utils kernel-install boot ukify  # For OpenRC systems
sys-kernel/installkernel dracut ukify uki
```

FILE `/etc/kernel/cmdline`

```
some-kernel-command-line-arguments
```

root # emerge --ask sys-kernel/installkernel

Note that while dracut can generate both an initramfs and a unified kernel image, ukify can only generate the latter and therefore the initramfs must be generated separately with dracut.

❗ Important

In the above configuration examples (for both Dracut and ukify) it is important to specify at least an appropriate `root=` parameter for the kernel command line to ensure that the Unified Kernel Image can find the root partition. This is not required for systemd based systems following the Discoverable Partitions Specification (DPS), in that case the embedded initramfs will be able to dynamically find the root partition.

Generic Unified Kernel Image (systemd only)

The prebuilt

`sys-kernel/gentoo-kernel-bin` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>) $\Rightarrow$ can optionally install a prebuilt generic unified kernel image containing a generic initramfs that is able to boot most systemd based systems. It can be installed by enabling the `generic-uki` (<https://packages.gentoo.org/useflags/generic-uki>) $\Rightarrow$ ([/wiki/USE_flag](#)) USE flag, and configuring installkernel ([/wiki/Installkernel](#)) to not generate a custom initramfs or unified kernel image:

FILE `/etc/portage/package.use/uki`

```
sys-kernel/gentoo-kernel-bin generic-uki
sys-kernel/installkernel -dracut -ukify -ugrd uki
```

Secure Boot

⚠ Warning

If following this section and manually compiling your own kernel, then make sure to follow the steps outlined in [Signing the kernel \(/wiki/Kernel#Optional:_Signing_the_kernel_image_.28Secure_Boot.29\)](#)

The generic Unified Kernel Image optionally distributed by `sys-kernel/gentoo-kernel-bin` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>) [Ⓐ] is already pre-signed. How to sign a locally generated unified kernel image depends on whether `dracut` or `ukify` is used. Note that the location of the key and certificate should be the same as the `SECUREBOOT_SIGN_KEY` and `SECUREBOOT_SIGN_CERT` as specified in `/etc/portage/make.conf`.

For `dracut`:

FILE `/etc/dracut.conf.d/uki.conf`

```
uefi="yes"
kernel_cmdline="some-kernel-command-line-arguments"
uefi_secureboot_key="/path/to/kernel_key.pem"
uefi_secureboot_cert="/path/to/kernel_key.pem"
```

For `ukify`:

FILE `/etc/kernel/uki.conf`

```
[UKI]
SecureBootPrivateKey=/path/to/kernel_key.pem
SecureBootCertificate=/path/to/kernel_key.pem
```

Kernel selection

It can be a wise move to use the `dist-kernel` on the first boot as it provides a very simple method to rule out system issues and kernel config issues. Always having a known working kernel to fallback on can speed up debugging and alleviate anxiety when updating that your system will no longer boot.

A common misconception is that a manually compiled kernel will use a lot less RAM than a pre configured distribution kernel. Due to the modular nature of the Linux kernel, only what is needed by the system is loaded and in most cases resulting in similar memory usage.

❗ Important

Only one selection is required in the following subsection, if unsure of which to use then go with the first listed for now. It's always possible to switch at a later date if required.

Ranked from least involved to most involved:

Full automation approach: Distribution kernels (/wiki/Handbook:AMD64/Installation/Kernel#Distribution_kernels)

A Distribution Kernel (/wiki/Project:Distribution_Kernel) is used to configure, automatically build, and install the Linux kernel, its associated modules, and (optionally, but enabled by default) an initramfs file. Future kernel updates are fully automated since they are handled through the package manager, just like any other system package. It is possible provide a custom kernel configuration file (/wiki/Project:Distribution_Kernel#Modifying_kernel_configuration) if customization is necessary. This is the least involved process and is perfect for new Gentoo users due to it working out-of-the-box and offering minimal involvement from the system administrator.

Manual approach (/wiki/Handbook:AMD64/Installation/Kernel#Alternative:_Manual_configuration)

New kernel sources are installed via the system package manager. The kernel is manually configured, built, and installed using the **eselect kernel** and a slew of **make** commands. Future kernel updates repeat the manual process of configuring, building, and installing the kernel files. This is the most involved process, but offers maximum control over the kernel update process.

The core around which all distributions are built is the Linux kernel. It is the layer between the user's programs and the system hardware. Although the handbook provides its users several possible kernel sources, a more comprehensive listing with more detailed descriptions is available at the kernel packages page (/wiki/Kernel/Packages).

Tip

Kernel installation tasks such as copying the kernel image to **/boot** or the EFI System Partition (/wiki/EFI_System_Partition), generating an initramfs (/wiki/Initramfs) and/or Unified Kernel Image (/wiki/Unified_Kernel_Image), updating bootloader configuration, can be automated with **installkernel** (/wiki/Installkernel). Users may wish to configure and install

`sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>) before proceeding. See the Kernel installation section below (/wiki/Handbook:AMD64/Installation/Kernel#Kernel_installation) for more more information.

Distribution kernels

Distribution Kernels (/wiki/Project:Distribution_Kernel) are ebuilds that cover the complete process of unpacking, configuring, compiling, and installing the kernel. The primary advantage of this method is that the kernels are updated to new versions by the package manager as part of `@world` upgrade. This requires no more involvement than running an **emerge** command. Distribution kernels default to a configuration supporting the majority of hardware, however two mechanisms are offered for customization: `savedconfig` and `config` snippets. See the project page for more details on configuration. (/wiki/Project:Distribution_Kernel#Modifying_kernel_configuration)

Optional: Signed kernel modules

The kernel modules in the prebuilt distribution kernel (`sys-kernel/gentoo-kernel-bin` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>)) are already signed. To sign the modules of kernels built from source enable the

modules-sign (<https://packages.gentoo.org/useflags/modules-sign>)  (/wiki/USE_flag) USE flag, and optionally specify which key to use for signing in /etc/portage/make.conf (/wiki//etc/portage/make.conf):

FILE /etc/portage/make.conf **Enable module signing**

```
USE="modules-sign"

# Optionally, to use custom signing keys.
MODULES_SIGN_KEY="/path/to/kernel_key.pem"
MODULES_SIGN_CERT="/path/to/kernel_key.pem" # Only required if the
MODULES_SIGN_KEY does not also contain the certificate.
MODULES_SIGN_HASH="sha512" # Defaults to sha512.
```

If `MODULES_SIGN_KEY` is not specified the kernel build system will generate a key, it will be stored in `/usr/src/linux-x.y.z/certs`. It is recommended to manually generate a key to ensure that it will be the same for each kernel release. A key may be generated with:

```
root # openssl req -new -noenc -utf8 -sha256 -x509 -outform PEM -out
kernel_key.pem -keyout kernel_key.pem
```

 Note

The `MODULES_SIGN_KEY` and `MODULES_SIGN_CERT` may be different files. For this example the pem file generated by OpenSSL includes both the key and the accompanying certificate, and thus both variables are set to the same value.

OpenSSL will ask some questions about the user generating the key, it is recommended to fill in these questions as detailed as possible.

Store the key in a safe location, at the very least the key should be readable only by the root user. Verify this with:

```
root # ls -l kernel_key.pem
```

```
-r----- 1 root root 3164 Jan  4 10:38 kernel_key.pem
```

If this outputs anything other than the above, correct the permissions with:

```
root # chown root:root kernel_key.pem
```

```
root # chmod 400 kernel_key.pem
```

Optional: Signing the kernel image (Secure Boot)

The kernel image in the prebuilt distribution kernel (`sys-kernel/gentoo-kernel-bin` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>) ) is already signed for use with Secure Boot (/wiki/Secure_Boot). To sign the kernel image of kernels built from source enable the `secureboot` (<https://packages.gentoo.org/useflags/secureboot>)  (/wiki/USE_flag) USE flag, and optionally specify which key to use for signing in /etc/portage/make.conf (/wiki//etc/portage/make.conf). Note that signing the kernel image for use with secureboot requires that the kernel modules are also signed, the same key may be used to sign both the kernel image and the kernel modules:

FILE /etc/portage/make.conf **Enable custom signing keys**

```
USE="modules-sign secureboot"
```

```
# Optionally, to use custom signing keys.
```

```
MODULES_SIGN_KEY="/path/to/kernel_key.pem"
```

```
MODULES_SIGN_CERT="/path/to/kernel_key.pem" # Only required if the  
MODULES_SIGN_KEY does not also contain the certificate.
```

```
MODULES_SIGN_HASH="sha512" # Defaults to sha512.
```

```
# Optionally, to boot with secureboot enabled, may be the same or different  
signing key.
```

```
SECUREBOOT_SIGN_KEY="/path/to/kernel_key.pem"
```

```
SECUREBOOT_SIGN_CERT="/path/to/kernel_key.pem"
```

Note

The **SECUREBOOT_SIGN_KEY** and **SECUREBOOT_SIGN_CERT** may be different files. For this example the pem file generated by OpenSSL includes both the key and the accompanying certificate, and thus both variables are set to the same value.

Note

For this example the same key that was generated to sign the modules is used to sign the kernel image. It is also possible to generate and use a second separate key for signing the kernel image. The same OpenSSL command as in the previous section may be used again.

See the above section for instructions on generating a new key, the steps may be repeated if a separate key should be used to sign the kernel image.

To successfully boot with Secure Boot enabled, the used bootloader must also be signed and the certificate must be accepted by the UEFI (/wiki/UEFI) firmware or Shim (/wiki/Shim). This will be explained later in the handbook.

Installing a distribution kernel

To build a kernel with Gentoo patches from source, type:

```
root # emerge --ask sys-kernel/gentoo-kernel
```

System administrators who want to avoid compiling the kernel sources locally can instead use precompiled kernel images:

```
root # emerge --ask sys-kernel/gentoo-kernel-bin
```

Important

Distribution Kernels (/wiki/Project:Distribution_Kernel), such as

sys-kernel/gentoo-kernel (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel>) 
and

sys-kernel/gentoo-kernel-bin (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>) 

by default, expect to be installed alongside an initramfs (/wiki/Handbook:AMD64/Installation/Kernel#Initramfs). Before running emerge to install the kernel users should ensure that

sys-kernel/installkernel (<https://packages.gentoo.org/packages/sys-kernel/installkernel>) 
has been configured to utilize an initramfs generator (for example Dracut (/wiki/Dracut))

as described in the `installkernel` section (</wiki/Handbook:AMD64/Installation/Kernel#Initramfs>).

Upgrading and cleaning up

Once the kernel is installed, the package manager will automatically update it to newer versions. The previous versions will be kept until the package manager is requested to clean up stale packages. To reclaim disk space, stale packages can be trimmed by periodically running `emerge` with the `--depclean` option:

```
root # emerge --depclean
```

Alternatively, to specifically clean up old kernel versions:

```
root # emerge --prune sys-kernel/gentoo-kernel sys-kernel/gentoo-kernel-bin
```

Tip

By design, `emerge` only removes the kernel build directory. It does not actually remove the kernel modules, nor the installed kernel image. To completely clean-up old kernels, the

`app-admin/eclean-kernel` (<https://packages.gentoo.org/packages/app-admin/eclean-kernel>)[Ⓐ] tool may be used.

Post-install/upgrade tasks

An upgrade of a distribution kernel is capable of triggering an automatic rebuild for external kernel modules installed by other packages (for example:

`sys-fs/zfs-kmod` (<https://packages.gentoo.org/packages/sys-fs/zfs-kmod>)[Ⓐ] or

`x11-drivers/nvidia-drivers` (<https://packages.gentoo.org/packages/x11-drivers/nvidia-drivers>)[Ⓐ]).

This automated behaviour is enabled by enabling the

`dist-kernel` (<https://packages.gentoo.org/useflags/dist-kernel>)[Ⓐ] (/wiki/USE_flag)

USE flag. When required, this same flag will also trigger re-generation of the `initramfs` (</wiki/Initramfs>).

It is highly recommended to enable this flag globally via `/etc/portage/make.conf` when using a distribution kernel:

FILE `/etc/portage/make.conf` **Enabling USE=dist-kernel**

```
USE="dist-kernel"
```

Manually rebuilding the initramfs or Unified Kernel Image

If required, manually trigger such rebuilds by, after a kernel upgrade, executing:

```
root # emerge --ask @module-rebuild
```

If any kernel modules (e.g. ZFS) are needed at early boot, rebuild the `initramfs` afterward via:

```
root # emerge --config sys-kernel/gentoo-kernel
```

```
root # emerge --config sys-kernel/gentoo-kernel-bin
```

After installing the Distribution Kernel successfully, it is now time to proceed to the next section: Configuring the system ([/wiki/Handbook:AMD64/Installation/System](#)).

Alternative: Manual configuration

Installing and Configuring the kernel sources

When manually compiling the kernel for amd64-based systems, Gentoo recommends the `sys-kernel/gentoo-sources` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-sources>)  package.

Choose an appropriate kernel source and install it using **emerge**:

```
root # emerge --ask sys-kernel/gentoo-sources
```

This will install the Linux kernel sources in **/usr/src/** using the specific kernel version in the path. It will not create a symbolic link by itself without the `symlink` (<https://packages.gentoo.org/useflags/symlink>)  ([/wiki/USE_flag](#)) `USE` flag being enabled on the chosen kernel sources package.

It is conventional for a **/usr/src/linux** symlink to be maintained, such that it refers to whichever sources correspond with the currently running kernel. However, this symbolic link will not be created by default. An easy way to create the symbolic link is to utilize `eselect`'s kernel module.

For further information regarding the purpose of the symlink, and how to manage it, please refer to [Kernel/Upgrade](#) ([/wiki/Kernel/Upgrade](#)).

First, list all installed kernels:

```
root # eselect kernel list
```

```
Available kernel symlink targets:
[1]   linux-6.18.8-gentoo
```

In order to create a symbolic link called **linux**, use:

```
root # eselect kernel set 1
```

```
root # ls -l /usr/src/linux
```

```
lrwxrwxrwx   1 root   root   12 Oct 13 11:04 /usr/src/linux ->
linux-6.18.8-gentoo
```

Manually configuring a kernel is commonly seen as one of the most difficult procedures a system administrator has to perform. Nothing is less true - after configuring a few kernels no one remembers that it was difficult! There are two ways for a Gentoo user to manage a manual kernel system, both of which are listed below:

Important

Only one selection is required in the following subsection, if unsure of which to use then go with the first listed for now. It's always possible to switch at a later date if required.

Option 1 - Modprobed-db process

A very easy way to manage the kernel is to first install `sys-kernel/gentoo-kernel-bin` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>)  and use the `sys-kernel/modprobed-db` (<https://packages.gentoo.org/packages/sys-kernel/modprobed-db>) 

to collect information about what the system requires. **modprobed-db** is a tool which monitors the system via crontab to add all modules of all devices over the system's life to make sure it everything a user needs is supported. For example, if an Xbox controller is added after installation, then **modprobed-db** will add the modules to be built next time the kernel is rebuilt.

For now please follow installing

`sys-kernel/gentoo-kernel-bin` (<https://packages.gentoo.org/packages/sys-kernel/gentoo-kernel-bin>) [Ⓐ] via `Distribution_kernels` (/wiki/Handbook:AMD64/Installation/Kernel#Distribution_kernels).

Next, install

`sys-kernel/modprobed-db` (<https://packages.gentoo.org/packages/sys-kernel/modprobed-db>) [Ⓐ]:

```
root # emerge --ask sys-kernel/modprobed-db
```

Please watch out for further steps related to `modprobed-db` in the Handbook.

More on this topic can be found in the `Modprobed-db` (</wiki/Modprobed-db>) article.

Option 2 - Assisted manual process

This method allows a user to have full control of how their kernel is built with as minimal help from outside tools as they wish. Some could consider this as making it hard for the sake of it.

However, with this choice one thing is true: it is vital to know the system when a kernel is configured manually. Most information can be gathered by emerging `sys-apps/pciutils` (<https://packages.gentoo.org/packages/sys-apps/pciutils>) [Ⓐ] which contains the **lspci** command:

```
root # emerge --ask sys-apps/pciutils
```

Note

Inside the chroot, it is safe to ignore any `pcilib` warnings (like *pcilib: cannot open /sys/bus/pci/devices*) that **lspci** might throw out.

Another source of system information is to run **lsmod** to see what kernel modules the installation CD uses as it might provide a nice hint on what to enable.

Now go to the kernel source directory.

```
root # cd /usr/src/linux
```

Tip

To view the full list of **make** arguments available for the kernel, run `make help`.

The kernel has a method of autodetecting the modules currently being used on the installed CD which will give a great starting point to allow a user to configure their own. This can be called by using:

```
root # make localmodconfig
```

Manually configuring should not be needed at this point, but if a user wish to check:

```
root # make nconfig
```

Now it's time to decide if modules signing is required in the steps listed in here (/wiki/Handbook:AMD64/Installation/Kernel#Optional:_Signed_kernel_modules_2)

If not, proceed to building described here (/wiki/Handbook:AMD64/Installation/Kernel#Compiling_and_installing)

Option 3 - Configuring by hand

The Linux kernel configuration has many, many sections and as configuring a kernel by hand is rarely needed thanks to the two tools listed above. The steps to do it by hand are now included at Kernel/Gentoo_Kernel_Configuration_Guide (/wiki/Kernel/Gentoo_Kernel_Configuration_Guide)

Optional: Signed kernel modules

To automatically sign the kernel modules enable `CONFIG_MODULE_SIG_ALL`:

KERNEL Sign kernel modules `CONFIG_MODULE_SIG_ALL`

```
[*] Enable loadable module support
  *-   Module signature verification
    [*]   Automatically sign all modules
    Which hash algorithm should modules be signed with? (Sign modules with
    SHA-512) --->
```

Optionally change the hash algorithm if desired.

To enforce that all modules are signed with a valid signature, enable `CONFIG_MODULE_SIG_FORCE` as well:

KERNEL Enforce signed kernel modules `CONFIG_MODULE_SIG_FORCE`

```
[*] Enable loadable module support
  *-   Module signature verification
    [*]   Require modules to be validly signed
    [*]   Automatically sign all modules
    Which hash algorithm should modules be signed with? (Sign modules with
    SHA-512) --->
```

To use a custom key, specify the location of this key in `CONFIG_MODULE_SIG_KEY`. If unspecified, the kernel build system will generate a key. It is recommended to generate one manually instead. This can be done with:

```
root # openssl req -new -nodes -utf8 -sha256 -x509 -outform PEM -out
kernel_key.pem -keyout kernel_key.pem
```

OpenSSL will ask some questions about the user generating the key, it is recommended to fill in these questions as detailed as possible.

Store the key in a safe location, at the very least the key should be readable only by the root user. Verify this with:

```
root # ls -l kernel_key.pem
```

```
-r----- 1 root root 3164 Jan  4 10:38 kernel_key.pem
```

If this outputs anything other than the above, correct the permissions with:

```
root # chown root:root kernel_key.pem
root # chmod 400 kernel_key.pem
```

KERNEL Specify signing key `CONFIG_MODULE_SIG_KEY`

```
-*- Cryptographic API --->
  Certificates for signature checking --->
    (/path/to/kernel_key.pem) File name or PKCS#11 URI of module signing
key
```

To also sign external kernel modules installed by other packages via `linux-mod-r1.eclass`, enable the `modules-sign` (<https://packages.gentoo.org/useflags/modules-sign>)  (/wiki/USE_flag) USE flag globally:

FILE `/etc/portage/make.conf` **Enable module signing**

```
USE="modules-sign"

# Optionally, when using custom signing keys.
MODULES_SIGN_KEY="/path/to/kernel_key.pem"
MODULES_SIGN_CERT="/path/to/kernel_key.pem" # Only required if the
MODULES_SIGN_KEY does not also contain the certificate
MODULES_SIGN_HASH="sha512" # Defaults to sha512
```

Note

MODULES_SIGN_KEY and **MODULES_SIGN_CERT** may point to different files. For this example, the pem file generated by OpenSSL includes both the key and the accompanying certificate, and thus both variables are set to the same value.

Optional: Signing the kernel image (Secure Boot)

When signing the kernel image (for use on systems with Secure Boot (/wiki/Secure_Boot) enabled) it is recommended to set the following kernel config options:

KERNEL **Lockdown for secureboot**

```

General setup --->
    Kexec and crash features --->
        [*] Enable kexec system
call

        [*] Enable kexec file based system
call

        [*]   Verify kernel signature during kexec_file_load()
syscall
        [*]   Require a valid signature in kexec_file_load()
syscall
        [*]   Enable ""image"" signature verification support

[*] Enable loadable module support
    *-   Module signature verification
        [*]   Require modules to be validly signed
        [*]   Automatically sign all modules
        Which hash algorithm should modules be signed with? (Sign modules with
SHA-512) --->

Security options --->
[*] Integrity subsystem
    [*] Basic module for enforcing kernel
lockdown

    [*]   Enable lockdown LSM early in
init
        Kernel default lockdown mode (Integrity) --->

[*]   Digital signature verification using multiple
keyrings
    [*]   Enable asymmetric keys
support

    *-   Require all keys on the integrity keyrings be
signed
    [*]   Provide keyring for platform/firmware trusted
keys
    [*]   Provide a keyring to which Machine Owner Keys may be
added
    [ ]   Enforce Machine Keyring CA Restrictions

```

Where ""image"" is a placeholder for the architecture specific image name. These options, from the top to the bottom: enforces that the kernel image in a kexec call must be signed (kexec allows replacing the kernel in-place), enforces that kernel modules are signed, enables lockdown *integrity* mode (prevents modifying the kernel at runtime), and enables various keychains.

On arches that do not natively support decompressing the kernel (e.g. **arm64** and **riscv**), the kernel must be built with its own decompressor (zboot):

KERNEL **zboot CONFIG_EFI_ZBOOT**

```

Device Drivers ---
>

Firmware Drivers ---
>

EFI (Extensible Firmware Interface) Support ---
>

[*] Enable the generic EFI decompressor

```

After compilation of the kernel, as explained in the next section, the kernel image must be signed. First install

`app-crypt/sbsigntools` (<https://packages.gentoo.org/packages/app-crypt/sbsigntools>)  and then sign the kernel image:

```

root # emerge --ask app-crypt/sbsigntools
root # sbsign /usr/src/linux-x.y.z/path/to/kernel-image --cert /path/to/
kernel_key.pem --key /path/to/kernel_key.pem --output /usr/src/linux-x.y.z/path/
to/kernel-image

```

Note

For this example, the same key that was generated to sign the modules is used to sign the kernel image. It is also possible to generate and use a second separate key for signing the kernel image. The same OpenSSL command as in the previous section may be used again.

Then proceed with the installation.

To automatically sign EFI executables installed by other packages, enable the `secureboot` (<https://packages.gentoo.org/useflags/secureboot>)  (`/wiki/USE_flag`) USE flag globally:

FILE `/etc/portage/make.conf` **Enable Secure Boot**

```

USE="modules-sign secureboot"

# Optionally, to use custom signing keys.
MODULES_SIGN_KEY="/path/to/kernel_key.pem"
MODULES_SIGN_CERT="/path/to/kernel_key.pem" # Only required if the
MODULES_SIGN_KEY does not also contain the certificate.
MODULES_SIGN_HASH="sha512" # Defaults to sha512

# Optionally, to boot with secureboot enabled, may be the same or different
signing key.
SECUREBOOT_SIGN_KEY="/path/to/kernel_key.pem"
SECUREBOOT_SIGN_CERT="/path/to/kernel_key.pem"

```

Note

SECUREBOOT_SIGN_KEY and **SECUREBOOT_SIGN_CERT** may point to different files. For this

example, the pem file generated by OpenSSL includes both the key and the accompanying certificate, and thus both variables are set to the same value.

Note

When generating an Unified Kernel Image (/wiki/Unified_Kernel_Image) with systemd's `ukify` the kernel image will be signed automatically before inclusion in the unified kernel image and it is not necessary to sign it manually.

Compiling and installing

With the configuration now done, it is time to compile and install the kernel. Exit the configuration and start the compilation process:

```
root # make -j$(nproc) && make modules_install
```

Note

It is possible to lower the parallel builds using **make -jX** with `x` being an integer number of parallel tasks that the build process is allowed to launch. This is similar to the instructions about `/etc/portage/make.conf` earlier, with the `MAKEOPTS` variable.

When the kernel has finished compiling, copy the kernel image to `/boot/`. This is handled by the **make install** command:

```
root # make install
```

This command will copy the kernel image to `/boot`. If

`sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>)  is installed it will call `/sbin/installkernel` instead and delegate the kernel installation. Instead of simply copying the kernel to `/boot`, `Installkernel` (</wiki/Installkernel>) installs each kernel with its version number in the file name. Additionally, `installkernel` provides a framework for automatically accomplishing various tasks relating to kernel installation, such as: generating an `initramfs` (</wiki/Initramfs>), building an Unified Kernel Image (/wiki/Unified_Kernel_Image), and updating the bootloader (</wiki/Bootloader>) configuration.

Continue the installation with [Configuring the system](/wiki/Handbook:AMD64/Installation/System) (</wiki/Handbook:AMD64/Installation/System>).

Filesystem information

Filesystem labels and UUIDs

Both MBR (BIOS) and GPT include support for *filesystem* labels and *filesystem* UUIDs. These attributes can be defined in `/etc/fstab` as alternatives for the `mount` command to use when attempting to find and mount block devices. Filesystem labels and UUIDs are identified by the `LABEL` and `UUID` prefix and can be viewed with the `blkid` command:

```
root # blkid
```

⚠ Warning

If the filesystem inside a partition is wiped, then the filesystem label and the UUID values will be subsequently altered or removed.

For uniqueness, readers who are using MBR-style partition tables are advised to use UUIDs rather than labels to specify mountable volumes in `/etc/fstab`.

❗ Important

UUIDs of the filesystem on a LVM volume and its LVM snapshots are identical, therefore using UUIDs to mount LVM volumes should be avoided.

Partition labels and UUIDs

Systems with GPT disklabel support offer additional 'robust' options to define partitions in `/etc/fstab`. Partition labels and partition UUIDs can be used to identify the block device's individual partition(s), regardless of what filesystem has been chosen for the partition itself. Partition labels and UUIDs are identified by the `PARTLABEL` and/or `PARTUUID` prefixes and can be viewed nicely in the terminal by running the `blkid` command.

Output for an **amd64** EFI system using the Discoverable Partition Specification UUIDs may like the following:

```
root # blkid
```

```
/dev/sr0: BLOCK_SIZE="2048" UUID="2023-08-28-03-54-40-00" LABEL="ISOIMAGE"  
TYPE="iso9660" PTTYPE="PMBR"  
/dev/loop0: TYPE="squashfs"  
/dev/sda2: PARTUUID="0657fd6d-a4ab-43c4-84e5-0933c84b4f4f"  
/dev/sda3: PARTUUID="1cdf763a-5b4c-4dbf-99db-a056c504e8b2"  
/dev/sda1: PARTUUID="c12a7328-f81f-11d2-ba4b-00a0c93ec93b"
```

While not always true for partition labels, using a UUID to identify a partition in `fstab` provides a guarantee that the bootloader will not be confused when looking for a certain volume, even if the *filesystem* is changed or re-written in the future. Using the older default block device files (`/dev/sd*N`) for defining the partitions in `fstab` is risky for systems that have SATA block devices regularly added or removed.

The naming for block device files depends on a number of factors, including how and in what order the disks are attached to the system. They also could show up in a different order depending on which of the devices are detected by the kernel first during the early boot

process. With this being stated, unless the system administrator intends to constantly fiddle with the disk ordering, using default block device files is a simple and straightforward approach.

About fstab

Under Linux, all partitions used by the system must be listed in **/etc/fstab** ([/wiki//etc/fstab](#)). This file contains the mount points of those partitions (where they are seen in the file system structure), how they should be mounted and with what special options (automatically or not, whether users can mount them or not, etc.)

Creating the fstab file

Note

If the init system being used is systemd, the partition UUIDs conform to the Discoverable Partition Specification as given in Preparing the disks ([/wiki/Handbook:AMD64/Installation/Disks](#)), and the system uses UEFI, then creating an fstab can be skipped, since systemd auto-mounts partitions that follow the spec.

The **/etc/fstab** file uses a table-like syntax. Every line consists of six fields, separated by whitespace (space(s), tabs, or a mixture of the two). Each field has its own meaning:

1. The first field shows the block special device or remote filesystem to be mounted. Several kinds of device identifiers are available for block special device nodes, including paths to device files, filesystem labels and UUIDs, and partition labels and UUIDs.
2. The second field shows the mount point at which the partition should be mounted.
3. The third field shows the type of filesystem used by the partition.
4. The fourth field shows the mount options used by **mount** when it wants to mount the partition. As every filesystem has its own mount options, so system admins are encouraged to read the mount man page (**man mount**) for a full listing. Multiple mount options are comma-separated.
5. The fifth field is used by dump to determine if the partition needs to be dumped or not. This can generally be left as `0` (zero).
6. The sixth field is used by **fsck** to determine the order in which filesystems should be checked if the system wasn't shut down properly. The root filesystem should have `1` while the rest should have `2` (or `0` if a filesystem check is not necessary).

Important

The default **/etc/fstab** file provided in Gentoo stage files is *not* a valid **fstab** file but instead a template that can be used to enter in relevant values.

```
root # nano /etc/fstab
```

DOS/Legacy BIOS systems

Let us take a look at how to write down the options for the **/boot** partition. This is just an example, and should be modified according to the partitioning decisions made earlier in the installation. In the **amd64** partitioning example, **/boot** is usually the **/dev/sda1** partition, with xfs recommended for the filesystem. It needs to be checked during boot, so we would write down:

FILE /etc/fstab An example DOS/Legacy BIOS boot line for /etc/fstab

```
# Adjust for any formatting differences and/or additional partitions
created from the "Preparing the disks" step
/dev/sda1  /boot      xfs      defaults      0 2
```

Some system administrators want the **/boot** partition to not be mounted automatically to improve their system's security. Those people should substitute the `defaults` with `noauto`. This does mean that those users will need to manually mount this partition every time they want to use it.

Add the rules that match the previously decided partitioning scheme and append rules for devices such as CD-ROM drive(s), and of course, if other partitions or drives are used, for those too.

Below is a more elaborate example of an **/etc/fstab** file:

FILE /etc/fstab A full /etc/fstab example for a DOS/Legacy BIOS system

```
# Adjust for any formatting differences and/or additional partitions
created from the "Preparing the disks" step
/dev/sda1  /boot      xfs      defaults      0 2
/dev/sda2  none       swap     sw            0 0
/dev/sda3  /          xfs      defaults,noatime 0 1

/dev/cdrom  /mnt/cdrom auto     noauto,user    0 0
```

UEFI systems

Below is an example of an **/etc/fstab** file for a system that will boot via UEFI firmware:

FILE /etc/fstab A full /etc/fstab example for an UEFI system

```
# Adjust for any formatting differences and/or additional partitions
created from the "Preparing the disks" step
/dev/sda1  /efi       vfat     umask=0077,tz=UTC 0 2
/dev/sda2  none       swap     sw            0 0
/dev/sda3  /          xfs      defaults,noatime 0 1

/dev/cdrom  /mnt/cdrom auto     noauto,user    0 0
```

DPS UEFI PARTUUID

Below is an example of an **/etc/fstab** file for a disk formatted with a GPT disklabel and Discoverable Partition Specification (DPS) UUIDs set for UEFI firmware:

FILE /etc/fstab DPS PARTUUID fstab example


```
# Adjust any formatting difference and additional partitions created from
the "Preparing the disks" step.
# This example shows a GPT disklabel with Discoverable Partition
Specification (DPS) UUID set:
PARTUUID=c12a7328-f81f-11d2-ba4b-00a0c93ec93b    /efi            vfat
umask=0077,tz=UTC                                0 2
PARTUUID=0657fd6d-a4ab-43c4-84e5-0933c84b4f4f    none            swap
sw                                                0 0
PARTUUID=4f68bce3-e8cd-4db1-96e7-fbcaf984b709    /              xfs
defaults,noatime                                0 1
```

When `auto` is used in the third field, it makes the **mount** command guess what the filesystem would be. This is recommended for removable media as they can be created with one of many filesystems. The `user` option in the fourth field makes it possible for non-root users to mount the CD.

To improve performance, most users would want to add the `noatime` mount option, which results in a faster system since access times are not registered (those are not needed generally anyway). This is also recommended for systems with solid state drives (SSDs). Users may wish to consider `lazytime` instead.

Tip

Due to degradation in performance, defining the `discard` mount option in **/etc/fstab** is not recommended. It is generally better to schedule block discards on a periodic basis using a job scheduler such as **cron** or a timer (systemd). See [Periodic fstrim jobs \(/wiki/SSD#Periodic_fstrim_jobs\)](#) for more information.

Double-check the **/etc/fstab** file, then save and quit to continue.

Networking information

It is important to note the following sections are provided to help the reader quickly setup their system to partake in a local area network.

For systems running OpenRC, a more detailed reference for network setup is available in the advanced network configuration ([/wiki/Handbook:AMD64/Networking/Introduction](#)) section, which is covered near the end of the handbook. Systems with more specific network needs may need to skip ahead, then return here to continue with the rest of the installation.

For more specific systemd network setup, please review see the networking portion ([/wiki/Systemd#Network](#)) of the systemd ([/wiki/Systemd](#)) article.

Hostname

One of the choices the system administrator has to make is name their PC. This seems to be quite easy, but lots of users are having difficulties finding the appropriate name for the hostname. To speed things up, know that the decision is not final - it can be changed afterwards. In the examples below, the hostname *tux* is used.

Set the hostname (OpenRC or systemd)

```
root # echo tux > /etc/hostname
```

systemd

To set the system hostname for a system currently *running* systemd, the **hostnamectl** utility may be used. During the installation process however, `systemd-firstboot` ([/wiki/Handbook:AMD64/Installation/System#Init_and_boot_configuration_systemd](#)) command must be used instead (see later on in handbook).

For setting the hostname to "tux", one would run:

```
root # hostnamectl hostname tux
```

View help by running **hostnamectl --help** or **man 1 hostnamectl**.

The hosts file

❗ Important

On OpenRC systems, the hostname chosen above *must* be configured in **/etc/hosts**.

An important next step may be to inform this new system about other hosts in its network environment. Network host names can be defined in the **/etc/hosts** file. Adding host names here will enable host name to IP addresses resolution for hosts that are not resolved by the nameserver.

```
root # nano /etc/hosts
```

FILE /etc/hosts **Filling in the networking information**

```
# This defines the current system and must be set
127.0.0.1      tux.homenetwork tux localhost
::1           tux.homenetwork tux localhost

# Optional definition of extra systems on the network
192.168.0.5    jenny.homenetwork jenny
192.168.0.6    benny.homenetwork benny
```

Save and exit the editor to continue.

Network

The Gentoo Linux installation process involves configuring networking in *two parts*. The configuration done in earlier steps enables networking for the *live environment*. Now it is time to apply the network configuration that will be used in the *installed environment*.

More detailed information about networking, including advanced topics like bonding, bridging, 802.1Q VLANs or wireless networking is covered in the advanced network configuration (</wiki/Handbook:AMD64/Networking/Introduction>) section.

❗ Important

Users should avoid using multiple network management daemons (eg. Network Manager, netifrc, systemd-networkd).

📝 Note

This section is not exhaustive and covers a few common methods.

DHCP via dhcpcd (any init system)

💡 Tip

Ethernet users with a home router generally prefer this option.

DHCP provides a reliable way for systems to obtain an IP address automatically. Most LANs operate a DHCP server, usually integrated with a router which provides internet access.

Systems that do not use WiFi can use **dhcpcd** directly to configure the network automatically. Alternatively, DHCP can be used by the other methods described in the next sections, which either provide their own DHCP client implementation, or use `net-misc/dhcpcd` (<https://packages.gentoo.org/packages/net-misc/dhcpcd>) .

To install:

```
root # emerge --ask net-misc/dhcpcd
```

To enable the service on OpenRC systems:

```
root # rc-update add dhcpcd default
```

To enable the service on systemd systems:

```
root # systemctl enable dhcpcd
```

The next time the system boots, **dhcpcd** should obtain an IP address from the DHCP server. See the [Dhcpcd](/wiki/Dhcpcd) (</wiki/Dhcpcd>) article for more details.

NetworkManager (any init system)

💡 Tip

WiFi users generally prefer this option.

NetworkManager (</wiki/NetworkManager>) provides a simple method to manage networks using Ethernet, WiFi, PPPoE or cellular via CLI, TUI or GUI.

First add the

`networkmanager` (<https://packages.gentoo.org/useflags/networkmanager>)  (/wiki/USE_flag) USE flag to `/etc/portage/make.conf`:

FILE `/etc/portage/make.conf`

```
...
USE="${USE} networkmanager"
...
```

Non desktop profiles will also need to add:

FILE `/etc/portage/package.use/networkmanager`

```
net-wireless/wpa_supplicant dbus
```

Next, emerge the package:

```
root # emerge --ask net-misc/networkmanager
```

Finally, enable the service to start at boot time:

OpenRC:

```
root # rc-update add NetworkManager default
systemd:
```

```
root # systemctl enable NetworkManager
```

For more information see the NetworkManager (</wiki/NetworkManager>) article.

netifrc (OpenRC)

Configuring the network

All networking information is gathered in `/etc/conf.d/net`. It uses a straightforward - yet perhaps not intuitive - syntax. Do not fear! Everything is explained below. A fully commented example that covers many different configurations is available in `/usr/share/doc/netifrc-*/net.example.bz2`.

First install `net-misc/netifrc` (<https://packages.gentoo.org/packages/net-misc/netifrc>) [Ⓐ]:

```
root # emerge --ask --noreplace net-misc/netifrc
```

DHCP is used by default. For DHCP to work, a DHCP client needs to be installed.

If the network connection needs to be configured because of specific DHCP options or because DHCP is not used at all, then open `/etc/conf.d/net`:

```
root # nano /etc/conf.d/net
```

Set both `config_eth0` and `routes_eth0` to enter IP address information and routing information:

Note

This assumes that the network interface will be called `eth0`. This is, however, very system dependent. It is recommended to assume that the interface is named the same as the interface name when booted from the installation media if the installation media is sufficiently recent. More information can be found in the Network interface naming (/wiki/Handbook:AMD64/Networking/Advanced#Network_interface_naming) section.

FILE `/etc/conf.d/net` **Static IP definition**

```
config_eth0="192.168.0.2 netmask 255.255.255.0 brd 192.168.0.255"
routes_eth0="default via 192.168.0.1"
```

To use DHCP, define `config_eth0`:

FILE `/etc/conf.d/net` **DHCP definition**

```
config_eth0="dhcp"
```

Please read `/usr/share/doc/netifrc-*/net.example.bz2` for a list of additional configuration options. Be sure to also read up on the DHCP client man page if specific DHCP options need to be set.

If the system has several network interfaces, then repeat the above steps for `config_eth1`, `config_eth2`, etc.

Now save the configuration and exit to continue.

Automatically start networking at boot

To have the network interfaces activated at boot, they need to be added to the default runlevel.

```
root # cd /etc/init.d
root # ln -s net.lo net.eth0
root # rc-update add net.eth0 default
```

If the system has several network interfaces, then the appropriate `net.*` files need to be created just like we did with `net.eth0`.

If, after booting the system, it is discovered the network interface name (which is currently documented as `eth0`) was wrong, then execute the following steps to rectify:

1. Update the `/etc/conf.d/net` file with the correct interface name (like `enp3s0` or `enp5s0`, instead of `eth0`).
2. Create new symbolic link (like `/etc/init.d/net.enp3s0`).
3. Remove the old symbolic link (`rm /etc/init.d/net.eth0`).
4. Add the new one to the default runlevel.
5. Remove the old one using `rc-update del net.eth0 default`.

systemd-networkd (systemd)

systemd (/wiki/Systemd) has it's own network management like OpenRC has `netifrc`.

To configure **systemd-networkd**, create a `*.network` file under the `/etc/systemd/network` directory. Files are written in INI style syntax, and usually given a name starting with two digits which determines the order they are loaded. It is recommended to use numbers lower than 70 for machine-specific configuration, to ensure it is not overridden by **systemd**-provided files. See the `systemd.network(5)` (<https://man.archlinux.org/man/systemd.network.5.en>)[ⓘ] (/wiki/Special:MyLanguage/man_page) man page for reference.

A `*.network` file contains a `[Match]` section, which specifies which interfaces the file will be applied to, and further sections, especially `[Network]` which describe the configuration to applied. As configuration files are loaded in alphabetical order, the first file matching an interface is used. As such, configuration files with less specific `[Match]` criteria would usually be given names with higher numbers than those with more specific criteria.

Below are some examples:

Wired DHCP connection:

FILE `/etc/systemd/network/50-dhcp.network` **DHCP**

```
[Match]
Name=en*
```

```
[Network]
DHCP=yes
```

Alternatively, **systemd-networkd** provides some useful configurations in **/usr/share/systemd/network**, which can be enabled by symlinking from **/etc/systemd/network**. A configuration similar to the above is **89-ethernet.network.example**.

```
root # ln -s /usr/share/systemd/network/89-ethernet.network.example /etc/systemd/network/89-ethernet.network
```

Wired connection with static IP address (uses CIDR (<https://en.wikipedia.org/wiki/CIDR>) notation for subnetting):

FILE **/etc/systemd/network/50-static.network** **Static**

[Match]

Name=enp1s0

[Network]

Address=192.168.1.10/24

Gateway=192.168.1.1

DNS=192.168.1.1

WiFi can also be configured with the help of Wpa supplicant (/wiki/Wpa_supplicant) or lwd (</wiki/lwd>):

FILE **/etc/systemd/network/50-wireless.network** **WiFi**

[Match]

Name=wlan0

[Network]

DHCP=yes

IgnoreCarrierLoss=3s

To enable the service to run at next boot up:

```
root # systemctl enable systemd-networkd.service
```

Optional: Networking tools

❗ Important

After rebooting into the new system, **wireless networks will be unavailable** (SSIDs unlisted) if they use the **insecure and obsolete** *WEP* (*WEP*) or *TKIP* (*WPA* or *WPA+WPA2*) protocols, even if they might be available from the boot-media system. There is an insecure workaround (/wiki/Warning_about_insecure_obsolete_network_hardware#Insecure_workaround) to enable connecting to these obsolete networks which can be used before reboot, though users should be aware of the security implications.

If extra tools are required when using **netifrc** or **systemd-networkd** to configure networking on the first reboot, it is necessary to install them now while networking is available in the live environment.

Installing a PPPoE client

If PPP is used to connect to the internet, install the `net-dialup/ppp` (<https://packages.gentoo.org/packages/net-dialup/ppp>)  package:

```
root # emerge --ask net-dialup/ppp
```

Install wireless networking tools

If the system will be connecting to wireless networks, install the `net-wireless/iw` (<https://packages.gentoo.org/packages/net-wireless/iw>) [Ⓐ] package for Open or WEP networks and/or the `net-wireless/wpa_supplicant` (https://packages.gentoo.org/packages/net-wireless/wpa_supplicant) [Ⓐ] package for WPA or WPA2 networks. `iw` is also a useful basic diagnostic tool for scanning wireless networks.

```
root # emerge --ask net-wireless/iw net-wireless/wpa_supplicant
```

System information

Root password

Set the root password using the `passwd` command.

```
root # passwd
```

Later an additional regular user account will be created for daily operations.

Init and boot configuration

OpenRC

When using OpenRC with Gentoo, it uses `/etc/rc.conf` to configure the services, startup, and shutdown of a system. Open up `/etc/rc.conf` and enjoy all the comments in the file. Review the settings and change where needed.

```
root # nano /etc/rc.conf
```

Next, open `/etc/conf.d/keymaps` to handle keyboard configuration. Edit it to configure and select the right keyboard.

```
root # nano /etc/conf.d/keymaps
```

Take special care with the `keymap` variable. If the wrong keymap is selected, then weird results will come up when typing on the keyboard.

Finally, edit `/etc/conf.d/hwclock` to set the clock options. Edit it according to personal preference.

```
root # nano /etc/conf.d/hwclock
```

If the hardware clock is not using UTC, then it is necessary to set `clock="local"` in the file. Otherwise the system might show clock skew behavior.

Systems setup from a desktop stage3 should run `dbus-uuidgen` at this point:

```
root # dbus-uuidgen --ensure=/etc/machine-id
```

This isn't required on non desktop stage3 created systems as `sys-apps/dbus` (<https://packages.gentoo.org/packages/sys-apps/dbus>) [Ⓐ] will automatically generate `/etc/machine-id` when the package is first installed.

systemd

First, it is recommended to run **systemd-machine-id-setup** and then **systemd-firstboot** which will prepare various components of the system are set correctly for the first boot into the new systemd environment. The passing the following options will include a prompt for the user to set a locale, timezone, hostname, root password, and root shell values. It will also assign a random machine ID to the installation:

```
root # systemd-machine-id-setup
root # systemd-firstboot --prompt
```

Next users should run **systemctl** to reset all installed unit files to the preset policy values:

Note

It is possible that a failure warning will print out after running the following command. This is normal, as Gentoo's LiveCD uses OpenRC.

```
root # systemctl preset-all --preset-mode=enable-only
```

It's possible to run the full preset changes but this may reset any services which were already configured during the process:

```
root # systemctl preset-all
```

These two steps will help ensure a smooth transition from the live environment to the installation's first boot.

System logger

OpenRC

Some tools are missing from the stage3 archive because several packages provide the same functionality. It is now up to the user to choose which ones to install.

The first tool decision is a logging mechanism for the system. Unix and Linux have an excellent history of logging capabilities - if needed, everything that happens on the system can be logged in a log file.

Gentoo offers several system logger utilities. A few of these include:

- `sysklogd` (</wiki/Sysklogd>)
(`app-admin/sysklogd` (<https://packages.gentoo.org/packages/app-admin/sysklogd>) [Ⓐ]) -
Offers the traditional set of system logging daemons. The default logging configuration works well out of the box which makes this package a good option for beginners.
- `syslog-ng` (</wiki/Syslog-ng>)
(`app-admin/syslog-ng` (<https://packages.gentoo.org/packages/app-admin/syslog-ng>) [Ⓐ])
- An advanced system logger. Requires additional configuration for anything beyond

logging to one big file. More advanced users may choose this package based on its logging potential; be aware additional configuration is a necessity for any kind of smart logging.

- `metalog` ([/wiki/Metalog](#))
(`app-admin/metalog` (<https://packages.gentoo.org/packages/app-admin/metalog>) $\triangle$) - A highly-configurable system logger.

There may be other system logging utilities available through the Gentoo ebuild repository as well, since the number of available packages increases on a daily basis.

✔ Tip

If `syslog-ng` is going to be used, it is recommended to install and configure `logrotate` ([/wiki/Logrotate](#)). `syslog-ng` does not provide any rotation mechanism for the log files. Newer versions (≥ 2.0) of `sysklogd` however handle their own log rotation.

To install the system logger of choice, emerge it. On OpenRC, add it to the default runlevel using **`rc-update`**. The following example installs and activates `app-admin/sysklogd` (<https://packages.gentoo.org/packages/app-admin/sysklogd>) $\triangle$ as the system's syslog utility:

```
root # emerge --ask app-admin/sysklogd
root # rc-update add sysklogd default
```

systemd

While a selection of logging mechanisms are presented for OpenRC-based systems, `systemd` includes a built-in logger called the **`systemd-journald`** service. The `systemd-journald` service is capable of handling most of the logging functionality outlined in the previous system logger section. That is to say, the majority of installations that will run `systemd` as the system and service manager can *safely skip* adding a additional syslog utilities.

See **`man journalctl`** for more details on using **`journalctl`** to query and review the systems logs.

For a number of reasons, such as the case of forwarding logs to a central host, it may be important to include *redundant* system logging mechanisms on a `systemd`-based system. This is a irregular occurrence for the handbook's typical audience and considered an advanced use case. It is therefore not covered by the handbook.

Cron daemon

OpenRC

Although it is optional and not required for every system, it is wise to install a cron daemon.

A cron daemon executes commands on scheduled intervals. Intervals could be daily, weekly, or monthly, once every Tuesday, once every other week, etc. A wise system administrator will leverage the cron daemon to automate routine system maintenance tasks.

All cron daemons support high levels of granularity for scheduled tasks, and generally include the ability to send an email or other form of notification if a scheduled task does not complete as expected.

Gentoo offers several possible cron daemons, including:

- `cronie` (</wiki/Cron#cronie>)
(`sys-process/cronie` (<https://packages.gentoo.org/packages/sys-process/cronie>) [Ⓐ]) -
cronie is based on the original cron and has security and configuration enhancements like the ability to use PAM and SELinux.
- `dcron` (/wiki/Cron#dcron_.28Dillon.27s_Cron.29)
(`sys-process/dcron` (<https://packages.gentoo.org/packages/sys-process/dcron>) [Ⓐ]) -
This lightweight cron daemon aims to be simple and secure, with just enough features to stay useful.
- `fcron` (</wiki/Cron#fcron>)
(`sys-process/fcron` (<https://packages.gentoo.org/packages/sys-process/fcron>) [Ⓐ]) - A
command scheduler with extended capabilities over cron and anacron.
- `bcron` (</wiki/Cron#bcron>)
(`sys-process/bcron` (<https://packages.gentoo.org/packages/sys-process/bcron>) [Ⓐ]) - A
younger cron system designed with secure operations in mind. To do this, the system is divided into several separate programs, each responsible for a separate task, with strictly controlled communications between parts.

Default: cronie

The following example uses

`sys-process/cronie` (<https://packages.gentoo.org/packages/sys-process/cronie>) [Ⓐ] (not to be confused with the NTP (/wiki/Network_Time_Protocol) daemon named `chrony` (</wiki/Chrony>)):

```
root # emerge --ask sys-process/cronie
```

Add cronie to the default system runlevel, which will automatically start it on power up:

```
root # rc-update add cronie default
```

Alternative: dcron

```
root # emerge --ask sys-process/dcron
```

If dcron is the go forward cron agent, an additional initialization command needs to be executed:

```
root # crontab /etc/crontab
```

Alternative: fcron

```
root # emerge --ask sys-process/fcron
```

If fcron is the selected scheduled task handler, an additional emerge step is required:

```
root # emerge --config sys-process/fcron
```

Alternative: bcron

bcron is a younger cron agent with built-in privilege separation.

```
root # emerge --ask sys-process/bcron
```

modprobed-db users

If using

`sys-kernel/modprobed-db` (<https://packages.gentoo.org/packages/sys-kernel/modprobed-db>) [Ⓐ] was picked as an option for manually compiling the kernel.

Now is the time to setup a crontab to periodically scan the system for hardware used.

FILE /etc/crontab **Run modprobed-db once every 6 hours**

```
0 */6 * * * /usr/bin/modprobed-db store &> /dev/null
```

At a later date of at least a week, please visit the kernel build section of the modprobed-db (/wiki/Modprobed-db#Building_kernels) article to complete the setup.

systemd

Similar to system logging, systemd-based systems include support for scheduled tasks out-of-the-box in the form of *timers*. systemd timers can run at a system-level or a user-level and include the same functionality that a traditional cron daemon would provide. Unless redundant capabilities are necessary, installing an additional task scheduler such as a cron daemon is generally unnecessary and can be safely skipped.

modprobed-db users

If using sys-kernel/modprobed-db (<https://packages.gentoo.org/packages/sys-kernel/modprobed-db>) was picked as an option for manually compiling the kernel.

Now is the time to setup a systemd timer to periodically scan the system for hardware used.

```
root # systemctl --user enable modprobed-db
```

At a later date of at least a week, please visit the kernel build section of the modprobed-db (/wiki/Modprobed-db#Building_kernels) article to complete the setup.

Optional: File indexing

In order to index the file system to provide faster file location capabilities, install mlocate (/wiki/Mlocate):

```
root # emerge --ask sys-apps/mlocate
```

Optional: Remote shell access

Tip

opensshd's default configuration does not allow root to login as a remote user. Please create a non-root user (/wiki/FAQ#How_do_I_add_a_normal_user.3F) and configure it appropriately to allow access post-installation if required, or adjust **/etc/ssh/sshd_config** to allow root.

To be able to access the system remotely after installation, **sshd** must be configured to start on boot.

For more in-depth details on the configuration of SSH, refer to the SSH (/wiki/SSH) article.

OpenRC

To add the **sshd** init script to the default runlevel on OpenRC:

```
root # rc-update add sshd default
```

If serial console access is needed (which is possible in case of remote servers), **agetty** must be configured.

Uncomment the serial console section in **/etc/inittab**:

```
root # nano -w /etc/inittab
```

```
# SERIAL CONSOLES
s0:12345:respawn:/sbin/agetty 9600 ttyS0 vt100
s1:12345:respawn:/sbin/agetty 9600 ttyS1 vt100
```

systemd

To enable the SSH server, run:

```
root # systemctl enable sshd
```

To enable serial console support, run:

```
root # systemctl enable getty@tty1.service
```

Optional: Shell completion

Bash

Bash is the default shell for Gentoo systems, and therefore installing completion extensions can aid in efficiency and convenience to managing the system. The `app-shells/bash-completion` (<https://packages.gentoo.org/packages/app-shells/bash-completion>) package will install completions available for Gentoo specific commands, as well as many other common commands and utilities:

```
root # emerge --ask app-shells/bash-completion
```

Post installation, bash completion for specific commands can be managed through **eselect**. See the Shell completion integrations section (/wiki/Bash#Shell_completion_integrations) of the bash article for more details.

Suggested: Time synchronization

❗ Important

Systems without a functioning Real-Time Clock (RTC) (/wiki/System_time#Software_clock_vs_Hardware_clock) must set the system time (/wiki/System_time) at every system start, and on regular intervals thereafter.

It is important to use some method of synchronizing the system clock with Internet time servers. This is often mandatory for systems without an RTC, but is also beneficial for systems with a RTC, as the battery could fail, and clock skew can accumulate.

The clock is usually synchronized via the Network Time Protocol (/wiki/Network_Time_Protocol), with an implementation such as `chrony` (</wiki/Chrony>).

chrony

Install `chrony` (not to be confused with the cron daemon named `cronie` (</wiki/Cron#cronie>)):

```
root # emerge --ask net-misc/chrony
```

See the [chrony \(/wiki/Chrony\)](/wiki/Chrony) article for further information, for example if more advanced configurations are required.

OpenRC

Add `chronyd` to the default runlevel to start it at boot, so time will be synchronized automatically:

```
root # rc-update add chronyd default
```

systemd

Start `chronyd` at boot, to have the time synchronized automatically:

```
root # systemctl enable chronyd.service
```

systemd-timesyncd

Alternatively, `systemd` users may wish to use the simpler **systemd-timesyncd** SNTP client which is installed by default and enabled with:

```
root # systemctl enable systemd-timesyncd.service
```

Filesystem tools

Depending on the filesystems used, it may be necessary to install the required file system utilities (for checking the filesystem integrity, (re)formatting file systems, etc.). Note that `ext4` user space tools

(`sys-fs/e2fsprogs` (<https://packages.gentoo.org/packages/sys-fs/e2fsprogs>) [ⓘ]) are already installed as a part of the `@system` set ([/wiki/System_set_\(Portage\)](/wiki/System_set_(Portage))).

The following table lists the tools to install if a certain filesystem tools will be needed in the installed environment.

Filesystem	Package
XFS (/wiki/XFS)	<code>sys-fs/xfsprogs</code> (https://packages.gentoo.org/packages/sys-fs/xfsprogs) [ⓘ]
ext4 (/wiki/Ext4)	<code>sys-fs/e2fsprogs</code> (https://packages.gentoo.org/packages/sys-fs/e2fsprogs) [ⓘ]
VFAT (/wiki/VFAT) (FAT32, ...)	<code>sys-fs/dosfstools</code> (https://packages.gentoo.org/packages/sys-fs/dosfstools) [ⓘ]
Btrfs (/wiki/Btrfs)	<code>sys-fs/btrfs-progs</code> (https://packages.gentoo.org/packages/sys-fs/btrfs-progs) [ⓘ]
F2FS (/wiki/F2FS)	<code>sys-fs/f2fs-tools</code> (https://packages.gentoo.org/packages/sys-fs/f2fs-tools) [ⓘ]
NTFS (/wiki/NTFS)	<code>sys-fs/ntfs3g</code> (https://packages.gentoo.org/packages/sys-fs/ntfs3g) [ⓘ]
ZFS (/wiki/ZFS)	<code>sys-fs/zfs</code> (https://packages.gentoo.org/packages/sys-fs/zfs) [ⓘ]

It's recommended that `sys-block/io-scheduler-udev-rules` (<https://packages.gentoo.org/packages/sys-block/io-scheduler-udev-rules>) be installed for the correct scheduler behavior with e.g. nvme devices:

```
root # emerge --ask sys-block/io-scheduler-udev-rules
```

Tip

For more information on filesystems in Gentoo see the filesystem article (</wiki/Filesystem>).

Now continue with Configuring the bootloader (</wiki/Handbook:AMD64/Installation/Bootloader>).

Selecting a boot loader

With the Linux kernel configured, system tools installed and configuration files edited, it is time to install the last important piece of a Linux installation: the boot loader.

The boot loader is responsible for firing up the Linux kernel upon boot - without it, the system would not know how to proceed when the power button has been pressed.

For **amd64**, we document how to configure GRUB (/wiki/Handbook:AMD64/Blocks/Bootloader#Default:_GRUB) for DOS/Legacy BIOS based systems, and GRUB, systemd-boot or EFI Stub for UEFI systems.

In this section of the Handbook a delineation has been made between *emerging* the boot loader's package and *installing* a boot loader to a system disk. Here the term *emerge* will be used to ask Portage (</wiki/Portage>) to make the software package available to the system. The term *install* will signify the boot loader copying files or physically modifying appropriate sections of the system's disk drive in order to render the boot loader *activated and ready to operate* on the next power cycle.

Default: GRUB

By default, the majority of Gentoo systems now rely upon GRUB (</wiki/GRUB>) (found in the `sys-boot/grub` (<https://packages.gentoo.org/packages/sys-boot/grub>) package), which is the direct successor to GRUB Legacy (/wiki/GRUB_Legacy). With no additional configuration, GRUB gladly supports older BIOS ("pc") systems. With a small amount of configuration, necessary before build time, GRUB can support more than a half a dozen additional platforms. For more information, consult the Prerequisites section (</wiki/GRUB#Prerequisites>) of the GRUB (</wiki/GRUB>) article.

Emerge

When using an older BIOS system supporting only MBR partition tables, no additional configuration is needed in order to emerge GRUB:

```
root # emerge --ask --verbose sys-boot/grub
```

A note for UEFI users: running the above command will output the enabled *GRUB_PLATFORMS* values before emerging. When using UEFI capable systems, users will need to ensure `GRUB_PLATFORMS="efi-64"` is enabled (as it is the case by default). If that is not the case for the setup, `GRUB_PLATFORMS="efi-64"` will need to be added to the `/etc/portage/make.conf` file *before* emerging GRUB so that the package will be built with EFI functionality:

```
root # echo 'GRUB_PLATFORMS="efi-64"' >> /etc/portage/make.conf
```

```
root # emerge --ask sys-boot/grub
```

If GRUB was somehow emerged without enabling `GRUB_PLATFORMS="efi-64"`, the line (as shown above) can be added to `make.conf` and then dependencies for the world package set (`/wiki/World_set_(Portage)`) can be re-calculated by passing the `--update --newuse` options to `emerge`:

```
root # emerge --ask --update --newuse --verbose sys-boot/grub
```

The GRUB software has now been merged onto the *system*, but it has not yet been installed as a secondary *bootloader*.

Install

Next, install the necessary GRUB files to the `/boot/grub/` directory via the `grub-install` command. Presuming the first disk (the one where the system boots from) is `/dev/sda`, one of the following commands will do:

DOS/Legacy BIOS systems

For DOS/Legacy BIOS systems:

```
root # grub-install /dev/sda
```

UEFI systems

❗ Important

Make sure the EFI system partition has been mounted *before* running `grub-install`. It is possible for `grub-install` to install the GRUB EFI file (`grubx64.efi`) into the wrong directory **without** providing *any* indication the wrong directory was used.

For UEFI systems:

```
root # grub-install --efi-directory=/efi
```

```
Installing for x86_64-efi platform.
Installation finished. No error reported.
```

Upon successful installation, the output should match the output of the previous command. If the output does not match exactly, then proceed to Debugging GRUB (`/wiki/Handbook:AMD64/Blocks/Bootloader#Debugging_GRUB`), otherwise jump to the Configure step (`/wiki/Handbook:AMD64/Blocks/Bootloader#GRUB_Configure`).

Optional: Secure Boot

To successfully boot with secure boot enabled the signing certificate must either be accepted by the UEFI (`/wiki/UEFI`) firmware, or shim (`/wiki/Shim`) must be used as a pre-loader. Shim is pre-signed with the third-party Microsoft Certificate, accepted by default by most UEFI motherboards.

How to configure the UEFI firmware to accept custom keys depends on the firmware vendor, which is beyond the scope of the handbook. Below is shown how to setup shim instead. Here it is assumed that the user has already followed the instructions in the previous sections to generate a signing key and to configure portage to use it. If this is not the case please return first to the Kernel installation ([/wiki/Handbook:AMD64/Installation/Kernel](#)) section.

The package `sys-boot/grub` (<https://packages.gentoo.org/packages/sys-boot/grub>) [Ⓐ] installs a prebuilt and signed stand-alone EFI executable if the `secureboot` (<https://packages.gentoo.org/useflags/secureboot>) [Ⓐ] ([/wiki/USE_flag](#)) USE flag is enabled. Install the required packages and copy the stand-alone grub, Shim, and the MokManager to the same directory on the EFI System Partition. For example:

```
root # emerge sys-boot/grub sys-boot/shim sys-boot/mokutil sys-boot/efibootmgr
root # cp /usr/share/shim/BOOTX64.EFI /efi/EFI/Gentoo/shimx64.efi
root # cp /usr/share/shim/mmx64.efi /efi/EFI/Gentoo/mmx64.efi
root # cp /usr/lib/grub/grub-x86_64.efi.signed /efi/EFI/Gentoo/grubx64.efi
```

Next register the signing key in shims MOKlist, this requires keys in the *DER* format, whereas **sbsign** and the kernel build system expect keys in the *PEM* format. In the previous sections of the handbook an example was shown to generate such a signing *PEM* key, this key must now be converted to the *DER* format:

```
root # openssl x509 -in /path/to/kernel_key.pem -inform PEM -out /path/to/
kernel_key.der -outform DER
```

Note

The path used here must be the path to the pem file containing the certificate belonging to the generated key. In this example both key and certificate are in the same pem file.

Then the converted certificate can be imported into Shims MOKlist, this command will ask to set some password for the import request:

```
root # mokutil --import /path/to/kernel_key.der
```

Note

When the currently booted kernel already trusts the certificate being imported, the message "Already in kernel trusted keyring." will be returned here. If this happens, re-run the above command with the argument **--ignore-keyring** added.

Next, register Shim with the UEFI firmware. In the following command, `boot-disk` and `boot-partition-id` must be replaced with the disk and partition identifier of the EFI system partition:

```
root # efibootmgr --create --disk /dev/boot-disk --part boot-partition-id --
loader '\EFI\Gentoo\shimx64.efi' --label 'GRUB via Shim' --unicode
```

Note that this prebuilt and signed stand-alone version of grub reads the *grub.cfg* from a different location than usual. Instead of the default **/boot/grub/grub.cfg** the config file should be in the same directory that the grub EFI executable is in, e.g. **/efi/EFI/Gentoo/grub.cfg**. When

`sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>) [Ⓐ] is used to install the kernel and update the grub configuration then the *GRUB_CFG* environment variable may be used to override the usual location of the grub config file.

For example:

```
root # grub-mkconfig -o /efi/EFI/Gentoo/grub.cfg
```


Or, via installkernel (/wiki/Installkernel):

FILE /etc/env.d/99grub

```
GRUB_CFG=/efi/EFI/Gentoo/grub.cfg
```

```
root # env-update
```

Note

The import process will not be completed until the system is rebooted. After completing all steps in the handbook, restart the system and Shim will load, it will find the import request registered by **mokutil**. The MokManager application will start and ask for the password that was set when creating the import request. Follow the on-screen instructions to complete the import of the certificate, then reboot the system into the UEFI menu and enable the Secure Boot setting.

Debugging GRUB

When debugging GRUB, there are a couple of quick fixes that may result in a bootable installation without having to reboot to a new live image environment.

In the event that "EFI variables are not supported on this system" is displayed somewhere in the output, it is likely the live image was not booted in EFI mode and is presently in Legacy BIOS boot mode. The solution is to try the removable GRUB step (/wiki/Handbook:AMD64/Blocks/Bootloader#GRUB_Install_EFI_systems_removable) mentioned below. This will overwrite the executable EFI file located at **/EFI/BOOT/BOOTX64.EFI**. Upon rebooting in EFI mode, the motherboard firmware may execute this default boot entry and execute GRUB.

If **grub-install** returns an error that says "Could not prepare Boot variable: Read-only file system", and the live environment was correctly booted in UEFI mode, then it should be possible to remount the efivarfs special mount as read-write and then re-run the aforementioned **grub-install** command (/wiki/Handbook:AMD64/Blocks/Bootloader#GRUB_Install_EFI_systems_command):

```
root # mount -o remount,rw,nosuid,nodev,noexec --types efivarfs efivarfs /sys/firmware/efi/efivars
```

This is caused by certain non-official Gentoo environments not mounting the special EFI filesystem by default. If the previous command does not run, then reboot using an official Gentoo live image environment in EFI mode.

Some motherboard manufacturers with poor UEFI implementations seem to *only* support the **/EFI/BOOT** directory location for the .EFI file in the EFI System Partition (ESP). The GRUB installer can create the .EFI file in this location automatically by appending the **--removable** option to the install command. Ensure the ESP has been mounted before running the following command; presuming it is mounted at **/efi** (as defined earlier), run:

```
root # grub-install --target=x86_64-efi --efi-directory=/efi --removable
```

This creates the 'default' directory defined by the UEFI specification, and then creates a file with the default name: **BOOTX64.EFI**.

Configure

Next, generate the GRUB configuration based on the user configuration specified in the **/etc/default/grub** file and **/etc/grub.d** scripts. In most cases, no configuration is needed by users as GRUB will automatically detect which kernel to boot (the highest one available in **/boot/**) and what the root file system is. It is also possible to append kernel parameters in **/etc/default/grub** using the **GRUB_CMDLINE_LINUX** variable.

To generate the final GRUB configuration, run the **grub-mkconfig** command:

```
root # grub-mkconfig -o /boot/grub/grub.cfg
```

```
Generating grub.cfg ...
Found linux image: /boot/vmlinuz-6.18.8-gentoo
Found initrd image: /boot/initramfs-genkernel-amd64-6.18.8-gentoo
done
```

The output of the command must mention that at least one Linux image is found, as those are needed to boot the system. If an **initramfs** is used or **genkernel** was used to build the kernel, the correct **initrd** image should be detected as well. If this is not the case, go to **/boot/** and check the contents using the **ls** command. If the files are indeed missing, go back to the kernel configuration and installation instructions.

Tip

The **os-prober** utility can be used in conjunction with GRUB to detect other operating systems from attached drives. Windows 7, 8.1, 10, and other distributions of Linux are detectable. Those desiring dual boot systems should emerge the **sys-boot/os-prober** (<https://packages.gentoo.org/packages/sys-boot/os-prober>) package then re-run the **grub-mkconfig** command (as seen above). If detection problems are encountered be sure to read the GRUB (</wiki/GRUB>) article in its entirety *before* asking the Gentoo community for support.

Alternative 1: systemd-boot

Another option is **systemd-boot** (</wiki/Systemd/systemd-boot>), which works on both OpenRC and systemd machines. It is a thin chainloader and works well with secure boot.

Emerge

To install **systemd-boot**, enable the **boot** (<https://packages.gentoo.org/useflags/boot>) ☒ (/wiki/USE_flag) **USE** flag and re-install **sys-apps/systemd** (<https://packages.gentoo.org/packages/sys-apps/systemd>) (for systemd systems) or **sys-apps/systemd-utils** (<https://packages.gentoo.org/packages/sys-apps/systemd-utils>) (for OpenRC systems):

FILE **/etc/portage/package.use/systemd-boot**

```
sys-apps/systemd boot
sys-apps/systemd-utils boot
```

```
root # emerge --ask sys-apps/systemd
```

Or

```
root # emerge --ask sys-apps/systemd-utils
```

Installation

Now, install the systemd-boot loader to the EFI System Partition (/wiki/EFI_System_Partition):

```
root # bootctl install
```

❗ Important

Make sure the EFI system partition has been mounted *before* running **bootctl install**.

When using this bootloader, before rebooting, verify that a new bootable entry exists using:

```
root # bootctl list
```

⚠ Warning

The kernel command line for new systemd-boot entries is read from **/etc/kernel/cmdline** or **/usr/lib/kernel/cmdline**. If neither file is present, then the kernel command line of the currently booted kernel is re-used (**/proc/cmdline**). On new installs it might therefore happen that the kernel command line of the live CD is accidentally used to boot the new kernel. The kernel command line for registered entries can be checked with:

```
root # bootctl list
```

If this does not show the desired kernel command line then create **/etc/kernel/cmdline** containing the correct kernel command line and re-install the kernel.

If no new entry exists, ensure the `sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>)[Ⓐ] package has been installed with the `systemd` (<https://packages.gentoo.org/useflags/systemd>)[Ⓑ] (/wiki/USE_flag) and `systemd-boot` (<https://packages.gentoo.org/useflags/systemd-boot>)[Ⓒ] (/wiki/USE_flag) USE flags enabled, and re-run the kernel installation.

For the distribution kernels:

```
root # emerge --ask --config sys-kernel/gentoo-kernel
```

For a manually configured and compiled kernel:

```
root # make install
```

❗ Important

When installing kernels for systemd-boot, no **root=** kernel command line argument is added by default. On systemd systems that are using an `initramfs` users may rely instead on `systemd-gpt-auto-generator` (/wiki/Systemd#Automatic_mounting_of_partitions_at_boot) to automatically find the root partition at boot. Otherwise users should manually specify the location of the root partition by setting **root=** in **/etc/kernel/cmdline** as well as any other kernel

command line arguments that should be used. And then reinstalling the kernel as described above.

Optional: Secure Boot

When the

`secureboot` (<https://packages.gentoo.org/useflags/secureboot>) $\Rightarrow$ (/wiki/USE_flag) USE flag is enabled, the `systemd-boot` EFI executable will be signed by portage automatically. Furthermore, `bootctl install` will automatically install the signed version.

To successfully boot with secure boot enabled the used certificate must either be accepted by the UEFI (</wiki/UEFI>) firmware, or shim (</wiki/Shim>) must be used as a pre-loader. Shim is pre-signed with the third-party Microsoft Certificate, accepted by default by most UEFI motherboards.

How to configure the UEFI firmware to accept custom keys depends on the firmware vendor, which is beyond the scope of the handbook. Below is shown how to setup shim instead. Here it is assumed that the user has already followed the instructions in the previous sections to generate a signing key and to configure portage to use it. If this is not the case please return first to the Kernel installation (</wiki/Handbook:AMD64/Installation/Kernel>) section.

```
root # emerge --ask sys-boot/shim sys-boot/mokutil sys-boot/efibootmgr
root # bootctl install --no-variables
root # cp /usr/share/shim/BOOTX64.EFI /efi/EFI/systemd/shimx64.efi
root # cp /usr/share/shim/mmx64.efi /efi/EFI/systemd/mmx64.efi
```

Then configure shim to load `systemd-boot` during boot:

```
root # echo "systemd-bootx64.efi," | iconv -f us-ascii -t utf-16le > /efi/EFI/systemd/options.csv
```

Shims MOKlist requires keys in the *DER* format, whereas **sbsign** and the kernel build system expect keys in the *PEM* format. In the previous sections of the handbook an example was shown to generate such a signing *PEM* key, this key must now be converted to the *DER* format:

```
root # openssl x509 -in /path/to/kernel_key.pem -inform PEM -out /path/to/kernel_key.der -outform DER
```

Note

The path used here must be the path to the pem file containing the certificate belonging to the generated key. In this example both key and certificate are in the same pem file.

Then the converted certificate can be imported into Shims MOKlist:

```
root # mokutil --import /path/to/kernel_key.der
```

Note

When the currently booted kernel already trusts the certificate being imported, the message "Already in kernel trusted keyring." will be returned here. If this happens, re-run the above command with the argument **--ignore-keyring** added.

And finally we register Shim with the UEFI firmware. In the following command, `boot-disk` and `boot-partition-id` must be replaced with the disk and partition identifier of the EFI system partition:

```
root # efibootmgr --create --disk /dev/boot-disk --part boot-partition-id --
loader '\EFI\systemd\shimx64.efi' --label 'Systemd-boot via Shim' --unicode
'\EFI\systemd\systemd-bootx64.efi'
```

Note

The import process will not be completed until the system is rebooted. After completing all steps in the handbook, restart the system and Shim will load, it will find the import request registered by **mokutil**. The MokManager application will start and ask for the password that was set when creating the import request. Follow the on-screen instructions to complete the import of the certificate, then reboot the system into the UEFI menu and enable the Secure Boot setting.

Alternative 2: EFI Stub

Computer systems with UEFI-based firmware technically do not need secondary bootloaders (e.g. GRUB) in order to boot kernels. Secondary bootloaders exist to *extend* the functionality of UEFI firmware during the boot process. Using GRUB (see the prior section) is typically easier and more robust because it offers a more flexible approach for quickly modifying kernel parameters at boot time.

System administrators who desire to take a minimalist, although more rigid, approach to booting the system can avoid secondary bootloaders and boot the Linux kernel as an EFI stub (/wiki/EFI_stub).

The `sys-boot/efibootmgr` (<https://packages.gentoo.org/packages/sys-boot/efibootmgr>)  application is a tool to used interact with UEFI firmware - the system's primary bootloader. Normally this looks like adding or removing boot entries to the firmware's list of bootable entries. It can also update firmware settings so that the Linux kernels that were previously added as bootable entries can be executed with additional options. These interactions are performed through special data structures called EFI variables (hence the need for kernel support of EFI vars).

Ensure the EFI stub (/wiki/EFI_stub) kernel article has been reviewed *before* continuing. The kernel must have specific options enabled to be directly bootable by the UEFI firmware. It may be necessary to recompile the kernel in order to build-in this support.

It is also a good idea to take a look at the **efibootmgr** (</wiki/Efibootmgr>) article for additional information.

Note

To reiterate, **efibootmgr** is *not* a requirement to boot an UEFI system; it is merely necessary to add an entry for an EFI-stub kernel into the UEFI firmware. When built appropriately with EFI stub support, the Linux kernel itself can be booted *directly*. Additional kernel command-line options can be built-in to the Linux kernel (there is a kernel configuration option called `CONFIG_CMDLINE`. Similarly, support for initramfs can be 'built-in' to the kernel as well.

To boot the kernel directly from the firmware, the kernel image must be present on the EFI System Partition (/wiki/EFI_System_Partition). This may be accomplished by enabling the `efistub` (<https://packages.gentoo.org/useflags/efistub>)  (/wiki/USE_flag) USE flag on `sys-kernel/installkernel` (<https://packages.gentoo.org/packages/sys-kernel/installkernel>) . Given that EFI Stub booting is not guaranteed to work on every UEFI system, the USE flag is stable masked, testing keywords must be accepted for `installkernel` to use this feature.

FILE `/etc/portage/package.accept_keywords/installkernel`

```
sys-kernel/installkernel  
sys-boot/uefi-mkconfig  
app-emulation/virt-firmware
```

FILE `/etc/portage/package.use/installkernel`

```
sys-kernel/installkernel efistub
```

Then reinstall `installkernel` ([/wiki/Installkernel](#)), create the `/efi/EFI/Gentoo` directory and reinstall the kernel:

```
root # emerge --ask sys-kernel/installkernel
```

```
root # mkdir -p /efi/EFI/Gentoo
```

For distribution kernels:

```
root # emerge --ask --config sys-kernel/gentoo-kernel{, -bin}
```

For manually managed kernels:

```
root # make install
```

Alternatively, when `installkernel` ([/wiki/Installkernel](#)) is not used, the kernel may be copied manually to the EFI System Partition, calling it **`bzImage.efi`**:

```
root # mkdir -p /efi/EFI/Gentoo
```

```
root # cp /boot/vmlinuz-* /efi/EFI/Gentoo/bzImage.efi
```

Install the **`efibootmgr`** package:

```
root # emerge --ask sys-boot/efibootmgr
```

Create boot entry called "Gentoo" for the freshly compiled EFI stub kernel within the UEFI firmware:

```
root # efibootmgr --create --disk /dev/sda --part 1 --label "gentoo" --loader  
"\EFI\Gentoo\bzImage.efi"
```

Note

The use of a backslash (`\`) as directory path separator is mandatory when using UEFI definitions.

If an initial RAM file system (`initramfs`) is used, copy it to the EFI System Partition as well, then add the proper boot option to it:

```
root # efibootmgr --create --disk /dev/sda --part 1 --label "gentoo" --loader  
"\EFI\Gentoo\bzImage.efi" --unicode "initrd=\EFI\Gentoo\initramfs.img"
```

Tip

Additional kernel command line options may be parsed by the firmware to the kernel by specifying them along with the `initrd=...` option as shown above.

With these changes done, when the system reboots, a boot entry called "gentoo" will be available.

Unified Kernel Image

If `installkernel` ([/wiki/Installkernel](#)) was configured to build and install unified kernel images. The unified kernel image should already be installed to the **EFI/Linux** directory on the EFI system partition, if this is not the case ensure the directory exists and then run the kernel installation again as described earlier in the handbook.

To add a direct boot entry for the installed unified kernel image:

```
root # efibootmgr --create --disk /dev/sda --part 1 --label "gentoo" --loader
"\EFI\Linux\gentoo-x.y.z.efi"
```

Other Alternatives

For other options that are not covered in the Handbook, see the full list of available bootloaders ([/wiki/Bootloader](#)).

Rebooting the system

Exit the chrooted environment and unmount all mounted partitions. Then type in that one magical command that initiates the final, true test: **reboot**.

```
(chroot) livecd # exit
livecd~# cd
livecd~# umount -l /mnt/gentoo/dev{/shm,/pts,}
livecd~# umount -R /mnt/gentoo
livecd~# reboot
```

Do not forget to remove the live image, otherwise it may be targeted again instead of the newly installed Gentoo system!

Once rebooted in the fresh Gentoo environment, continue to the [Finalizing the Gentoo installation](#) ([/wiki/Handbook:AMD64/Installation/Finalizing](#)) for important information on setting up the new installation, and finally some tips on how to start a productive Gentoo Linux journey.

Portage maintenance

Portage by default preserves copies of downloaded files, on local storage: source tarballs in **/var/cache/distfiles** and binary packages in **/var/cache/binhost/gentoo**. If an update downloads a newer version of these files, the earlier versions are still preserved.

It's good practice on a Gentoo system to keep the current versions of these source tarballs and binary packages, to be able to recover systems, in case of file corruption for example (after a certain amount of time, there isn't always a guarantee that these files will still be available for download).

Keeping the older, non current, versions of these files however can needlessly consume storage space, adding up over time, and potentially wasting space, if the older versions are not required.

Gentoo provides the gentoolkit (/wiki/Gentoolkit) suite of tools to help with system administration, which notably provides the eclean (/wiki/Eclean) tool to easily remove old versions of downloaded source tarballs and binary packages, to prevent storage requirements from growing indefinitely.

First install

app-portage/gentoolkit (<https://packages.gentoo.org/packages/app-portage/gentoolkit>) :

```
root # emerge --ask app-portage/gentoolkit
```

For cleaning old source tarballs, use:

```
root # eclean-dist
```

For cleaning old binpkgs, use:

```
root # eclean-pkg
```

User administration

Adding a user for daily use

Working as root on a Unix/Linux system is dangerous and should be avoided as much as possible. Therefore it is strongly recommended to add one or more standard user account(s) for day-to-day use.

The groups the user is member of define what activities the user can perform. The following table lists a number of important groups:

Group	Description
audio	Enable the user account to access the audio devices.
cdrom	Enable the user account to directly access optical devices.
cron	Enable the user account to access time-based job scheduling via cron . Note: user accounts on systems running the systemd service system can use systemd timers and user service files instead of cron jobs.
floppy	Enable the user account to directly access ancient mechanical devices known as floppy drives. This group is not generally used on modern systems.
usb	Enable the user account able to access USB devices.
video	Enables the user account to access video capturing hardware and hardware acceleration.
wheel	Enables the user account able to use the su (<i>substitute user</i>) command, which allows switching to the root account or other accounts. For single user systems that include a root account, it is a good idea to add this group for the primary standard user.

For instance, to create a user called larry (/wiki/User:Larry) who is a member of the *wheel*, *users*, and *audio* groups, log in as root first (only root can create users) and run **useradd**:

Login: root

Password: (Enter the root password)

When setting passwords for standard user accounts, it is good security practice to avoid using the same or a similar password as set for the root user.

Handbook authors recommended to use a password at least 16 characters in length, with a value fully unique from every other user on the system.

```
root # useradd -m -G users,wheel,audio -s /bin/bash larry
```

```
root # passwd larry
```

Password: (Enter the password for larry)

Re-enter password: (Re-enter the password to verify)

Temporarily elevating privileges

If a user ever needs to perform some task as root, they can use **su** - to temporarily receive root privileges. Another way is to use the **sudo** (/wiki/Sudo)

(app-admin/sudo (<https://packages.gentoo.org/packages/app-admin/sudo>) ) or **doas** (/wiki/Doas) (app-admin/doas (<https://packages.gentoo.org/packages/app-admin/doas>) ) utilities which are, if correctly configured, very secure.

Disabling root login

Warning

Before disabling the root login, ensure that a user account is a member of the *wheel* group and that a method to elevate user privilege exists; otherwise root access will be locked and system administration will be impossible without performing recovery. Some common methods to elevate user privilege include:

app-admin/sudo (<https://packages.gentoo.org/packages/app-admin/sudo>) ,

app-admin/doas (<https://packages.gentoo.org/packages/app-admin/doas>) , or

systemd's run0.

To prevent possible threat actors from logging in as root, deleting the root password and/or disabling root login can help improve security.

To disable root login:

```
root # passwd -l root
```

To delete the root password and disable login:

```
root # passwd -dl root
```

Disk cleanup

Removing installation artifacts

With the Gentoo installation finished and the system rebooted, if everything has gone well, the stage file and other installation artifacts - such as DIGEST, CONTENT, or *.asc (PGP signature) files - can now be safely removed.

The files are located in the / directory and can be removed with the following command:

```
root # rm /stage3-*.tar.*
```

Where to go from here

Not sure where to go from here? There are many paths to explore... Gentoo provides its users with lots of possibilities and therefore has lots of documented (and less documented) features to explore here on the wiki and on other Gentoo related sub-domains (see the Gentoo online (/wiki/Handbook:AMD64/Installation/Finalizing#Gentoo_online) section below).

Additional documentation

It is important to note that, due to the number of choices available in Gentoo, the documentation provided by the handbook is limited in scope - it mainly focuses on the basics of getting a Gentoo system up and running and basic system management activities. The handbook intentionally excludes instructions on graphical environments, details on hardening, and other important administrative tasks. That being stated, there are more sections of the handbook to assist readers with more basic functions.

Readers should definitely take a look at the next part of the handbook entitled Working with Gentoo (/wiki/Handbook:AMD64/Working/Portage) which explains how to keep the software up to date, install additional software packages, details on USE flags, the OpenRC init system, and various other informative topics relating to managing a Gentoo system post-installation.

Apart from the handbook, readers should also feel encouraged to explore other corners of the Gentoo wiki to find additional, community-provided documentation. The Gentoo wiki team also offers a documentation topic overview (/wiki/Main_Page#Documentation_topics) which lists a selection of wiki articles by category. For instance, it refers to the localization guide (/wiki/Localization/Guide) to make a system feel more at home (particularly useful for users who speak English as a second language).

The majority of users with desktop use cases will setup graphical environments in which to work natively. There are many community maintained 'meta' articles for supported desktop environments (DEs) (/wiki/Desktop_environment) and window managers (WMs) (/wiki/Window_manager). Readers should be aware that each DE will require slightly different setup steps, which will lengthen add complexity to bootstrapping.

Many other Meta articles (/wiki/Category:Meta) exist to provide our readers with high level overviews of available software within Gentoo.

Gentoo online

❗ Important

Readers should note that all official Gentoo sites online are governed by Gentoo's code of conduct (/wiki/Project:Council/Code_of_conduct). Being active in the Gentoo community

is a privilege, not a right, and users should be aware that the code of conduct exists for a reason.

With the exception of the Libera.Chat hosted internet relay chat (IRC) network and the mailing lists, most Gentoo websites require an account on a per site basis in order to ask questions, open a discussion, or enter a bug.

Forums and IRC

Every user is welcome on our Gentoo forums (<https://forums.gentoo.org>) or on one of our internet relay chat channels (<https://www.gentoo.org/get-involved/irc-channels/>). It is easy to search for the forums to see if an issue experienced on a fresh Gentoo install has been discovered in the past and resolved after some feedback. The likelihood of other users experiencing the installation issues by first-time Gentoo can be surprising. It is advised users search the forums and the wiki before asking for assistance in Gentoo support channels.

Mailing lists

Several mailing lists (<https://www.gentoo.org/get-involved/ mailing-lists/>) are available to the community members who prefer to ask for support or feedback over email rather than create a user account on the forums or IRC. Users will need to follow the instructions in order to subscribe to specific mailing lists.

Bugs

Sometimes after reviewing the wiki, searching the forums, and seeking support in the IRC channel or mailing lists there is no known solution to a problem. Generally this is a sign to open a bug on Gentoo's Bugzilla site (<https://bugs.gentoo.org>).

Development guide

Readers who desire to learn more about developing Gentoo can take a look at the Development guide (<https://devmanual.gentoo.org/>). This guide provides instructions on writing ebuilds, working with eclasses, and provides definitions for many general concepts (<https://devmanual.gentoo.org/general-concepts/index.html>) behind Gentoo development.

Closing thoughts

Gentoo is a robust, flexible, and excellently maintained distribution. The developer community is happy to hear feedback on how to make Gentoo an even *better* distribution.

As a reminder, any feedback for *this handbook* should follow the guidelines detailed in the How do I improve the Handbook? (/wiki/Handbook:Main_Page#How_do_I_improve_the_Handbook.3F) section at the beginning of the handbook.

We look forward to seeing how our users will choose to implement Gentoo to fit their unique use cases and needs.

Retrieved from "<https://wiki.gentoo.org/index.php?title=Handbook:AMD64/Full/Installation&oldid=229552> (<https://wiki.gentoo.org/index.php?title=Handbook:AMD64/Full/Installation&oldid=229552>)"

- This page was last edited on 4 January 2015, at 15:16.
- [Privacy policy \(/wiki/Gentoo_Wiki:Privacy_policy\)](/wiki/Gentoo_Wiki:Privacy_policy)
- [About Gentoo Wiki \(/wiki/Gentoo_Wiki:About\)](/wiki/Gentoo_Wiki:About)
- [Disclaimers \(/wiki/Gentoo_Wiki:General_disclaimer\)](/wiki/Gentoo_Wiki:General_disclaimer)

© 2001–2026 Gentoo Authors

Gentoo is a trademark of the Gentoo Foundation, Inc. and of Förderverein Gentoo e.V. The contents of this document, unless otherwise expressly stated, are licensed under the [CC-BY-SA-4.0 \(https://creativecommons.org/licenses/by-sa/4.0/\)](https://creativecommons.org/licenses/by-sa/4.0/) license. The [Gentoo Name and Logo Usage Guidelines \(https://www.gentoo.org/inside-gentoo/foundation/name-logo-guidelines.html\)](https://www.gentoo.org/inside-gentoo/foundation/name-logo-guidelines.html) apply.
